

**PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL
ESCOLA POLITÉCNICA
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO**

**ALGORITMOS DE
CRIPTOGRAFIA: AVALIAÇÃO E
COMPARAÇÃO DE RECURSOS
PÓS-QUÂNTICOS EM
AMBIENTES COMPUTACIONAIS
DISTINTOS**

**ANDRÉ MENEZES BINS
MARCOS DO NASCIMENTO FERREIRA**

Trabalho de Conclusão II apresentado
como requisito parcial à obtenção
do grau de Bacharel em Ciência da
Computação na Pontifícia Universidade
Católica do Rio Grande do Sul.

Orientador: Prof. Edson Ifarraguirre Moreno

**Porto Alegre
2026**

LISTA DE FIGURAS

2.1	Representação de um esquema de assinatura digital [46]	16
2.2	Exemplo de curva elíptica	21
2.3	Processo de geração e verificação de assinatura digital no ECDSA [46] . . .	23
2.4	Exemplo de estrutura da <i>hypertree</i> do SPHINCS+ [5]	31
3.1	Diagrama de componentes da arquitetura	41
4.1	Comparação do pico de memória líquido do ciclo completo com mensagem de 256 bytes em ambas plataformas	55
4.2	Comparação do pico de memória líquido do ciclo completo com mensagem de 100 KB em ambas plataformas	57
4.3	Comparação do pico de memória líquido do ciclo completo com mensagem de 100 MB em ambas plataformas	59
4.4	Comparação do tempo líquido de geração de chaves em ambas plataformas	61
4.5	Comparação do tempo líquido de assinatura com mensagem de 256 bytes em ambas plataformas	63
4.6	Comparação do tempo líquido de assinatura com mensagem de 100 MB em ambas plataformas	65
4.7	Comparação do tempo líquido de verificação com mensagem de 256 bytes em ambas plataformas	67
4.8	Comparação do tempo líquido de verificação com mensagem de 100 MB em ambas plataformas	69
4.9	Comparação do tempo líquido do ciclo completo com mensagem de 256 bytes em ambas plataformas	71
4.10	Comparação do tempo líquido do ciclo completo com mensagem de 100 MB em ambas plataformas	73

LISTA DE TABELAS

2.1	Categorias de Segurança do NIST [24]	28
4.1	Algoritmos pós-quânticos avaliados e suas especificações [34] [29]	50

LISTA DE SIGLAS

- 3-DES – *Triple Data Encryption Standard* (DES triplo)
- AES – *Advanced Encryption Standard* (padrão de criptografia avançado)
- API – *Application Programming Interface* (Interface de Programação de Aplicações)
- CA – *Certificate Authority* (Autoridade Certificadora)
- CDHP – *Computational Diffie–Hellman Problem* (problema computacional de *Diffie–Hellman*)
- CPU – *Central Processing Unit* (unidade central de processamento)
- CSV – *Comma-Separated Values* (valores separados por vírgula)
- CVP – *Closest Vector Problem* (problema do vetor mais próximo)
- DES – *Data Encryption Standard* (padrão de criptografia de dados)
- DLP – *Discrete Logarithm Problem* (problema do logaritmo discreto)
- ECC – *Elliptic Curve Cryptography* (criptografia de curva elíptica)
- ECDH – *Elliptic Curve Diffie-Hellman* (Diffie-Hellman de curvas elípticas)
- ECDLP – *Elliptic Curve Discrete Logarithm* (logaritmo discreto de curvas elípticas)
- ECDSA – *Elliptic Curve Digital Signature Algorithm* (assinatura digital de curvas elípticas)
- FFT – *Fast Fourier Transform* (transformada rápida de Fourier)
- FTS – *Few Time Signatures* (assinaturas de poucos usos)
- GB – *Gigabytes*
- GNFS – *General Number Field Sieve*
- HTTPS – *Hypertext Transfer Protocol Secure* (protocolo de transferência de hipertexto seguro)
- I/O – *Input/Output* (entrada e saída)
- IOT – *Internet of Things* (Internet das Coisas)
- KB – *Kilobytes*

KEM – *Key Encapsulation Mechanism* (mecanismo de encapsulamento de chaves)

LWE – *Learning With Errors* (aprendizado com erros)

ML-DSA – *Module-Lattice-Based Digital Signature Algorithm*

M-LWE – *Module Learning With Errors* (aprendizado com erros em módulo)

MB – *Megabytes* (megabytes)

MSIS – *Module Short Integer Solution* (solução de inteiros curtos em módulo)

NIST – *National Institute of Standards and Technology* (Instituto Nacional de Padrões e Tecnologia)

NTT – *Number Theoretic Transform* (transformada numérica teórica)

NUMA – *Non-Uniform Memory Access* (acesso não uniforme à memória)

OTS – *One Time Signatures* (assinatura de uso único)

PQC – *Post-quantum Cryptography* (criptografia pós-quântica)

QFT – *Quantum Fourier Transform* (transformada quântica de Fourier)

RAM – *Random Access Memory* (memória de acesso aleatório)

RSA – *Rivest-Shamir-Adleman*

SHA – *Secure Hash Algorithm* (algoritmo de hash seguro)

TLS – *Transport Layer Security* (segurança da camada de transporte)

SUMÁRIO

1	INTRODUÇÃO	11
2	FUNDAMENTAÇÃO TEÓRICA	13
2.1	CRIPTOGRAFIA CLÁSSICA	13
2.1.1	CRIPTOGRAFIA SIMÉTRICA	14
2.1.2	CRIPTOGRAFIA ASSIMÉTRICA	14
2.2	COMPUTAÇÃO QUÂNTICA	24
2.2.1	ALGORITMO DE SHOR	24
2.3	CRIPTOGRAFIA PÓS-QUÂNTICA	26
2.3.1	CONTEXTUALIZAÇÃO	27
2.3.2	ALGORITMO FALCON	28
2.3.3	ALGORITMO ML-DSA (CRYSTALS-DILITHIUM)	29
2.3.4	ALGORITMO SPHINCS+	30
2.4	SISTEMAS EMBARCADOS	32
2.4.1	LIMITAÇÕES DE RECURSOS EM SISTEMAS EMBARCADOS	32
2.4.2	ASSINATURAS DIGITAIS EM SISTEMAS EMBARCADOS E IOT	34
2.4.3	IMPLICAÇÕES DAS RESTRIÇÕES PARA A CRIPTOGRAFIA PÓS-QUÂNTICA	35
3	FERRAMENTA DE BENCHMARK: ARQUITETURA E IMPLEMENTAÇÃO	37
3.1	VISÃO GERAL	37
3.2	TECNOLOGIAS E DEPENDÊNCIAS	38
3.2.1	DEPENDÊNCIAS CRIPTOGRÁFICAS	39
3.2.2	FRAMEWORK DE BENCHMARKING: BENCHEXEC	40
3.3	ARQUITETURA DO SISTEMA	40
3.3.1	VISÃO GERAL DOS COMPONENTES	41

3.3.2	COMPONENTES PRINCIPAIS	42
3.3.3	DECISÕES DE DESIGN	46
4	ANÁLISE DE DESEMPENHO DE ALGORITMOS DE ASSINATURA DIGITAL EM AMBIENTES COMPUTACIONAIS DISTINTOS	49
4.1	METODOLOGIA EXPERIMENTAL	49
4.1.1	ALGORITMOS AVALIADOS	49
4.1.2	AMBIENTES COMPUTACIONAIS	50
4.1.3	PARÂMETROS DE EXECUÇÃO E MÉTRICAS AVALIADAS	51
4.2	RESULTADOS	52
4.2.1	AVALIAÇÃO DE CONSUMO DE MEMÓRIA	52
4.2.2	AVALIAÇÃO DE TEMPO DE PROCESSAMENTO	58
4.3	CONSIDERAÇÕES FINAIS	72
5	CONCLUSÕES E TRABALHOS FUTUROS	77
5.1	CONTRIBUIÇÕES	78
5.2	OPORTUNIDADES DE TRABALHOS FUTUROS	79
	REFERÊNCIAS	81

1. INTRODUÇÃO

A criptografia é um campo fundamental que desempenha um papel vital na conquista dos principais objetivos da segurança da informação [36]. Seu objetivo é prover a privacidade da informação, de modo que pessoas autorizadas sejam capazes de compreender seu conteúdo, enquanto pessoas não autorizadas não tenham essa capacidade [36]. Com o avanço da tecnologia, a criptografia passou a ser essencial em diversas áreas, tais como finanças, armazenamento de dados e comunicação digital. Ela é usada, por exemplo, para proteger transações em sistemas de pagamento, resguardar dados sensíveis mesmo com acesso físico ao dispositivo onde estejam armazenados e preservar a privacidade em aplicativos de mensagens [3].

Atualmente, grande parte dos algoritmos criptográficos clássicos utilizados baseiam sua segurança na dificuldade computacional de resolver problemas matemáticos específicos, como a fatoração de grandes números inteiros [3]. Entretanto, com o advento da computação quântica, esse cenário pode mudar. A computação quântica utiliza princípios da mecânica quântica para realizar cálculos e resolver problemas complexos de forma significativamente mais rápida que os computadores convencionais [38]. Isso representa uma séria ameaça à criptografia clássica, especialmente aos algoritmos de criptografia assimétrica [3].

Diante desse desafio, surge a criptografia pós-quântica (*Post-quantum Cryptography*, PQC) [3] como uma extensão às propostas criptográficas anteriores. Essa abordagem explora algoritmos criptográficos que são resistentes a ataques de computadores quânticos. Seu objetivo é aprimorar a segurança dos dados em um futuro em que dispositivos quânticos estejam disponíveis para realizar tais ataques [3]. Isso torna a criptografia pós-quântica um campo de extrema relevância para a área de segurança da informação, visto que sua adoção é fundamental para a manutenção da privacidade e integridade dos dados na era da computação quântica [3].

Neste contexto, o presente trabalho realiza um estudo comparativo entre algoritmos de assinatura digital clássicos e pós-quânticos, com foco na avaliação de seu desempenho em diferentes ambientes computacionais. Os algoritmos são analisados a partir de métricas como tempo de processamento e consumo de memória, considerando também o nível de segurança associado a cada esquema. Para isso, foi desenvolvida uma ferramenta de *benchmarking* capaz de realizar medições reproduzíveis e comparáveis entre plataformas distintas. Os experimentos foram conduzidos em um computador de propósito geral e em um sistema embarcado, permitindo observar o comportamento dos algoritmos em cenários com diferentes capacidades de hardware. A partir dessa análise, buscou-se avaliar

a viabilidade técnica da adoção de soluções pós-quânticas em diferentes contextos computacionais, contribuindo para o entendimento de seu desempenho e aplicabilidade prática.

Este trabalho está organizado da seguinte forma: o Capítulo 2 apresenta os fundamentos teóricos necessários para compreensão do tema, incluindo conceitos de criptografia clássica, computação quântica, criptografia pós-quântica e sistemas embarcados. O Capítulo 3 descreve o desenvolvimento e a arquitetura da ferramenta de *benchmarking* utilizada para os experimentos. O Capítulo 4 apresenta e discute os resultados obtidos nas medições de desempenho dos algoritmos em diferentes plataformas. Por fim, o Capítulo 5 apresenta as conclusões do estudo e propõe direções para trabalhos futuros.

2. FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os conceitos fundamentais necessários para a compreensão do presente trabalho. Inicialmente, são abordados os princípios da criptografia clássica, com foco especial na criptografia assimétrica e nos mecanismos de assinatura digital, discutindo seus fundamentos, funcionamento e principais aplicações.

Em seguida, são explorados os fundamentos da computação quântica e os desafios que essa tecnologia impõe à segurança da informação, evidenciando o impacto do algoritmo de Shor sobre os sistemas criptográficos tradicionais. A partir desse contexto, é introduzida a criptografia pós-quântica (PQC), destacando suas principais famílias de algoritmos, as categorias de segurança definidas pelo *National Institute of Standards and Technology* (NIST) e os esquemas selecionados em seu processo de padronização.

Por fim, são abordados os conceitos de sistemas embarcados e suas restrições de recursos, contextualizando as implicações dessas características para a adoção prática de algoritmos de criptografia pós-quântica. Essa discussão é essencial para compreender os desafios técnicos e de desempenho envolvidos na implementação de assinaturas digitais em plataformas com recursos computacionais mais modestos, como dispositivos embarcados.

2.1 CRIPTOGRAFIA CLÁSSICA

Criptografia é o campo da segurança da informação que utiliza princípios matemáticos para alcançar o sigilo de mensagens [3]. Como dito anteriormente, o objetivo é prover uma forma de comunicação apenas entre pares autorizados [36]. Além disso, a criptografia dá suporte aos princípios fundamentais da segurança da informação, como privacidade, integridade, autenticação, não repúdio e verificação de identidade [3][46].

No campo da criptografia, a mensagem original é conhecida como texto claro e serve como entrada para o algoritmo de encriptação. Este transforma a mensagem original (texto claro) em recurso protegido (texto cifrado). Essa transformação ocorre por meio de substituições e operações definidas pelo algoritmo de criptografia escolhido [36][46]. O texto cifrado é um conjunto de dados, aparentemente aleatório e ininteligível, que pode ser revertido de volta a mensagem original por meio de um algoritmo de deciptação [46].

Os algoritmos de encriptação e deciptação recebem como entrada chaves criptográficas, que são valores independentes da mensagem e do algoritmo e que influenciam

diretamente no resultado desses algoritmos [46]. Essas chaves são usadas pelo algoritmo de encriptação nas transformações e substituições, fazendo com que sejam produzidas saídas diferentes dependendo da chave fornecida como entrada [46]. Para o algoritmo de deciptação deve ser fornecida uma chave compatível com a chave utilizada no processo de encriptação para obtenção correta do texto claro [36].

As chaves utilizadas nos algoritmos de encriptação e deciptação podem ser iguais ou diferentes, a depender do tipo de sistema criptográfico adotado. Chaves únicas levam ao conceito de criptografia simétrica, enquanto o uso de chaves distintas para encriptação e deciptação caracteriza a criptografia assimétrica. Estes conceitos são explorados a seguir.

2.1.1 CRIPTOGRAFIA SIMÉTRICA

Na criptografia simétrica, os processos de encriptação e deciptação são realizados utilizando uma única chave secreta [3]. A criptografia simétrica é muito utilizada para ocultar conteúdos de qualquer tamanho, como mensagens, arquivos e senhas [46].

Chaves elaboradas com comprimento suficiente e geradas com aleatoriedade adequada levam a esquemas mais seguros contra ataques de força bruta com os recursos computacionais disponíveis atualmente [3]. Entretanto, a principal limitação da criptografia simétrica está na necessidade de manter a chave em sigilo no momento em que ela precisa ser distribuída entre as partes envolvidas na comunicação [3]. Alguns exemplos de algoritmos de criptografia simétrica são o padrão de criptografia de dados (em inglês, *Data Encryption Standard*, DES), o DES triplo (*Triple Data Encryption Standard*, 3-DES) e o padrão de criptografia avançado (*Advanced Encryption Standard*, AES), sendo este último o padrão atualmente recomendado para uso seguro em aplicações modernas.

2.1.2 CRIPTOGRAFIA ASSIMÉTRICA

Na criptografia assimétrica, também conhecida como criptografia de chave pública, a encriptação e a deciptação utilizam chaves distintas, mas matematicamente relacionadas: uma chave pública, que pode ser distribuída livremente, e uma chave privada, que deve ser mantida em segredo [3].

O funcionamento desses sistemas se baseia em funções matemáticas complexas que têm como característica a existência de uma operação fácil de realizar em uma direção

(criptação), mas computacionalmente inviável de inverter sem a posse da outra chave correspondente (decriptação) [46].

Um dos possíveis usos da criptografia assimétrica é a encriptação de mensagens com o objetivo de garantir a confidencialidade da comunicação. Nesse cenário, a mensagem é encriptada com a chave pública do destinatário e só pode ser decriptada corretamente utilizando sua chave privada [46]. Esse modelo elimina a necessidade de compartilhamento prévio de chaves por meios seguros, uma limitação presente na criptografia simétrica, sendo útil para proteger dados em canais inseguros [23] [46].

Porém, na prática, esse uso direto é pouco comum, já que os algoritmos assimétricos apresentam um custo computacional significativamente maior do que os simétricos, tornando-os inadequados para a cifragem de grandes volumes de dados. Por isso, a criptografia assimétrica é frequentemente utilizada para realizar a troca segura de chaves, permitindo que duas partes estabeleçam uma chave secreta comum por meio de protocolos como o *Diffie-Hellman* [46]. Essa chave, então, pode ser usada em algoritmos simétricos para proteger comunicações subsequentes com maior eficiência [46].

Além da cifragem e da troca segura de chaves, a criptografia assimétrica também dá suporte aos mecanismos de assinatura digital, tema de especial relevância para este trabalho e detalhado na subseção 2.1.2.1.

Entre os principais algoritmos de criptografia assimétrica estão o *Rivest-Shamir-Adleman* (RSA), a criptografia de curva elíptica (*Elliptic Curve Cryptography*, ECC) e o protocolo de troca de chaves *Diffie-Hellman*, todos abordados nas subseções seguintes.

2.1.2.1 ASSINATURAS DIGITAIS

As assinaturas digitais representam uma das aplicações mais importantes da criptografia assimétrica, pois permitem que o remetente associe de forma segura sua identidade a uma mensagem por meio de uma assinatura gerada com sua chave privada. Qualquer destinatário pode, então, verificar a autenticidade dessa mensagem utilizando a chave pública correspondente [23, 46].

O processo de assinatura digital envolve duas etapas principais. Primeiro, é gerado um resumo criptográfico (*hash*) da mensagem, que condensa seu conteúdo em um valor de tamanho fixo e praticamente único. Em seguida, esse *hash* é cifrado com a chave privada do emissor, resultando na assinatura digital. Durante a verificação, o destinatário recalcula o *hash* da mensagem recebida e o compara com o valor obtido ao decifrar a assinatura com a chave pública do remetente. Se ambos coincidirem, a assinatura é considerada válida [3, 46].

Esse mecanismo garante três propriedades fundamentais da segurança da informação [28]. A autenticidade assegura que a assinatura foi realmente produzida por quem detém a chave privada correspondente, permitindo confirmar a origem da mensagem e a identidade do remetente. A integridade garante que o conteúdo assinado não foi alterado após a assinatura, pois qualquer modificação, mesmo mínima, resultaria em um *hash* diferente durante a verificação. Por fim, o não-repúdio impede que o autor negue posteriormente ter realizado a assinatura, já que apenas ele possui a chave privada capaz de gerá-la, fornecendo uma evidência criptográfica de autoria e responsabilidade [28].

A Figura 2.1 ilustra o funcionamento geral do processo de assinatura digital, desde a geração do *hash* até a verificação por meio da chave pública [46]. À esquerda, o emissor Bob calcula o *hash* da mensagem M e o cifra com sua chave privada, gerando a assinatura S . À direita, a receptora Alice recebe a mensagem M e a assinatura S , calcula novamente o *hash* da mensagem recebida e decifra a assinatura com a chave pública de Bob, obtendo os valores h e h' , respectivamente. Por fim, os dois valores de *hash* são comparados: se forem idênticos, a assinatura é considerada válida, garantindo que a mensagem não foi alterada e que foi realmente enviada por Bob.

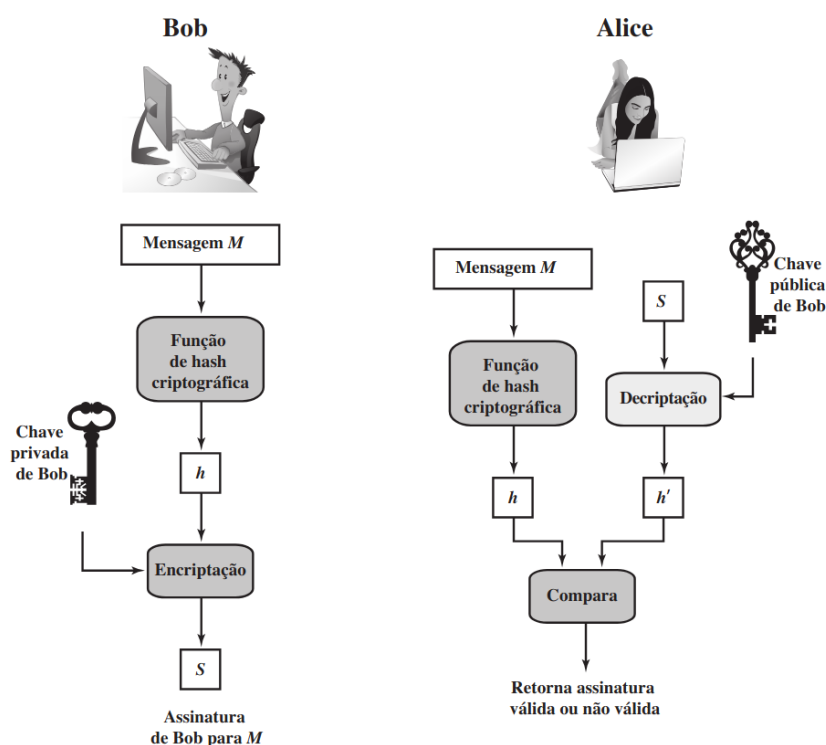


Figura 2.1: Representação de um esquema de assinatura digital [46]

Além do funcionamento conceitual, as assinaturas digitais possuem ampla aplicação prática em diferentes contextos tecnológicos, conforme discutido a seguir.

Aplicações das Assinaturas Digitais

As assinaturas digitais são amplamente utilizadas para garantir autenticidade, integridade e não-repúdio em ambientes digitais. Entre suas diversas aplicações, destacam-se exemplos como os protocolos de comunicação segura, a validação de documentos eletrônicos, a distribuição de software e as tecnologias baseadas em *blockchain*, conforme descrito a seguir.

- Comunicações seguras via TLS — Um dos principais usos das assinaturas digitais está nos protocolos de comunicação segura, como o *Transport Layer Security* (TLS), utilizado para proteger conexões web por meio do *Hypertext Transfer Protocol Secure* (HTTPS). Nesse processo, o servidor apresenta ao cliente um certificado digital assinado por uma Autoridade Certificadora (CA), o que permite verificar a autenticidade da chave pública. Durante o *handshake*, servidor e cliente também trocam assinaturas digitais que são verificadas pela outra parte, garantindo que ambos são legítimos. Assim, as assinaturas digitais asseguram a identidade dos participantes e permitem a criação de um canal criptografado confiável para a troca de dados [14].
- Documentos e contratos digitais — As assinaturas digitais são amplamente empregadas para conferir validade jurídica a documentos eletrônicos e contratos digitais. Elas asseguram autenticidade, integridade e não-repúdio, permitindo que acordos eletrônicos tenham o mesmo efeito legal de documentos assinados em papel. Em diversas jurisdições, legislações específicas estabelecem que documentos assinados digitalmente são juridicamente válidos e admissíveis como prova, incluindo setores como comércio eletrônico e governo eletrônico [22].
- Atualizações de software e distribuição de código — As assinaturas digitais também são fundamentais na distribuição de *software*, pois permitem verificar a integridade e a origem de programas antes de sua instalação. O processo de *code signing* consiste em assinar digitalmente aplicativos, atualizações e *firmwares*, de forma que os dispositivos possam confirmar que o código realmente veio do fornecedor autorizado e não foi alterado [9]. Esse mecanismo é amplamente utilizado em sistemas operacionais, navegadores, *drivers* e lojas de aplicativos, sendo uma das principais formas de proteção contra *softwares* maliciosos distribuídos como se fossem legítimos.
- Criptomoedas e blockchain — No contexto do *blockchain*, tecnologia que sustenta o *Bitcoin* e outras criptomoedas, as assinaturas digitais são fundamentais para garantir a segurança das transações. Elas fornecem não-repúdio, impedindo que um participante negue a autoria de uma operação já registrada, e asseguram a autenticidade e integridade dos dados armazenados no livro-razão distribuído. O uso de assinaturas

digitais permite que apenas o detentor da chave privada correspondente possa autorizar a movimentação de fundos, enquanto qualquer participante da rede pode verificar a transação por meio da chave pública. Assim, assinaturas digitais não apenas evitam fraudes e falsificações, mas também fornecem a base de confiança que torna possível o funcionamento descentralizado de sistemas de criptomoedas [12].

A segurança e a eficiência desses mecanismos de assinatura dependem diretamente das propriedades matemáticas dos algoritmos de criptografia assimétrica utilizados como base. Entre os mais empregados estão o RSA e a criptografia de curva elíptica (ECC), apresentados nas subseções seguintes.

2.1.2.2 RSA

O RSA é um algoritmo de criptografia assimétrica criado em 1978 que implementa duas ideias muito importantes na criptografia, a criptografia de chave pública e as assinaturas digitais [23]. O RSA se fundamenta no problema da fatoração de grandes números primos e na aritmética modular. Para a geração do par de chaves, são seguidos os seguintes passos [23]:

1. Escolha dois grandes números primos p e q aleatórios
2. Calcule $n = p \times q$
3. Calcule $\varphi(n) = (p - 1) \times (q - 1)$
4. Escolha um inteiro d tal que $1 < d < \varphi(n)$ e que seja coprimo a $\varphi(n)$, satisfazendo a seguinte equação: $\text{mdc}(d, \varphi(n)) = 1$ (onde mdc é máximo divisor comum dos dois números)
5. Calcule e tal que $(e \times d) \pmod{\varphi(n)} = 1$ ou $e \times d \equiv 1 \pmod{\varphi(n)}$

A chave pública é formada pelo par de inteiros (e, n) enquanto a chave privada é formada por (d, n) [23]. A operação de encriptação E consiste em elevar o texto claro M ao expoente e módulo n , resultando no texto cifrado C :

$$C \equiv E(M) \equiv M^e \pmod{n}$$

Na operação de decifração D , o texto cifrado C é elevado ao expoente d módulo n :

$$M \equiv D(C) \equiv C^d \pmod{n}$$

Uma observação importante é que a mensagem M deve ser menor que o módulo n . Abaixo, um exemplo será mostrado para um melhor entendimento do algoritmo. Primeiro serão geradas as chaves:

1. $p = 2 ; q = 7$
2. $n = 2 \times 7 = 14$
3. $\varphi(n) = (2 - 1) \times (7 - 1) = 1 \times 6 = 6$
4. $d = 5$; pois $1 < 5 < 6$ e $\text{mdc}(5, 6) = 1$
5. $e = 11$; pois $11 \times 5 \pmod{6} = 55 \pmod{6} = 1$

Logo, a chave pública é o par de inteiros $(11, 14)$ e a chave privada é $(5, 14)$. Agora, vamos encriptar o texto claro $M = 2$ utilizando a chave pública $(11, 14)$:

$$C = E(2) = 2^{11} \pmod{14} = 2048 \pmod{14} = 4$$

Portanto, o texto cifrado C é igual a 4. Agora vamos decifrar o texto cifrado usando a chave privada $(5, 14)$:

$$M = D(4) = 4^5 \pmod{14} = 1024 \pmod{14} = 2$$

Podemos observar que ao decriptar o texto cifrado, conseguimos voltar ao texto claro, que era 2. Uma observação a ser feita é de que nesse exemplo foram usados números muito pequenos para p e q . Em um uso real do algoritmo, seriam utilizados números muito maiores, visto que a segurança do RSA está baseada na dificuldade do problema de fatoração de grandes números inteiros [23].

Embora n seja conhecido publicamente, sua fatoração em p e q , que permitiria calcular $\varphi(n)$ e, conseqüentemente, d , é considerada computacionalmente inviável quando p e q são suficientemente grandes [23]. Nenhum algoritmo clássico eficiente conhecido consegue realizar a fatoração de números com mais de dois mil bits em tempo viável, o que garante a segurança prática do RSA contra algoritmos clássicos até hoje [3][23]. Além disso, mesmo métodos alternativos de se obter d , como o cálculo de $\varphi(n)$ diretamente ou a obtenção de raízes modulares, foram demonstrados como tão difíceis quanto fatorar n , o que reforça a robustez do sistema [23]. Contudo, com o advento da computação quântica, que será explorada mais à frente, essa segurança pode estar em risco.

2.1.2.3 TROCA DE CHAVES DIFFIE-HELLMAN

A troca de chaves *Diffie-Hellman* é considerada uma das bases da criptografia de chave pública [44]. A finalidade do algoritmo é permitir que dois usuários troquem uma chave simétrica com segurança, que pode, então, ser usada para a criptografia subsequente das mensagens [46]. A segurança do algoritmo *Diffie-Hellman* baseia-se na dificuldade de se calcular logaritmos discretos [46] .

O protocolo de troca de chaves *Diffie-Hellman* tem como objetivo permitir que duas partes, tradicionalmente denominadas Alice e Bob, estabeleçam um segredo compartilhado sobre um canal inseguro, sem que terceiros possam derivar tal segredo a partir das informações públicas trocadas [44].

Para isso, são definidos dois parâmetros públicos:

- Um número primo p
- Um gerador do subgrupo multiplicativo do corpo finito F_p^* , isto é, g é um gerador de ordem $p - 1$

Cada participante escolhe um valor secreto aleatório:

- Alice escolhe $a \in \mathbb{Z}_{p-1}$
- Bob escolhe $b \in \mathbb{Z}_{p-1}$

Com base nesses valores secretos, as chaves públicas são computadas da seguinte forma:

- Alice computa $A = g^a \bmod p$ e envia esse valor a Bob,
- Bob computa $B = g^b \bmod p$ e envia esse valor a Alice.

A partir das chaves públicas recebidas e de seus segredos, ambos os participantes podem calcular o mesmo valor compartilhado:

- Alice calcula $S = B^a \bmod p = (g^b)^a = g^{ab} \bmod p$
- Bob calcula $S = A^b \bmod p = (g^a)^b = g^{ab} \bmod p$

Dessa forma, ambos obtêm o mesmo valor $S = g^{ab} \bmod p$, que pode ser utilizado como chave simétrica para cifragem de mensagens subsequentes.

A segurança do protocolo reside na dificuldade computacional do problema do logaritmo discreto (*Discrete Logarithm Problem*, DLP) e, mais diretamente, do problema computacional de *Diffie–Hellman* (*Computational Diffie–Hellman Problem*, CDHP), que consiste em calcular $g^{ab} \bmod p$ conhecendo apenas g , g^a e g^b sem acesso a a ou b [44].

2.1.2.4 CRIPTOGRAFIA DE CURVA ELÍPTICA

A criptografia de curva elíptica (*Elliptic Curve Cryptography*, ECC) consiste em uma abordagem de criptografia assimétrica baseada em curvas elípticas sobre campos finitos [15]. Uma curva elíptica é definida por uma equação cúbica em duas variáveis, com coeficientes em um corpo finito[46]. Um exemplo de curva elíptica pode ser visualizado na Figura 2.2.

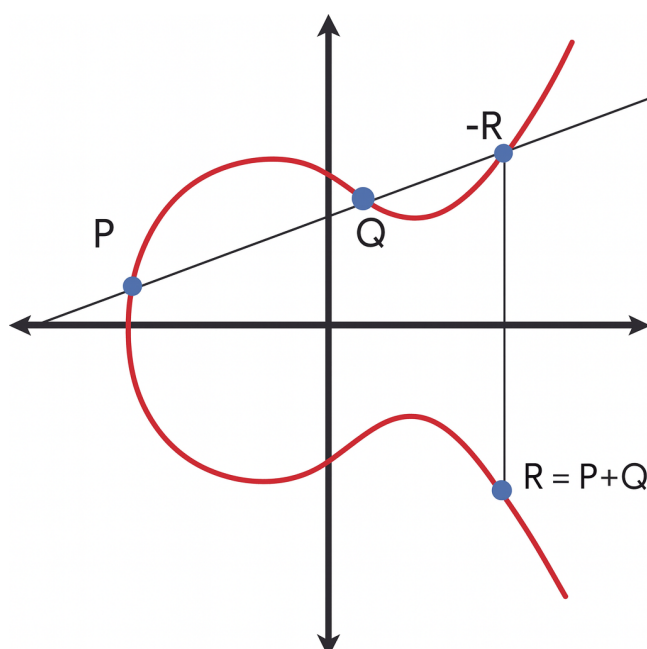


Figura 2.2: Exemplo de curva elíptica

Em geral, a forma reduzida da equação de uma curva elíptica segue a forma a seguir:

$$y^2 = x^3 + ax + b$$

O conjunto de pontos de uma curva elíptica forma um grupo abeliano finito, onde a operação de grupo é a adição de pontos. Dado dois pontos diferentes P e Q na curva, é possível calcular $R = P + Q$, sendo R um ponto pertencente à curva, como pode ser visto na Figura 2.2. Para a criptografia na curva elíptica, é utilizada uma operação de multiplicação escalar, que consiste na multiplicação escalar de um ponto P por um inteiro k [15][46]:

$$kP = \underbrace{P + P + \dots + P}_{k \text{ vezes}}$$

Essa operação é fundamental para a aplicação criptográfica da ECC, pois é ela que define a segurança no algoritmo tendo em vista que é fácil calcular aP conhecendo a e P , mas extremamente difícil obter o a a partir do P e do aP , o que constitui o problema do logaritmo discreto em curvas elípticas (*Elliptic Curve Discrete Logarithm*, ECDLP). A segurança do ECC portanto se baseia no ECDLP, um problema considerado computacionalmente intratável com os algoritmos conhecidos atualmente.

Suponha que Alice e Bob desejem se comunicar de forma segura usando criptografia de curva elíptica. Para isso, todos concordam publicamente com:

- uma curva elíptica definida sobre um corpo finito;
- um ponto fixo F da curva, também público.

Cada participante escolhe um número inteiro aleatório secreto como chave privada. Por exemplo, Alice escolhe A_k e calcula sua chave pública como:

$$A_P = A_k F$$

Bob faz o mesmo, publicando:

$$B_P = B_k F$$

Se Alice deseja enviar uma mensagem para Bob, ela pode calcular a chave secreta compartilhada da seguinte forma:

$$K = A_k B_P = A_k (B_k F) = B_k (A_k F) = B_k A_P$$

Ou seja, graças à comutatividade dos grupos abelianos, dos quais os pontos da curva elíptica pertencem, ambos calculam o mesmo ponto na curva, mesmo sem revelarem suas chaves privadas. Esse ponto pode então ser usado como base para uma chave simétrica (por exemplo, para cifrar com AES ou DES) [15]. Esse procedimento corresponde ao protocolo *Elliptic Curve Diffie–Hellman* (ECDH), uma variação do clássico *Diffie–Hellman* adaptada para curvas elípticas [44].

Além do uso em troca de chaves, a criptografia de curva elíptica também desempenha um papel central nos esquemas de assinatura digital modernos. O *Elliptic Curve Digital Signature Algorithm* (ECDSA) segue a mesma lógica geral das outras assinaturas digitais

tradicionais, mas executa suas operações sobre pontos de uma curva elíptica definida em um corpo finito. O esquema utiliza parâmetros públicos, como os coeficientes da curva, um ponto base, sua ordem e um número primo, a partir dos quais cada assinante gera um par de chaves: uma privada, mantida em segredo, e uma pública, derivada matematicamente da primeira [46].

Durante o processo de assinatura, o emissor combina sua chave privada com um número aleatório e com o valor de *hash* da mensagem para gerar dois valores inteiros que compõem a assinatura digital. Na verificação, o destinatário utiliza a chave pública e os mesmos parâmetros da curva para confirmar se esses valores correspondem corretamente à mensagem recebida, validando assim a assinatura [46].

A Figura 2.3 apresenta uma visão mais detalhada do processo de geração e verificação de assinaturas no ECDSA [46]. Nela, q representa um número primo, a e b são os coeficientes da curva elíptica, G é o ponto base da curva e n é a ordem desse ponto.

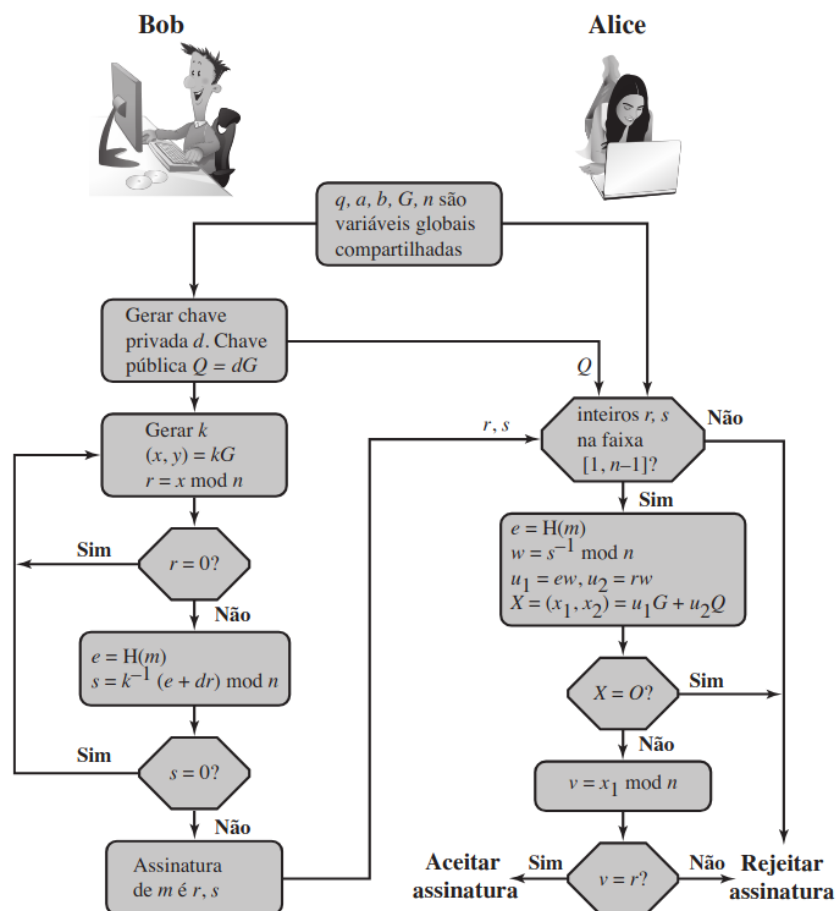


Figura 2.3: Processo de geração e verificação de assinatura digital no ECDSA [46]

O ECDSA, padrão definido pelo NIST, é amplamente adotado devido à eficiência da criptografia de curva elíptica, que proporciona segurança comparável à de outros esquemas, como o RSA, utilizando chaves de tamanho significativamente menor [46].

2.2 COMPUTAÇÃO QUÂNTICA

A computação quântica é um novo paradigma computacional fundamentado nos princípios da mecânica quântica, como superposição, emaranhamento e interferência [38].

Diferentemente dos computadores tradicionais, que processam informações com base em dados representados de forma binária, os computadores quânticos operam com qubits, que podem assumir combinações lineares de estados graças ao fenômeno de superposição [38]. Os algoritmos executados em computadores quânticos são chamados de algoritmos quânticos [25]. Esses algoritmos são capazes de resolver determinados problemas que são considerados computacionalmente inviáveis para os computadores convencionais com os algoritmos atualmente conhecidos, como o problema da fatoração de grandes inteiros e o problema do logaritmo discreto [3, 38]. Como esses problemas constituem a base da segurança de muitos sistemas criptográficos atuais, os computadores quânticos e seus algoritmos representam uma séria ameaça à criptografia clássica, especialmente à criptografia assimétrica [3, 38].

Alguns dos algoritmos quânticos mais conhecidos são os algoritmos de *Shor*, *Simon* e *Grover* [25]. O algoritmo quântico de *Shor*, em particular, foi desenvolvido para resolver o problema da fatoração de grandes inteiros em tempo polinomial, oferecendo uma vantagem exponencial em relação aos melhores algoritmos clássicos conhecidos para essa tarefa [38].

2.2.1 ALGORITMO DE SHOR

O algoritmo de *Shor* é um dos marcos mais significativos da computação quântica, pois propõe uma forma eficiente de resolver o problema da fatoração de inteiros, que constitui a base da segurança de muitos sistemas criptográficos, como o RSA. Desenvolvido por *Peter Shor* em 1994, o algoritmo explora propriedades fundamentais da mecânica quântica, como superposição e emaranhamento, para acelerar exponencialmente o processo de fatoração, alcançando uma complexidade de tempo aproximadamente $O((\log N)^3)$, em

contraste com os algoritmos clássicos mais eficientes, como o *General Number Field Sieve* (GNFS), que possuem complexidade subexponencial [42].

O funcionamento do algoritmo de Shor pode ser dividido em duas partes principais: uma parte clássica e uma parte quântica. Na etapa clássica, escolhe-se aleatoriamente um número $a < N$ tal que $\text{mdc}(a, N) = 1$. Em seguida, a parte quântica é responsável por determinar o menor período r tal que $a^r \equiv 1 \pmod{N}$, chamado de ordem de a . Essa etapa é realizada por meio da Transformada Quântica de Fourier (*Quantum Fourier Transform*, QFT), aplicada sobre um registrador quântico que armazena uma superposição de estados baseados nos valores de $a^x \pmod{N}$ [30].

Uma vez obtido r , se r for par e $a^{r/2} \not\equiv -1 \pmod{N}$, então $\text{mdc}(a^{r/2} \pm 1, N)$ revela um fator não trivial de N . O componente mais custoso da parte quântica é a implementação da exponenciação modular em superposição, que exige a decomposição do processo em subcircuitos aritméticos reversíveis, como somadores e multiplicadores quânticos [30].

Além da fatoração de inteiros, o algoritmo de Shor pode ser generalizado para resolver o problema do logaritmo discreto, que fundamenta a segurança de criptossistemas como o Diffie–Hellman e a ECC. Nessa variante, aplicada a grupos de pontos de curvas elípticas, são utilizados registradores em superposição para representar combinações lineares de pontos da curva da forma $|x\rangle|P + xQ\rangle$, em que P e Q são pontos conhecidos e relacionados pelo logaritmo discreto que se deseja determinar. Após aplicar a QFT apropriada e medir os registradores, obtêm-se informações suficientes para, via pós-processamento clássico, recuperar o logaritmo discreto [35]. Essa abordagem tem sido especialmente estudada para curvas definidas sobre campos binários $\mathbb{F}_{2^n}$, representando uma ameaça concreta à segurança de algoritmos baseados em curvas elípticas.

A seguir, apresenta-se um exemplo simplificado de execução do algoritmo com $N = 15$, cujos fatores primos são 3 e 5:

1. Escolha de um número $a < N$ tal que $\text{mdc}(a, N) = 1$:

Escolhemos $a = 2$. Como $\text{mdc}(2, 15) = 1$, podemos prosseguir com o algoritmo.

2. Parte quântica: determinar o período r tal que $a^r \equiv 1 \pmod{N}$:

Utiliza-se um computador quântico para encontrar o menor inteiro r tal que:

$$2^r \pmod{15} = 1.$$

Para este exemplo simples, mesmo com cálculo manual, temos:

$$2^1 \bmod 15 = 2,$$

$$2^2 \bmod 15 = 4,$$

$$2^3 \bmod 15 = 8,$$

$$2^4 \bmod 15 = 1.$$

Assim, o período é $r = 4$.

3. Cálculo dos fatores de N :

Com $r = 4$, calculamos:

$$a^{r/2} = 2^2 = 4,$$

e:

$$\text{mdc}(4 - 1, 15) = \text{mdc}(3, 15) = 3, \text{mdc}(4 + 1, 15) = \text{mdc}(5, 15) = 5.$$

Portanto, os fatores de $N = 15$ são 3 e 5.

Este exemplo ilustra o poder do algoritmo de Shor na fatoração de inteiros e evidencia o papel crucial da etapa quântica. Enquanto algoritmos clássicos possuem complexidade subexponencial para esse problema, a versão quântica o resolve com complexidade polinomial, comprometendo a segurança de sistemas criptográficos baseados na dificuldade de fatoração, como o RSA, em um cenário com computadores quânticos suficientemente avançados.

2.3 CRIPTOGRAFIA PÓS-QUÂNTICA

A criptografia pós-quântica (PQC) é o ramo da criptografia que busca desenvolver algoritmos de criptografia capazes de resistir a ataques realizados por adversários que utilizam computadores quânticos [3]. Isso se torna necessário, pois os algoritmos assimétricos mais utilizados atualmente têm sua segurança baseada em problemas matemáticos complexos, tal como a fatoração de grandes números primos e o problema do logaritmo discreto [3]. Como já mencionado, esses problemas podem ser resolvidos de forma eficiente por algoritmos quânticos, representando uma grande ameaça à criptografia clássica [3]. Por outro lado, algoritmos simétricos são menos afetados por ataques quânticos, já que o algoritmo de Grover apenas reduz pela metade a segurança efetiva, o que pode ser compensado com chaves maiores [3].

Diante dessas limitações dos sistemas assimétricos clássicos, surgem diversas famílias de algoritmos de criptografia pós-quântica, cada uma baseada em diferentes estru-

turas matemáticas. Entre as mais destacadas, está a criptografia baseada em reticulados, que se fundamenta em problemas difíceis envolvendo essas estruturas, que são grades regulares de pontos no espaço [3]. Também se destaca a criptografia baseada em funções *hash*, que utiliza dois princípios básicos dessas funções, a resistência a colisões e a pré-imagem [3]. Além disso, há abordagens baseadas em códigos corretores de erros, equações multivariadas e estruturas não comutativas, que oferecem alternativas para a construção de sistemas criptográficos resistentes a ataques quânticos [3].

2.3.1 CONTEXTUALIZAÇÃO

O *National Institute of Standards and Technology* (NIST) é o principal órgão responsável pela definição de padrões criptográficos nos Estados Unidos, com ampla influência global na área de segurança da informação [27]. O NIST lidera um processo internacional de seleção e padronização de algoritmos de criptografia pós-quântica, com o objetivo de substituir algoritmos clássicos vulneráveis a ataques quânticos [24][34].

Em julho de 2022, o NIST anunciou os primeiros algoritmos selecionados para padronização no contexto da criptografia pós-quântica [26]. Entre eles, está o CRYSTALS-Kyber, um algoritmo baseado em reticulados, destinado à criptografia geral. Para assinaturas digitais, foram selecionados três algoritmos: o ML-DSA e o Falcon, ambos também baseados em reticulados, e o SPHINCS+, que utiliza funções *hash* como base.

Além de selecionar os algoritmos candidatos à padronização, o NIST também propõe uma classificação em categorias de segurança, que orientam sua aplicação conforme o nível de proteção exigido. Essas categorias são baseadas na força computacional necessária para comprometer um esquema criptográfico, medida em termos do número estimado de operações que um atacante precisaria realizar para quebrá-lo [24].

Tradicionalmente, esse conceito era expresso em bits de segurança. Por exemplo, um algoritmo possui 128 bits de segurança se são necessárias aproximadamente 2^{128} operações para comprometê-lo [24]. Porém, dada a incerteza em estimar com precisão a capacidade dos futuros computadores quânticos, o NIST começou a utilizar uma abordagem baseada em categorias de segurança para os algoritmos de criptografia pós-quântica [24].

Essas categorias são definidas com base na equivalência com algoritmos simétricos conhecidos, como AES e SHA [24]. Por exemplo, a Categoria 1 equivale à segurança de um ataque por busca de chave em uma cifra de bloco de 128 bits (AES-128), enquanto a Categoria 5 representa o nível mais alto, comparável ao AES-256 [24].

Essas categorias servem como referência para orientar o uso prático dos algoritmos pós-quânticos, de acordo com a sensibilidade dos dados e o tempo pelo qual precisam permanecer protegidos [24]. Todas as categorias e suas equivalências podem ser visualizadas na Tabela 2.1.

Categoria	Tipo de Ataque	Exemplo
1	Busca de chave em um bloco cifrado com chave de 128 bits	AES-128
2	Busca por colisão em uma função hash de 256 bits	SHA-256
3	Busca de chave em um bloco cifrado com chave de 192 bits	AES-192
4	Busca por colisão em uma função hash de 384 bits	SHA-384
5	Busca de chave em um bloco cifrado com chave de 256 bits	AES-256

Tabela 2.1: Categorias de Segurança do NIST [24]

Paralelamente ao processo de padronização de algoritmos de criptografia pós-quântica, o NIST estabeleceu um cronograma de transição para a descontinuação dos algoritmos clássicos de assinatura digital. De acordo com as diretrizes publicadas, algoritmos como RSA e ECDSA serão considerados *deprecated* a partir de 2030 e *disallowed* a partir de 2035 para fins de assinatura digital [24]. No contexto das recomendações do NIST, o termo *deprecated* indica que um algoritmo ainda pode ser utilizado, mas passa a apresentar risco de segurança reconhecido. Já o status *disallowed* significa que o algoritmo deixa de ser aprovado para o propósito declarado, não sendo mais permitido seu uso em conformidade com os perfis e recomendações do NIST.

O relatório também enfatiza a urgência dessa transição diante do modelo de ameaça conhecido como *harvest now, decrypt later*, no qual agentes mal-intencionados podem coletar hoje dados cifrados com algoritmos vulneráveis, visando decifrá-los futuramente quando computadores quânticos se tornarem viáveis. Esse cenário reforça a importância da transição para soluções de criptografia pós-quântica, alinhadas com as diretrizes estabelecidas pelo NIST.

2.3.2 ALGORITMO FALCON

O algoritmo Falcon é um dos finalistas da padronização de criptografia pós-quântica (PQC) promovida pelo NIST, sendo baseado em estruturas de reticulado do tipo NTRU. Sua segurança decorre da dificuldade de resolver equações em anéis de polinômios, especificamente problemas associados ao reticulado NTRU. Diferente de outras abordagens baseadas em *Learning With Errors* (LWE), Falcon utiliza operações polinomiais tanto no domínio dos números complexos quanto no domínio dos inteiros. A operação central do Falcon é a multiplicação polinomial, que é realizada de forma eficiente por meio das transformadas

de Fourier (*Fast Fourier Transform*, FFT) e das transformadas numéricas teóricas (*Number Theoretic Transform*, NTT), dependendo do tipo de coeficiente envolvido. Essas multiplicações são cruciais em várias etapas do algoritmo, como na geração de chaves e no processo de assinatura, e são responsáveis por grande parte da carga computacional do esquema [16].

O processo de geração de chaves no Falcon é particularmente sofisticado, consistindo na resolução de uma equação NTRU para produzir os polinômios secretos f, g, F, G que satisfazem certas condições algébricas. Essa etapa utiliza extensivamente FFT/NTT para resolver a equação de forma eficiente, e os resultados intermediários são reutilizados para economizar recursos. Em seguida, o algoritmo constrói uma árvore LDL* (decomposição de Cholesky modificada) para viabilizar a amostragem por *trapdoor*, fundamental para gerar assinaturas seguras. A assinatura, por sua vez, é produzida através do algoritmo *ffSampling*, que realiza uma aproximação do vetor mais próximo (*Closest Vector Problem*, CVP) com ajuda de representações em baixa dimensão que se alinham com as transformações FFT aplicadas previamente. A verificação é relativamente simples: consiste em recalcular valores e verificar se a norma dos vetores resultantes está dentro de um limite estabelecido [16] [40].

2.3.3 ALGORITMO ML-DSA (CRYSTALS-DILITHIUM)

O ML-DSA (*Module-Lattice-Based Digital Signature Algorithm*) é a versão padronizada pelo NIST do algoritmo CRYSTALS-Dilithium [29]. Trata-se de um esquema de assinatura digital baseado em reticulados (*lattices*), cuja segurança decorre da dificuldade dos problemas MLWE (*Module Learning With Errors*) e MSIS (*Module Short Integer Solution*) [37][29], que são versões modulares dos problemas clássicos de reticulados, considerados difíceis até mesmo para adversários quânticos. Embora a proposta original tenha sido publicada sob o nome CRYSTALS-Dilithium [11][29], este trabalho adota a nomenclatura padronizada ML-DSA, refletindo a versão oficialmente aprovada e utilizada nas implementações avaliadas.

Para gerar as chaves, o esquema escolhe vetores secretos pequenos s_1 e s_2 amostrados de forma uniforme (evitando a amostragem gaussiana), calcula uma matriz pública A a partir de uma *seed* ρ usando SHAKE-128 e, a partir dela, computa $t = As_1 + s_2$. O vetor t junto com a matriz A formam a chave pública expandida [11][29].

No processo de assinatura, o assinante gera um vetor aleatório y , calcula um compromisso $w = Ay$ e constrói um desafio $c = H(w, \mu)$, sendo μ o hash da mensagem. Esse desafio é então usado junto com o segredo para computar $z = cs_1 + y$. Para evitar vaza-

mento de informações sobre a chave secreta, é realizada uma amostragem por rejeição: se z ficar fora de determinados limites, a tentativa de assinatura é descartada e um novo y é gerado [11][29].

Na verificação, o verificador usa componentes incluídos na assinatura para reconstruir w , recalcula o hash para obter o desafio e então verifica se z está dentro da norma permitida e se a relação matemática definida foi satisfeita[29].

2.3.4 ALGORITMO SPHINCS+

O SPHINCS+ é um esquema de assinatura digital baseado exclusivamente em funções *hash*, desenvolvido para oferecer segurança mesmo diante de ataques de computadores quânticos [5]. Ele representa uma evolução do SPHINCS original [4], com melhorias significativas em desempenho e tamanho de assinatura, sendo uma das propostas selecionadas no processo de padronização da NIST para algoritmos de assinatura pós-quânticos.

Uma das principais características do SPHINCS+ é ser um esquema *stateless*, ou seja, que não exige o gerenciamento de estado entre assinaturas consecutivas [5]. Em contraste, esquemas baseados em hash tradicionais, como os que utilizam assinaturas de uso único (*One Time Signatures*, OTS), exigem o controle das chaves já utilizadas para evitar sua reutilização pois isso comprometeria a segurança. Essa exigência torna-se problemática em ambientes distribuídos ou com backups, onde existem inconsistências [5]. No SPHINCS+, cada assinatura é derivada de forma determinística a partir de uma semente secreta e de um índice computado com base na mensagem, o que, aliado ao uso de uma grande estrutura hierárquica e esquemas de poucas utilizações, garante a não reutilização de chaves sem depender de controle de estado [5].

O SPHINCS+ organiza seu processo de assinatura por meio de uma estrutura hierárquica composta por múltiplas árvores de Merkle [5]. Nessas árvores, as folhas podem conter dois tipos de esquemas: assinaturas de uso único (OTS) e assinaturas de poucas utilizações (*Few Time Signatures*, FTS) [5]. A raiz da árvore mais alta representa a chave pública do esquema. Nas camadas intermediárias, as folhas são esquemas OTS usados para assinar as raízes das árvores do nível inferior, formando uma estrutura conhecida como *hypertree* [5]. Já nas árvores do nível mais baixo, as folhas são esquemas FTS responsáveis por assinar diretamente o resumo da mensagem (*hash* da mensagem) [5]. A estrutura geral do esquema pode ser visualizada na Figura 2.4.

Durante a fase de geração de chaves, basta computar a árvore superior para obter sua raiz, que será divulgada como chave pública [7]. No momento da assinatura, uma

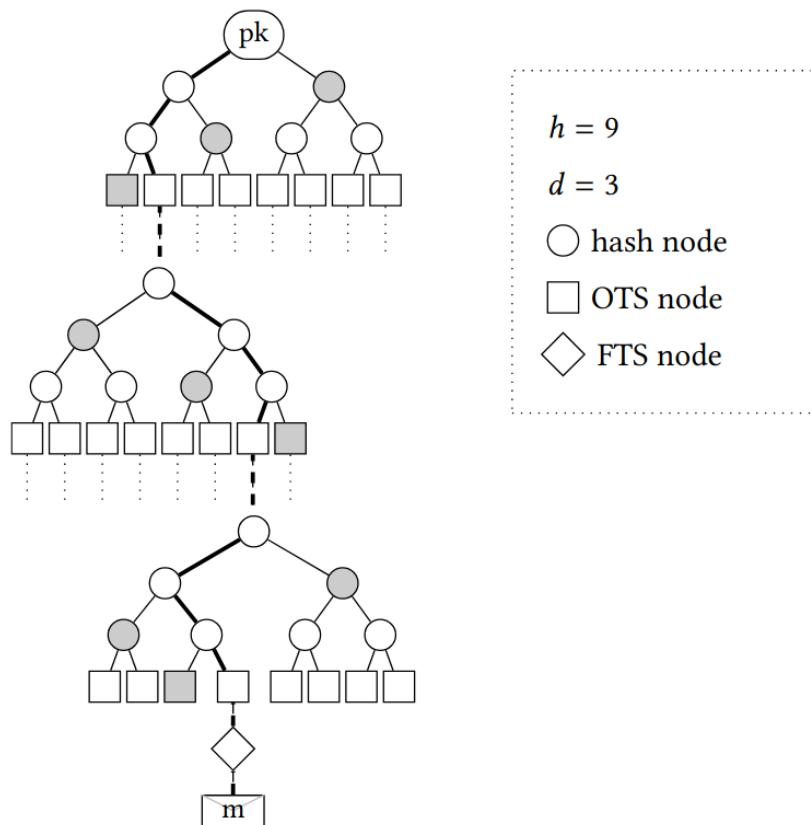


Figura 2.4: Exemplo de estrutura da *hypertree* do SPHINCS+ [5]

árvore do nível inferior e uma folha contendo o esquema FTS são selecionadas de forma determinística com base no *hash* da mensagem [7]. A assinatura também inclui o caminho de autenticação da folha selecionada até o nó raiz da árvore mais alta, que permite reconstruir a sequência de *hashes* até a raiz da árvore [5]. No processo de verificação, o receptor utiliza esse caminho para recalcular a raiz da estrutura hierárquica a partir da assinatura e da mensagem, e então compara esse valor com a chave pública previamente conhecida [7].

Além disso, o SPHINCS+ é altamente parametrizável, permitindo a escolha entre diferentes funções *hash*, categorias de segurança estabelecidas pelo NIST e perfis de desempenho. O esquema pode ser otimizado para menor tempo de assinatura (sufixo *f*) ou para assinaturas mais compactas (sufixo *s*) [5]. Essa flexibilidade torna o SPHINCS+ adequado para diversos tipos de aplicações, com exigências distintas de segurança e desempenho.

Por utilizar apenas funções *hash*, que são primitivas criptográficas bem compreendidas e com fortes garantias formais, o SPHINCS+ é visto como uma das opções mais conservadoras e robustas para a era pós-quântica [5]. Embora seu custo computacional seja mais elevado em comparação com algoritmos clássicos como RSA ou o algoritmo de assinatura digital de curvas elípticas (*Elliptic Curve Digital Signature Algorithm*, ECDSA),

essa desvantagem é compensada por sua flexibilidade, segurança formal e ausência de dependência de problemas matemáticos específicos [5].

2.4 SISTEMAS EMBARCADOS

O termo sistema embarcado refere-se a equipamentos eletrônicos desenvolvidos para executar uma função específica e com capacidade de processar dados. Esses sistemas se diferenciam dos computadores de uso geral, que são projetados para múltiplas aplicações [10].

Com os avanços em áreas como Internet das Coisas (IoT), computação em nuvem e computação de borda, o conceito evoluiu para incluir plataformas mais complexas, como aquelas baseadas em arquiteturas *multicore* e sistemas *System-on-Chip* (SoC), desde que empregadas em aplicações de propósito específico [10]. Nesse contexto, dispositivos modernos como *smartphones*, *smart TVs* e outros eletrônicos avançados também podem ser considerados sistemas embarcados [10].

Além disso, a emergência da IoT ampliou ainda mais esse papel ao conectar sistemas embarcados em redes heterogêneas capazes de fornecer novos serviços e aumentar a eficiência de aplicações já existentes. O que distingue sistemas embarcados tradicionais dos embarcados em IoT é justamente a conectividade: dispositivos antes isolados agora são projetados para interagir com outros equipamentos e com usuários por meio da Internet [39]. Essa interconexão transforma os sistemas embarcados em elementos fundamentais de ecossistemas distribuídos, abrangendo áreas como saúde, cidades inteligentes, automação industrial e ambientes domésticos [39].

2.4.1 LIMITAÇÕES DE RECURSOS EM SISTEMAS EMBARCADOS

Uma característica marcante dos sistemas embarcados é que eles frequentemente operam sob restrições severas de recursos, impostas por requisitos de custo, consumo energético e tamanho reduzido. Diferentemente de computadores de propósito geral, esses dispositivos frequentemente dispõem de pouca memória, capacidade de processamento limitada e fontes de energia restritas, fatores que impactam diretamente o desenvolvimento de aplicações mais complexas [43] [20]. Nesta seção, são discutidas as principais limitações de recursos, como memória, processamento e energia.

Cabe destacar que o ecossistema de sistemas embarcados é bastante diverso, abrangendo desde dispositivos extremamente restritos até plataformas mais complexas e potentes, o que resulta em diferentes níveis de limitação em memória, processamento e energia [43, 13].

2.4.1.1 MEMÓRIA

A memória é um dos recursos mais limitados em sistemas embarcados, frequentemente restrita a apenas alguns *kilobytes* (KB) de memória RAM e a capacidades reduzidas de armazenamento [13]. Essa limitação impõe fortes restrições tanto ao tamanho do código quanto à memória dinâmica utilizada para dados temporários e *buffers* de comunicação, que precisam ser rigidamente controlados [21]. Em razão disso, aplicações embarcadas exigem implementações compactas e eficientes, evitando desperdícios que poderiam inviabilizar a execução das tarefas previstas. O ecossistema de sistemas embarcados, entretanto, é bastante heterogêneo, abrangendo também dispositivos mais avançados com vários gigabytes (GB) de memória [13].

2.4.1.2 PROCESSAMENTO

A performance é apontada como um requisito central no projeto de sistemas embarcados, especialmente quando combinada a restrições de energia, exigindo soluções otimizadas que conciliem capacidade computacional reduzida com confiabilidade [8]. O processamento em sistemas embarcados também pode ser fortemente limitado, o que impõe desafios de previsibilidade temporal e capacidade de resposta. Muitas aplicações se enquadram na categoria de sistemas em tempo real, nos quais não apenas o resultado lógico das computações importa, mas também o instante em que é produzido. Em cenários críticos, como dispositivos médicos ou de controle aeronáutico, qualquer atraso pode comprometer a segurança, tornando a previsibilidade de execução um aspecto essencial do desenvolvimento [8].

2.4.1.3 ENERGIA

O consumo de energia é considerado um dos requisitos não funcionais mais críticos no projeto de sistemas embarcados, uma vez que esses dispositivos geralmente dependem de fontes limitadas, como baterias [8]. Quando a energia se esgota, o sistema deixa de funcionar, comprometendo totalmente sua operação [8]. Mesmo em dispositivos conectados à rede elétrica, o controle do consumo energético é essencial, pois o excesso

pode gerar calor elevado, degradando o desempenho do *chip* e até causando danos físicos ao *hardware* [8]. Para lidar com essa limitação, são empregadas técnicas de análise, que buscam estimar com precisão o gasto energético desde as fases iniciais do desenvolvimento, e de otimização, voltadas a reduzir o consumo sem violar restrições de desempenho, confiabilidade e tempo de execução [8].

2.4.2 ASSINATURAS DIGITAIS EM SISTEMAS EMBARCADOS E IoT

As assinaturas digitais também desempenham papel central em sistemas embarcados e dispositivos de IoT, onde garantem que mesmo equipamentos de baixo custo e recursos limitados possam operar de forma segura. Vários dos usos discutidos na seção 2.1.2.1, como a assinatura de *software*, a distribuição de atualizações e a autenticação de mensagens, são igualmente relevantes nesse domínio, mas devem ser adaptados às restrições específicas de processamento, memória e energia desses dispositivos.

2.4.2.1 SECURE BOOT E INTEGRIDADE DE FIRMWARE

O *secure boot* é um mecanismo fundamental para assegurar a integridade do *software* básico em sistemas embarcados. Nesse processo, cada estágio da inicialização é verificado por meio de assinaturas digitais, de forma que apenas códigos assinados podem ser executados. Caso qualquer verificação falhe, o processo de inicialização é interrompido. Esse procedimento estabelece uma cadeia de confiança que se inicia na raiz de confiança do dispositivo e se estende até o sistema operacional, garantindo que o equipamento não carregue *firmware* malicioso ou adulterado [17].

2.4.2.2 ATUALIZAÇÕES OVER-THE-AIR

Em redes de IoT, as atualizações de *firmware* são frequentemente realizadas de forma remota (*over-the-air*), uma vez que os dispositivos geralmente estão implantados em locais de difícil acesso físico. Esse processo é essencial para corrigir falhas, adicionar novas funcionalidades e estender a vida útil dos equipamentos. Contudo, ele introduz riscos de segurança, pois, sem validação adequada, um invasor poderia injetar *firmware* malicioso nos dispositivos. Para mitigar esse problema, o *firmware* transmitido deve ter sua autenticidade e integridade verificadas, o que pode ser feito por meio da assinatura digital da imagem de atualização e sua posterior validação no dispositivo receptor. Dessa forma,

garante-se que apenas código legítimo seja instalado, protegendo a operação de sistemas embarcados em larga escala [1].

2.4.2.3 AUTENTICAÇÃO DE MENSAGENS E IDENTIDADE DE DISPOSITIVOS

A autenticação é um requisito fundamental em sistemas embarcados e ambientes de IoT, pois sem ela um invasor pode se passar por um dispositivo legítimo, comprometendo toda a rede. Para enfrentar esse desafio, utiliza-se assinaturas digitais, que permitem verificar tanto a identidade de dispositivos quanto a autenticidade das mensagens transmitidas. Com isso, dispositivos podem assinar digitalmente suas comunicações, e os receptores validam as mensagens usando a chave pública correspondente, assegurando que a informação realmente se origina do dispositivo autorizado e que não foi alterada durante a transmissão [41].

2.4.2.4 ATESTAÇÃO REMOTA

A atestação remota é um mecanismo de segurança no qual um dispositivo gera evidências sobre seu estado interno, como a integridade do *firmware*, e as envia a um verificador externo. Esse processo segue o modelo de *challenge-response*, em que o verificador solicita informações e o dispositivo responde com um relatório protegido por assinatura digital. Dessa forma, garante-se que apenas dispositivos íntegros sejam reconhecidos como confiáveis, já que as assinaturas digitais permitem validar a autenticidade e a integridade das evidências fornecidas [2].

2.4.3 IMPLICAÇÕES DAS RESTRIÇÕES PARA A CRIPTOGRAFIA PÓS-QUÂNTICA

As limitações de memória, processamento e energia têm impacto direto na viabilidade de adoção de algoritmos de criptografia pós-quântica (PQC) em sistemas embarcados. Embora esses algoritmos ofereçam segurança reforçada contra futuros adversários quânticos, eles geralmente apresentam custos computacionais e de armazenamento significativamente maiores do que as soluções clássicas. Em dispositivos convencionais, tais requisitos já representam um desafio; em plataformas embarcadas, que variam de sistemas altamente restritos a configurações mais robustas, as limitações de recursos tornam sua integração ainda mais complexa. Mesmo em dispositivos que não se enquadram entre os mais limitados, a adoção de algoritmos de criptografia pós-quântica ainda pode impor desafios significativos de desempenho e uso de memória.

Em ambientes mais limitados, a própria disponibilidade de memória pode inviabilizar a execução de implementações de PQC, tornando esse recurso um fator decisivo na escolha do algoritmo a ser adotado [13]. Além disso, algoritmos de assinatura pós-quânticos tendem a demandar maior capacidade de processamento em comparação com soluções tradicionais, o que pode comprometer o uso em dispositivos restritos [18]. Diferentes algoritmos apresentam impactos variados em termos de tempo de execução, uso de memória e consumo energético, evidenciando a necessidade de analisar cuidadosamente os *trade-offs* de cada algoritmo antes da adoção prática [19] [34].

Dessa forma, a integração de PQC em sistemas embarcados requer a escolha de algoritmos mais adequados ao contexto, equilibrando desempenho, consumo de recursos e nível de segurança de forma a garantir sua viabilidade real.

3. FERRAMENTA DE BENCHMARK: ARQUITETURA E IMPLEMENTAÇÃO

Para viabilizar a análise comparativa proposta neste trabalho, foi desenvolvida uma ferramenta de *benchmarking* dedicada à avaliação de desempenho de algoritmos de assinatura digital. Esta ferramenta foi projetada para fornecer medições precisas e reproduzíveis das primitivas criptográficas, permitindo uma comparação justa entre algoritmos clássicos e pós-quânticos em diferentes ambientes computacionais.

Este capítulo está organizado da seguinte forma: a Seção 3.1 apresenta uma visão geral da ferramenta; a Seção 3.2 descreve as tecnologias e dependências utilizadas; a Seção 3.3 detalha a arquitetura do sistema; e, por fim, a Seção 3.3.3 discute as principais decisões de design adotadas.

3.1 VISÃO GERAL

A ferramenta de *benchmarking* implementa um sistema completo de medição de desempenho para algoritmos de assinatura digital, abrangendo tanto os esquemas criptográficos clássicos amplamente utilizados quanto os algoritmos de criptografia pós-quântica padronizados pelo NIST. O sistema foi desenvolvido na linguagem C, com integração direta às bibliotecas criptográficas OpenSSL 3, liboqs e oqs-provider, garantindo acesso às implementações de referência dos algoritmos avaliados. Para garantir isolamento de recursos e medições precisas, a ferramenta utiliza o *framework* BenchExec, que provê controle rigoroso sobre CPU, memória e I/O através de cgroups v2.

O escopo da ferramenta compreende a avaliação de algoritmos de assinatura digital clássicos, incluindo RSA-2048 e ECDSA com curva elíptica secp256r1 (prime256v1), e algoritmos pós-quânticos das famílias ML-DSA (anteriormente Dilithium), Falcon e SPHINCS+, abrangendo múltiplas variantes de cada família para permitir análise em diferentes níveis de segurança conforme as categorias definidas pelo NIST. Além disso, são disponibilizados diversos outros algoritmos de assinaturas digitais, principalmente pós-quânticos, totalizando mais de 100 variantes disponíveis.

As primitivas criptográficas avaliadas incluem três operações fundamentais: *key-gen* (geração de par de chaves pública/privada), *sign* (criação de assinatura digital sobre uma mensagem) e *verify* (verificação da autenticidade de uma assinatura). Essas operações representam os componentes essenciais de qualquer esquema de assinatura digital

e são medidas de forma isolada para permitir uma análise granular do desempenho de cada algoritmo. Além disso, foi desenvolvida uma opção para medir as três primitivas em conjunto.

As métricas coletadas pela ferramenta abrangem múltiplas dimensões de desempenho. O tempo de execução é medido de duas formas complementares: *walltime* (tempo de parede), que representa o tempo total decorrido incluindo esperas por I/O e interrupções do sistema, e *cputime* (tempo de CPU), que contabiliza apenas o tempo efetivo de processamento. A memória é medida através do pico de utilização durante a execução.

A ferramenta foi desenvolvida para operar em plataformas representativas de diferentes cenários de uso. As plataformas-alvo incluem sistemas Ubuntu 24.04 LTS ou superior em arquiteturas x86_64, representando ambientes de uso geral com recursos computacionais extensos. Para avaliar sistemas embarcados, a ferramenta foi adaptada para execução em Raspberry Pi 3 Model B com Raspberry Pi OS, permitindo análise de viabilidade em sistemas embarcados. Essa diversidade de plataformas possibilita avaliar não apenas o desempenho absoluto dos algoritmos, mas também sua adequação a diferentes contextos de aplicação. Para facilitar a replicação do ambiente em ambas as plataformas, foram desenvolvidos *scripts* automatizados de configuração que realizam a instalação de dependências, obtenção de bibliotecas em versões fixas e compilação de todos os componentes.

Para garantir a reprodutibilidade dos experimentos e facilitar trabalhos futuros, o código-fonte completo da ferramenta foi disponibilizado publicamente ¹. O repositório inclui todos os componentes desenvolvidos, *scripts* automatizados de configuração para ambas as plataformas-alvo, e documentação detalhada no arquivo *README* contendo instruções passo a passo para instalação e execução de *benchmarks*.

3.2 TECNOLOGIAS E DEPENDÊNCIAS

A implementação da ferramenta de *benchmark* fundamenta-se em um conjunto selecionado de bibliotecas e *frameworks* que garantem tanto a correção das implementações criptográficas quanto a precisão das medições de desempenho. Esta seção apresenta as dependências que formam a base do sistema.

¹<https://github.com/andrebins16/pqsign-bench>

3.2.1 DEPENDÊNCIAS CRIPTOGRÁFICAS

A camada criptográfica da ferramenta é construída sobre três componentes principais que operam de forma integrada: o *framework* OpenSSL, que provê a infraestrutura base e os algoritmos clássicos; a biblioteca liboqs, que implementa os algoritmos pós-quânticos; e o oqs-provider, que integra ambos através da arquitetura de *providers* do OpenSSL.

3.2.1.1 OPENSSL

O OpenSSL 3 constitui o *framework* criptográfico base da ferramenta, sendo utilizado diretamente do sistema operacional [33]. A versão 3 introduziu a *provider architecture*, um modelo de extensibilidade que permite carregar implementações criptográficas adicionais de forma modular através de bibliotecas dinâmicas em tempo de execução. A API EVP (*Enhanced Virtual Private API*) fornece uma interface unificada para operações criptográficas, permitindo que geração de chaves, assinatura e verificação sejam realizadas de forma uniforme independentemente do algoritmo subjacente. Essa uniformidade é essencial para a ferramenta de *benchmark*, garantindo que o mesmo código seja utilizado para todos os algoritmos avaliados, minimizando vieses nas comparações de desempenho.

3.2.1.2 LIBOQS

A biblioteca liboqs, desenvolvida pelo projeto Open Quantum Safe [31], fornece implementações em C de algoritmos de criptografia pós-quântica. Neste trabalho, foi utilizada a versão 0.14.0 para garantir reprodutibilidade dos experimentos. A biblioteca implementa diversos algoritmos padronizados pelo NIST para diferentes primitivas criptográficas, incluindo mecanismos de encapsulamento de chaves (KEMs) e esquemas de assinatura digital.

3.2.1.3 OQS-PROVIDER

O oqs-provider [32] atua como ponte entre a liboqs e o OpenSSL, implementando a *interface de provider* do OpenSSL 3. Neste trabalho, foi utilizada a versão 0.10.0. O oqs-provider expõe os algoritmos pós-quânticos da liboqs através da API EVP do OpenSSL, permitindo que sejam utilizados de forma transparente com as mesmas funções `EVP_PKEY_*` dos algoritmos clássicos.

3.2.2 FRAMEWORK DE BENCHMARKING: BENCHEXEC

A execução confiável de *benchmarks* apresenta desafios técnicos que vão além da simples medição de tempo e memória. Medições ingênuas podem produzir resultados imprecisos ou incorretos, especialmente quando o *software* avaliado gera processos filhos ou quando múltiplas execuções ocorrem em paralelo. Problemas como interferência entre processos, compartilhamento de recursos de *hardware* e variações na frequência da CPU podem introduzir não-determinismo e comprometer a reprodutibilidade dos experimentos [6].

Para *benchmarking* confiável, foram identificados requisitos indispensáveis [6], incluindo: medição e limitação precisa de recursos considerando processos filhos, terminação confiável de processos, alocação deliberada de núcleos de CPU, respeito à arquitetura NUMA, prevenção de *swapping* e isolamento entre execuções. Métodos tradicionais como *ulimit* e amostragem periódica de recursos falham em garantir esses requisitos. A solução proposta baseia-se em funcionalidades do *kernel* Linux: *cgroups v2* (*control groups*) e *namespaces* [6]. Os *cgroups* permitem gerenciamento hierárquico de grupos de processos, oferecendo controladores para medição e limitação precisa de CPU, memória e alocação de núcleos. Os *namespaces* provêm isolamento a nível de sistema operacional, permitindo que cada execução ocorra em um *container* isolado, prevenindo interferências.

O BenchExec [45] é uma implementação *open-source* que encapsula essas tecnologias em uma ferramenta prática. Sua arquitetura modular consiste de dois componentes principais: *runexec*, responsável pela execução isolada de uma única instância da ferramenta avaliada, e *benchexec*, que orquestra conjuntos completos de experimentos.

Neste trabalho, utilizamos o componente *runexec* devido à sua simplicidade e adequação aos requisitos. Para cada execução de primitiva criptográfica, o *runexec* cria um *container* isolado, aloca recursos via *cgroups*, executa o binário e coleta métricas de tempo e memória. O isolamento via *namespaces* garante que diferentes execuções não interfiram entre si, enquanto os *cgroups* asseguram medições precisas mesmo com múltiplos processos filhos.

3.3 ARQUITETURA DO SISTEMA

A ferramenta de *benchmark* foi projetada seguindo uma arquitetura modular baseada em componentes especializados e bem definidos. Os componentes interagem através de um fluxo de execução coordenado por uma orquestração central, permitindo separação

clara entre preparação de dados, medição isolada, execução de primitivas criptográficas e análise de resultados.

3.3.1 VISÃO GERAL DOS COMPONENTES

A Figura 3.1 ilustra a organização dos componentes e suas interações. O sistema é composto por sete componentes principais, divididos entre componentes desenvolvidos neste trabalho (em azul) e componentes externos utilizados, como *frameworks* ou bibliotecas (em amarelo).

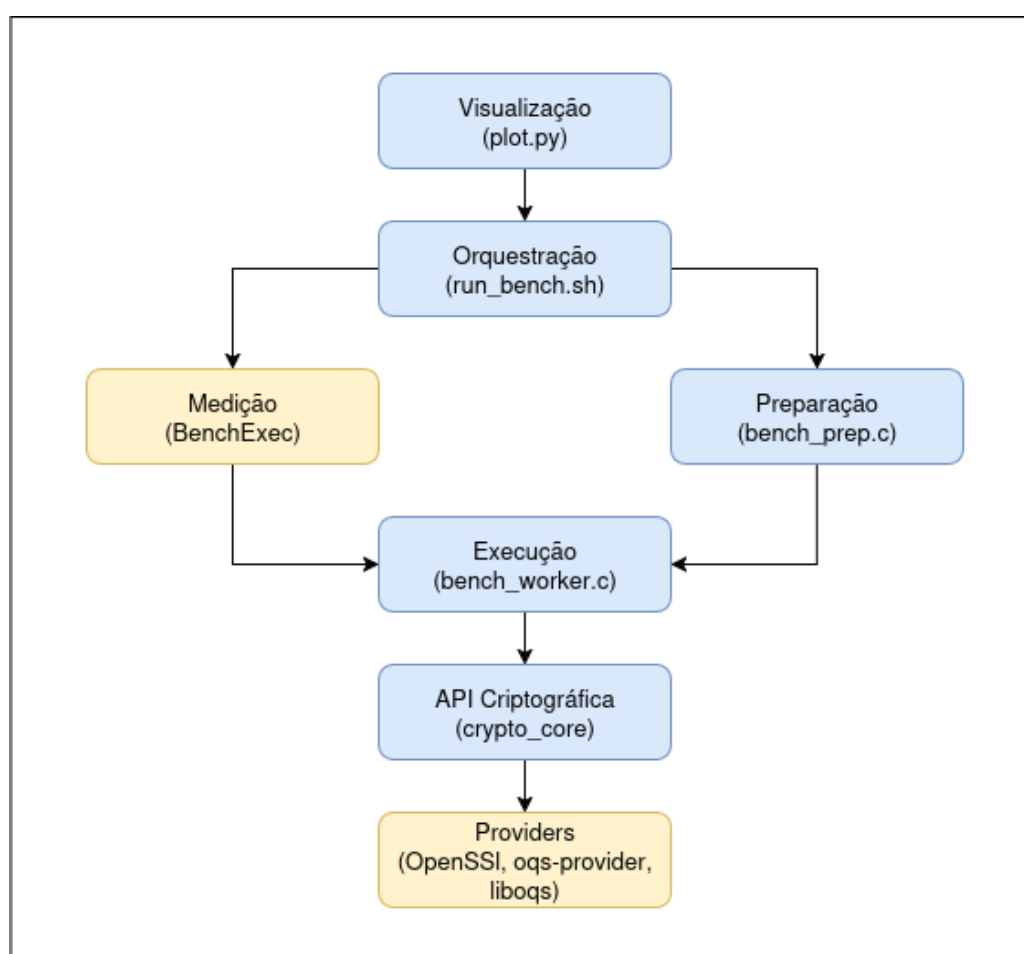


Figura 3.1: Diagrama de componentes da arquitetura

O fluxo de execução do *benchmark* é coordenado pelo componente de Orquestração, que gerencia a sequência completa de operações. Inicialmente, para operações que requerem dados pré-computados (assinatura e verificação), o componente de Preparação é invocado para gerar dados utilitários contendo chaves e assinaturas serializadas em formato Base64. Estes dados utilitários são então fornecidos como entrada para o componente de Execução, que realiza as operações criptográficas propriamente ditas. A execução não

ocorre diretamente: o componente de Medição encapsula o componente de Execução em um ambiente isolado (*container* Linux) e coleta métricas precisas de tempo de CPU, tempo de parede e memória através de cgroups. Tanto o componente de Preparação quanto o de Execução dependem da API Criptográfica, que oferece funções unificadas para operações sobre qualquer algoritmo (clássico ou pós-quântico) através da comunicação com a camada de Providers. Esta última encapsula as bibliotecas criptográficas externas (OpenSSL, oqs-provider e liboqs), que contêm as implementações reais dos algoritmos avaliados. Após múltiplas repetições e coleta de métricas, o componente de Orquestração calcula estatísticas (médias aparadas, desvios padrão) e consolida os resultados em um arquivo CSV. Por fim, o componente de Visualização possibilita o processamento deste CSV e a geração de gráficos comparativos para análise dos resultados.

3.3.2 COMPONENTES PRINCIPAIS

Esta subseção descreve individualmente os principais componentes que integram o sistema de *benchmarking*.

3.3.2.1 PROVIDERS (OpenSSL, oqs-provider, liboqs)

A camada de *Providers* é composta exclusivamente por bibliotecas externas que fornecem as implementações criptográficas reais dos algoritmos avaliados no *benchmark*. O OpenSSL 3 provê, através de seu *provider* padrão, as implementações dos algoritmos clássicos (RSA e ECDSA) e a infraestrutura base para operações criptográficas. A biblioteca liboqs contém implementações em C dos algoritmos pós-quânticos. O oqs-provider atua como ponte entre essas duas bibliotecas, implementando a *interface de provider* do OpenSSL 3 e expondo os algoritmos da liboqs através da API EVP. Esta integração permite que todos os algoritmos, sejam clássicos ou pós-quânticos, sejam acessados de forma uniforme através das mesmas funções do OpenSSL. Os *providers* são carregados dinamicamente em tempo de execução através da configuração especificada no arquivo `openssl-oqs.cnf` e permanecem ativos durante toda a sessão de *benchmarking*.

3.3.2.2 API CRIPTOGRÁFICA (`crypto_core`)

O componente da API Criptográfica é implementado em C nos arquivos `crypto_core.c` e `crypto_core.h`, oferecendo uma camada de abstração sobre os *providers* criptográficos. Seu objetivo é fornecer uma interface simples e unificada para as operações

de *benchmarking*, ocultando as complexidades da API EVP do OpenSSL e as diferenças entre famílias de algoritmos. As funções principais incluem: `bc_init_providers` e `bc_shutdown_providers` para gerenciamento do ciclo de vida dos providers; `bc_keygen` para geração de pares de chaves, recebendo apenas o nome do algoritmo e utilizando internamente `EVP_PKEY_keygen` para retornar as chaves; `bc_sign` para criação de assinaturas, utilizando internamente `EVP_DigestSign` para combinar *hash* e assinatura em uma única operação; `bc_verify` para verificação de assinaturas através de `EVP_DigestVerify`. Adicionalmente, o componente oferece funções auxiliares para serialização de chaves em formato PKCS#8 DER (`bc_pkey_to_pkcs8_der` e `bc_pkcs8_der_to_pkey`) e conversão para Base64 (`bc_b64_print_line` e `bc_b64_decode`), permitindo transporte de chaves e assinaturas via linha de comando. Um tratamento especial é implementado para algoritmos de curva elíptica, onde a curva `secp256r1` é configurada automaticamente via `OSSL_PARAM` durante a geração de chaves. Esta abstração garante que os componentes de nível superior utilizem código idêntico independentemente do algoritmo sendo avaliado.

3.3.2.3 PREPARAÇÃO (`bench_prep`)

O componente de Preparação, o `bench_prep.c`, é um executável desenvolvido em C responsável pela geração de dados utilitários (chaves e assinaturas pré-computadas) antes que qualquer medição ocorra. Seu propósito é evitar que o custo computacional da preparação de dados contamine as medições das primitivas criptográficas. O executável oferece dois subcomandos: `genkey -alg <ALGORITHM>`, que gera um par de chaves para o algoritmo especificado e retorna a chave privada serializada em Base64; e `gensig`, que pode receber uma chave existente (`-key-b64`) ou gerar uma nova (`-alg`), e produz uma assinatura válida para uma mensagem de tamanho especificado (`-msg-len`). As mensagens utilizadas são geradas de forma determinística como sequências de bytes zero, garantindo reprodutibilidade e consistência com o `bench_worker`: dado o mesmo tamanho, a mesma mensagem é sempre utilizada. As chaves e assinaturas são serializadas no formato PKCS#8 DER e convertidas para Base64 para facilitar o transporte via linha de comando. Os dados utilitários gerados são armazenados em variáveis na memória pelo *script* de orquestração e reutilizados em todas as repetições de um mesmo cenário de teste. Esta abordagem reduz significativamente o *overhead* nas medições do `bench_worker`.

3.3.2.4 EXECUÇÃO (`bench_worker`)

O componente de Execução, o `bench_worker.c`, é um executável em C que realiza a execução das primitivas criptográficas sob medição do BenchExec. Projetado para ser extremamente focado, recebe pela linha de comando os dados utilitários prontos (cha-

ves e assinaturas em Base64), realiza o parsing de argumentos e a decodificação desses dados, inicializa os contextos criptográficos necessários, executa a operação especificada e termina imediatamente. As mensagens para assinatura e verificação são geradas deterministicamente como sequências de bytes zero do tamanho especificado, garantindo consistência com o `bench_prep`. Oferece cinco subcomandos: `list-algs` lista os algoritmos disponíveis nos providers; `keygen -alg <ALGORITHM>` gera um par de chaves; `sign -key-b64 <KEY> -msg-len <N>` assina uma mensagem usando chave pré-gerada; `verify -key-b64 <KEY> -sig-b64 <SIG> -msg-len <N>` verifica uma assinatura pré-gerada; e `all -alg <ALGORITHM> -msg-len <N>` executa a sequência completa *keygen*, *sign* e *verify*. Todos os subcomandos, exceto `list-algs`, aceitam a *flag* `-baseline`, que desativa a execução da primitiva criptográfica mantendo todo o restante do código ativo (*parsing* de argumentos, decodificação Base64, inicialização de contextos EVP, alocações de memória). Isso permite medir o *overhead* puro do *framework*, possibilitando o cálculo aproximado tanto do tempo líquido quanto do consumo líquido de memória das primitivas através da subtração entre medições normais e *baseline*.

3.3.2.5 MEDIÇÃO (BenchExec)

O componente de Medição utiliza a ferramenta externa `runexec` do *framework* BenchExec para encapsular cada execução do `bench_worker`. Para cada invocação, recebe do *script* de orquestração os parâmetros de limite de recursos (tempo, memória, núcleos de CPU) e a linha de comando completa a executar. Após isso, cria o ambiente isolado através de *namespaces* e *cgroups*, executa o `bench_worker` até conclusão ou violação de limites, e retorna as métricas coletadas (*walltime*, *cputime*, *memory*, *exitcode*) em formato de pares chave-valor. Estas métricas são então capturadas pelo *script* de orquestração e armazenadas em arquivos temporários para posterior agregação estatística. O `runexec` garante que todas as medições sejam isoladas e que nenhum processo sobreviva após o término da execução, independentemente de como o `bench_worker` tenha finalizado.

3.3.2.6 ORQUESTRAÇÃO (run_bench.sh)

O componente de Orquestração é implementado como um *script* Bash que coordena o fluxo completo do benchmark, o `run_bench.sh`. O *script* aceita diversos parâmetros de linha de comando que controlam seu comportamento: `-algs` especifica a lista de algoritmos a serem avaliados (separados por vírgula); `-ops` define quais operações executar (*keygen*, *sign*, *verify*, *all*); `-reps` determina o número de repetições por cenário; `-sizes` define os tamanhos de mensagem em bytes a serem testados; `-trim-pct` especifica o percentual de valores extremos a remover no cálculo de estatísticas; `-baseline` habilita ou desabilita

medições de *overhead* e tempo líquido; `-outdir` define o diretório de saída; e parâmetros adicionais como `-timelimit`, `-memlimit` e `-cores` são repassados diretamente ao `runexec` para controlar limites de recursos e alocação de núcleos de CPU.

Suas responsabilidades abrangem todo o ciclo de execução: na fase de inicialização, configura variáveis de ambiente necessárias, valida a presença de binários requeridos (`bench_prep`, `bench_worker`, `runexec`) e cria o diretório de saída. Coleta metadados completos do sistema, incluindo modelo de CPU, quantidade de memória, versão do *kernel*, versões de bibliotecas (OpenSSL, liboqs, oqs-provider), topologia de *hardware* além dos parâmetros de execução fornecidos pelo usuário, armazenando tudo em `system_info.json` para garantir reprodutibilidade dos experimentos. Em seguida, entra no *loop* principal que itera sobre todas as combinações de algoritmo × operação × tamanho de mensagem especificadas. Para cada combinação, se a operação requer dados utilitários (*sign* ou *verify*), invoca `bench_prep` e armazena a saída em variáveis na memória. Executa então o número especificado de repetições: primeiro em modo normal, invocando `runexec <argumentosRunExec>` - `bench_worker <operação> <argumentos>` e capturando as métricas (*walltime*, *cputime*, *memory*, *exitcode*) em arquivos temporários; depois, se *baseline* está habilitado, executa as mesmas repetições com a *flag* `-baseline` adicionada ao `bench_worker`.

Após completar todas as repetições de um cenário, calcula estatísticas robustas: ordena os valores coletados, remove o percentual especificado de valores extremos de cada lado (*trimmed mean* para reduzir influência de *outliers*), calcula média e desvio padrão dos valores restantes para cada métrica. Para cenários com *baseline* habilitado, calcula métricas líquidas subtraindo o *overhead* medido do valor total puro.

Escreve então uma linha no `results.csv` contendo: informações de identificação (algoritmo, operação, tamanho de mensagem, número de repetições, percentual de trim); métricas *RAW* (tempo e memória da execução completa, incluindo primitiva e *overhead*); métricas *BASELINE* (tempo e memória do overhead isolado, quando habilitado); e métricas *NET* (tempo e memória líquidos estimados da primitiva, calculados como *RAW* - *BASELINE*). Este processo se repete para todas as combinações especificadas, consolidando os resultados em um único arquivo CSV que serve como entrada para análises posteriores e visualizações.

3.3.2.7 VISUALIZAÇÃO (plot.py)

O componente de Visualização é implementado como um *script* em Python denominado `plot.py`, responsável por processar o arquivo consolidado `results.csv` gerado pelo componente de orquestração e produzir saídas gráficas e tabulares que sintetizam os resultados obtidos. O script utiliza as bibliotecas *pandas* para manipulação de dados e *mat-*

matplotlib para renderização dos gráficos, ambas devendo ser instaladas separadamente no ambiente Python local.

A execução do `plot.py` gera duas saídas principais dentro do diretório de resultados (`results_dir`):

- Diretório *plots*: contém os gráficos em formato `.png`, organizados em subpastas conforme o tipo de métrica avaliada. São incluídas as métricas brutas (`wall_raw`, `cpu_raw`, `mem_raw`), as métricas de overhead quando o modo `-baseline` é utilizado (`wall_base`, `cpu_base`, `mem_base`), e as métricas líquidas resultantes da subtração do *overhead* (`wall_net`, `cpu_net`, `mem_net`). Cada gráfico é apresentado no formato *horizontal bar chart*, em que o eixo Y lista os algoritmos avaliados e o eixo X representa o valor da métrica (tempo em milissegundos ou memória em megabytes (MB)). Barras de erro indicam o desvio padrão calculado a partir das repetições. Os gráficos são agrupados automaticamente por operação (*keygen*, *sign*, *verify*, *all*) e tamanho de mensagem.
- Diretório *subTables*: armazena, para cada gráfico gerado, um arquivo `.csv` contendo apenas os dados correspondentes àquele gráfico. Esses arquivos incluem as colunas *algorithm*, *value* e *std*, representando respectivamente o nome do algoritmo, o valor médio da métrica e o desvio padrão associado. Essa organização permite análises mais detalhadas, pois esses arquivos são muito menores, o que facilita a visualização de métricas específicas, sem necessidade de analisar o arquivo `results.csv` completo.

Essa estrutura de visualização proporciona uma separação clara entre as representações gráficas e os dados numéricos, facilitando a análise, a replicação e a comparação dos resultados obtidos em diferentes plataformas e configurações experimentais.

3.3.3 DECISÕES DE DESIGN

As decisões de *design* da ferramenta foram orientadas pela adoção do BenchExec como *framework* de medição. Uma abordagem mais simples seria medir o tempo e memória diretamente no código através de chamadas como `clock_gettime` imediatamente antes e depois da execução de cada primitiva criptográfica. Esta abordagem teria menor *overhead* e variabilidade reduzida: o código seria autocontido, sem necessidade de serialização de dados, processos externos ou comunicação via linha de comando. No entanto, optou-se por utilizar o BenchExec para seguir as melhores práticas de *benchmarking* confiável identificadas na literatura [6], que incluem isolamento via *containers*, medição precisa via `cgroups`

considerando processos filhos e alocação deliberada de recursos de *hardware*. Esta escolha introduz *overhead* adicional e maior variabilidade devido à comunicação entre processos, mas garante medições mais confiáveis, reproduzíveis e comparáveis, representando um *trade-off* favorável entre simplicidade de implementação e confiabilidade metodológica.

O uso do BenchExec impõe que cada execução medida seja um processo isolado, invocado externamente. Isso motivou a separação entre preparação (`bench_prep`) e execução (`bench_worker`). Como o BenchExec mede o processo completo do `bench_worker`, operações de preparação como geração de chaves para *sign/verify* precisam ocorrer externamente para não contaminar as medições. A solução gera dados utilitários (chaves e assinaturas) com `bench_prep` antes de qualquer medição, armazena-os em variáveis na memória do *script* de orquestração, e os reutiliza em todas as repetições. Assim, o `bench_worker` recebe dados prontos e mede apenas a primitiva alvo.

Como o `bench_worker` é invocado como processo externo, dados precisam ser transportados via linha de comando, motivando a serialização em PKCS#8 DER e conversão para Base64. Adicionalmente, as mensagens são geradas deterministicamente como bytes zero para evitar transportar grandes volumes de dados: apenas o tamanho é especificado, e a mensagem é recriada localmente de forma idêntica em `bench_prep` e `bench_worker`. Estas escolhas minimizam o *overhead* de transporte mantendo reprodutibilidade.

Mesmo sendo um processo isolado, o `bench_worker` ainda executa código auxiliar inevitável: *parsing* de argumentos da linha de comando, decodificação Base64, deserialização de chaves, criação da mensagem determinística e inicialização de contextos criptográficos. Para tentar minimizar esse *overhead* residual, implementou-se o mecanismo de *baseline*: a `flag -baseline` executa todo o código exceto a primitiva criptográfica, permitindo medir apenas o *overhead*. Com medições *RAW* (primitiva + *overhead*) e *BASELINE* (apenas *overhead*), calcula-se o tempo líquido: $NET = RAW - BASELINE$.

Finalmente, para lidar com variabilidade inerente às medições, que é amplificada pelo *overhead* da comunicação entre processos, utiliza-se média podada: após múltiplas repetições, remove-se N% dos valores extremos antes de calcular estatísticas, descartando *outliers* causados por fatores não completamente controláveis.

Em síntese, a adoção do BenchExec introduziu complexidade arquitetural significativa e custos mensuráveis em termos de *overhead* e variabilidade. Entretanto, esses custos são amplamente compensados pelos benefícios metodológicos: as medições seguem rigorosamente as melhores práticas estabelecidas na literatura de *benchmarking* confiável [6], garantindo isolamento adequado, reprodutibilidade entre diferentes ambientes de execução e aumentando a confiabilidade das medições.

4. ANÁLISE DE DESEMPENHO DE ALGORITMOS DE ASSINATURA DIGITAL EM AMBIENTES COMPUTACIONAIS DISTINTOS

Este capítulo apresenta os resultados experimentais obtidos com a ferramenta de *benchmarking* desenvolvida, conforme descrito no Capítulo 3. O objetivo é avaliar comparativamente o desempenho de algoritmos de assinatura digital clássicos e pós-quânticos em diferentes plataformas computacionais, com foco no tempo de processamento e análise exploratória do consumo de memória, considerando os diferentes níveis de segurança oferecidos.

A investigação contribui para a compreensão da viabilidade prática de adoção de criptografia pós-quântica em ambientes computacionais distintos. O capítulo está organizado da seguinte forma: a Seção 4.1 descreve a metodologia experimental adotada; a Seção 4.2 apresenta os resultados obtidos; e, por fim, a Seção 4.3 discute os resultados encontrados e suas implicações práticas.

4.1 METODOLOGIA EXPERIMENTAL

Esta seção detalha o protocolo experimental adotado, apresentando os algoritmos selecionados para análise, as características das plataformas computacionais utilizadas, os parâmetros de execução dos testes e as métricas coletadas durante os experimentos.

4.1.1 ALGORITMOS AVALIADOS

Foram avaliados 19 variantes de algoritmos de assinatura digital, divididos em duas categorias: algoritmos clássicos amplamente utilizados e algoritmos pós-quânticos padronizados pelo NIST.

Os algoritmos clássicos selecionados foram o RSA-2048 e o ECDSA com a curva *secp256r1* (*prime256v1*), representando padrões amplamente utilizados atualmente em aplicações de assinatura digital. Esses algoritmos serviram como base de comparação, fornecendo uma referência de desempenho para as implementações pós-quânticas. Para os algoritmos pós-quânticos, foram selecionadas múltiplas variantes de três famílias principais: ML-DSA, Falcon e SPHINCS+. As variantes abrangem diferentes níveis de segurança conforme as categorias definidas pelo NIST e diferentes perfis de otimização.

A Tabela 4.1 apresenta as especificações consolidadas dos algoritmos pós-quânticos avaliados, incluindo a categoria de segurança segundo a classificação do NIST e os tamanhos das chaves públicas, privadas e assinaturas digitais, todos expressos em *bytes* [34] [29]. Essa comparação evidencia os compromissos entre segurança e tamanho das estruturas criptográficas, cuja compreensão é fundamental para orientar a escolha de algoritmos mais adequados a diferentes contextos de aplicação.

Algoritmo	Categoria de segurança NIST	Tamanho da chave pública	Tamanho da chave privada	Tamanho da assinatura
ML-DSA-44	2	1312	2560	2420
ML-DSA-65	3	1952	4032	3309
ML-DSA-87	5	2592	4896	4627
Falcon-512	1	897	1281	666
Falcon-1024	5	1793	2305	1280
SPHINCS+—SHA2-128f-simple	1	32	64	17088
SPHINCS+—SHA2-128s-simple	1	32	64	7856
SPHINCS+—SHA2-192f-simple	3	48	96	35664
SPHINCS+—SHA2-192s-simple	3	48	96	16224
SPHINCS+—SHA2-256f-simple	5	64	128	49856
SPHINCS+—SHA2-256s-simple	5	64	128	29792
SPHINCS+—SHAKE-128f-simple	1	32	64	17088
SPHINCS+—SHAKE-128s-simple	1	32	64	7856
SPHINCS+—SHAKE-192f-simple	3	48	96	35664
SPHINCS+—SHAKE-192s-simple	3	48	96	16224
SPHINCS+—SHAKE-256f-simple	5	64	128	49856
SPHINCS+—SHAKE-256s-simple	5	64	128	29792

Tabela 4.1: Algoritmos pós-quânticos avaliados e suas especificações [34] [29]

4.1.2 AMBIENTES COMPUTACIONAIS

Os experimentos foram realizados em duas plataformas computacionais com arquiteturas e capacidades distintas, de modo a representar diferentes cenários de aplicação para algoritmos de assinatura digital. O objetivo foi comparar o comportamento dos algoritmos em um ambiente de alto desempenho e em um sistema embarcado.

A primeira plataforma corresponde a um *notebook* com sistema Ubuntu 24.04.3 LTS, equipado com um processador Intel Core i7-12700H (12^a geração), com 14 núcleos físicos e 20 *threads* lógicos, frequência base de 2.3 GHz e *cache* L3 de 24 MiB. O sistema possui 16 GB de memória RAM. Essa configuração representa um ambiente de alto desempenho, adequado para servir como referência nos testes comparativos.

A segunda plataforma corresponde a um Raspberry Pi 3 Model B, representando um dispositivo embarcado com recursos computacionais reduzidos em comparação ao primeiro sistema. O sistema executa o Raspberry Pi OS (64 bits, baseado em Debian GNU/Linux 12 Bookworm) sobre um processador ARM Cortex-A53 quad-core de 64 bits, com frequência de 1.4 GHz e 1 GB de memória RAM. O Raspberry Pi 3B representa uma categoria intermediária de dispositivos embarcados, com recursos consideravelmente mais modestos que os de um computador pessoal, mas superiores aos de plataformas altamente restritas. Futuramente, seria interessante estender a análise para dispositivos ainda mais limitados, avaliando o comportamento dos algoritmos em cenários de recursos críticos.

Em ambas as plataformas, o ambiente de *software* foi mantido consistente, utilizando o OpenSSL 3.0.17 integrado às bibliotecas liboqs 0.14.0 e oqs-provider 0.10.0. O *framework* BenchExec 3.30 foi empregado para orquestração e medição dos experimentos. Essa padronização garantiu a comparabilidade direta dos resultados obtidos entre as duas arquiteturas.

4.1.3 PARÂMETROS DE EXECUÇÃO E MÉTRICAS AVALIADAS

Os experimentos seguiram um protocolo padronizado de execução, garantindo comparabilidade entre algoritmos e plataformas. Cada algoritmo foi avaliado nas quatro operações fundamentais: geração de chaves, assinatura, verificação e ciclo completo (execução sequencial das três operações). Cada operação foi repetida 100 vezes, e as médias e desvios padrão foram calculados após a aplicação de uma média aparada de 15 % para redução de valores atípicos.

As mensagens de entrada foram testadas em três tamanhos distintos: 256 bytes, 100 KB e 100 MB, de forma a avaliar o impacto do tamanho da mensagem no tempo de processamento e no consumo de memória. Os experimentos foram executados com o modo *baseline* habilitado, permitindo a compensação do custo de execução do ambiente. Assim, todas as análises apresentadas neste capítulo consideram os valores líquidos obtidos pela diferença entre as métricas brutas e os respectivos valores de *baseline*.

Os limites de tempo e memória foram ajustados conforme a capacidade de cada sistema, com base em uma análise empírica. Foram realizadas execuções preliminares para identificar tempos típicos de processamento e o consumo de memória dos algoritmos em ambas as plataformas. A partir desses testes, definiram-se limites médios suficientemente amplos para evitar falhas por esgotamento de recursos, sem comprometer a comparação entre os ambientes. No ambiente Ubuntu, foi configurado um limite de 5 s de tempo de CPU, 6 s de tempo de parede e 500 MB de memória. No Raspberry Pi 3 Model B, os

limites foram expandidos para 35 s de CPU, 36 s de tempo de parede e o mesmo limite de 500 MB de memória. Todas as execuções foram realizadas com o governador de CPU performance ativo, assegurando condições estáveis de medição.

Durante a execução dos experimentos, foram coletadas as métricas de tempo de parede (*wall time*), tempo de CPU (*cpu time*) e pico de memória (*memory peak*) para cada operação e algoritmo. As análises apresentadas neste capítulo concentram-se prioritariamente no tempo de CPU líquido. O pico de memória líquido também foi coletado, embora essa métrica seja discutida de forma mais breve devido a limitações identificadas nos resultados, conforme detalhado na Seção 4.3.

Os resultados consolidados foram exportados em formato CSV e processados posteriormente pelo módulo de visualização desenvolvido, responsável pela geração automática de gráficos e tabelas comparativas das métricas avaliadas.

4.2 RESULTADOS

Esta seção apresenta os resultados obtidos a partir do protocolo experimental descrito na Seção 4.1. A análise é estruturada em duas partes principais: consumo de memória, avaliado a partir do pico de memória líquido, e tempo de processamento, avaliado com base no tempo de CPU líquido, sendo a segunda métrica o foco principal da análise. Em ambas as métricas, são mantidas as mesmas condições de execução entre plataformas, permitindo uma comparação direta do comportamento dos algoritmos em diferentes ambientes computacionais.

É importante ressaltar que, embora o procedimento de *baseline* reduza significativamente o *overhead* do ambiente experimental, os valores líquidos apresentados constituem aproximações que podem conter *overhead* residual. Assim, a análise concentra-se prioritariamente nos padrões relativos de desempenho entre os algoritmos e entre as plataformas, ao invés de valores absolutos isolados, permitindo identificar tendências comparativas robustas.

4.2.1 AVALIAÇÃO DE CONSUMO DE MEMÓRIA

Nesta subseção, é analisado o pico de memória líquido observado durante a execução dos experimentos. Essa métrica corresponde ao maior valor de memória efetivamente utilizado pelo processo durante a execução da primitiva criptográfica, após a sub-

tração do valor obtido no modo *baseline*. Esse procedimento busca eliminar o impacto de fatores externos ao núcleo da operação, como a inicialização do ambiente, o carregamento de bibliotecas e a alocação das mensagens de teste. Assim, o pico de memória líquido não inclui a memória alocada pelo próprio código experimental para armazenar a mensagem de entrada, pois esta foi contabilizada no modo *baseline*.

Durante os experimentos, observou-se que as métricas de memória apresentaram maior variação entre execuções em comparação às de tempo, especialmente nas operações individuais (geração de chaves, assinatura e verificação), além de alguns resultados pontualmente inconsistentes ou inesperados. Diante dessas oscilações e para evitar uma análise excessivamente extensa e repetitiva, optou-se por realizar a avaliação dos resultados de memória de forma breve, incluindo apenas a operação do ciclo completo, que engloba geração de chaves, assinatura e verificação. Essa abordagem é suficiente para caracterizar o comportamento global de uso de memória, uma vez que o pico tende a ocorrer em uma das etapas do ciclo, refletindo um limite superior do consumo observado em condições práticas.

A análise é apresentada separadamente para cada tamanho de mensagem (256 bytes, 100 KB e 100 MB), considerando conjuntamente os resultados das duas plataformas. Para cada caso, são exibidos gráficos com os valores médios e desvio padrão dos picos de memória líquidos em *megabytes*.

Mensagens de 256 bytes

A Figura 4.1 apresenta os resultados do pico de memória líquido obtido durante o ciclo completo com mensagens de 256 bytes. O consumo de memória mostrou-se bastante homogêneo entre todos os algoritmos, situando-se na faixa entre 1 MB e 2 MB. Entre os esquemas clássicos, o RSA apresentou o menor consumo em ambas as plataformas, seguido de perto pelo ECDSA.

No ambiente Ubuntu, os algoritmos SPHINCS+–SHAKE e SPHINCS+–SHA2-128 apresentaram os menores consumos entre os esquemas pós-quânticos, aproximadamente 1,5 vezes superiores aos dos algoritmos clássicos. Em seguida, vieram as variantes SHA2-192 e SHA2-256 do SPHINCS+, enquanto ML-DSA e Falcon registraram os maiores picos de memória entre os pós-quânticos, embora com diferença modesta, cerca de duas vezes o consumo dos clássicos no máximo.

No Raspberry Pi 3B, os algoritmos SPHINCS+–SHAKE e todas as variantes do ML-DSA apresentaram picos aproximadamente 1,3 vezes maiores que os clássicos, enquanto as versões SHA2 do SPHINCS+ e Falcon alcançaram por volta de 1,5 vezes o consumo dos esquemas clássicos.

Ao comparar as plataformas, observam-se padrões gerais semelhantes, com pequenas diferenças pontuais. No Raspberry Pi, o ML-DSA apresentou consumo relativamente menor que no Ubuntu, enquanto as variantes SPHINCS+—SHA2-128 mostraram leve aumento. De modo geral, o uso de memória no Raspberry Pi foi ligeiramente menor em comparação ao Ubuntu.

Para mensagens pequenas, os picos de memória permaneceram próximos entre os algoritmos, especialmente no Raspberry Pi. As variações entre níveis de segurança e entre as variantes *f-simple* e *s-simple* do SPHINCS+ foram pouco expressivas, com maior impacto observado apenas nas diferenças entre as famílias de *hash* SHA2 e SHAKE.

Mensagens de 100 KB

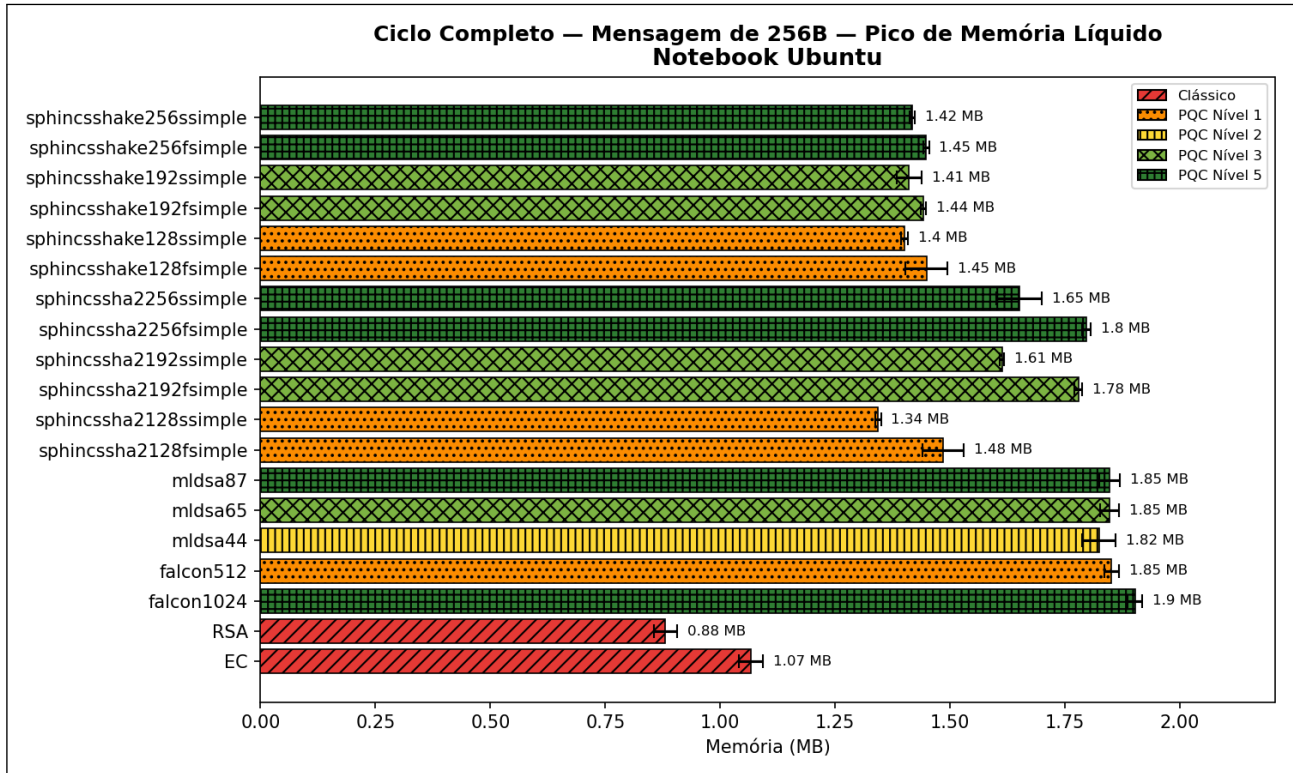
A Figura 4.2 apresenta os resultados do pico de memória líquido durante a execução do ciclo completo com mensagens de 100 KB. Os padrões observados foram praticamente idênticos aos das mensagens de 256 bytes, com valores bastante próximos entre todos os algoritmos. O consumo médio situou-se entre 0,8 MB e 2,3 MB, mantendo diferenças discretas entre famílias e plataformas.

Entre os esquemas clássicos, o RSA manteve o menor consumo de memória em ambas as plataformas, seguido pelo ECDSA, com a diferença entre ambos sendo ligeiramente maior no Ubuntu.

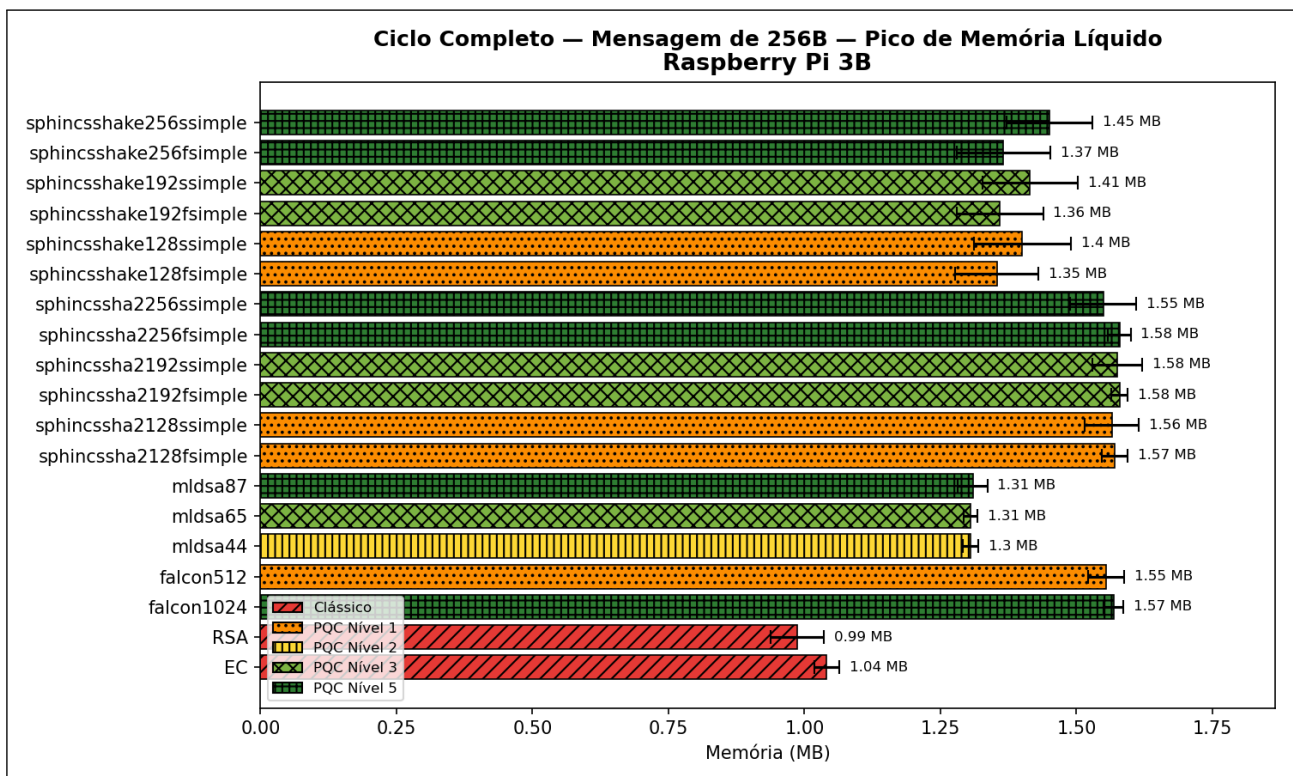
Nos esquemas pós-quânticos, o comportamento observado no Ubuntu foi semelhante ao do tamanho anterior. Os algoritmos SPHINCS+—SHAKE e SPHINCS+—SHA2-128 apresentaram os menores consumos, cerca de 1,5 vezes superior aos clássicos. Em seguida, vieram SPHINCS+—SHA2-192, SPHINCS+—SHA2-256 e ML-DSA, com picos próximos a duas vezes o valor dos clássicos, enquanto o Falcon registrou ligeiro aumento adicional, mantendo-se, contudo, na mesma faixa geral.

No Raspberry Pi 3B, os padrões se repetiram com valores ligeiramente menores e variação mais uniforme entre famílias. O ML-DSA apresentou o menor consumo entre os pós-quânticos, aproximadamente 1,6 vezes maior que os clássicos. Os algoritmos SPHINCS+ e o Falcon-512 registraram valores semelhantes, em torno de 1,9 vezes maiores, com as variantes SHAKE ligeiramente mais econômicas, enquanto o Falcon-1024 apresentou o maior consumo entre os pós-quânticos, embora ainda próximo dos demais.

Entre as plataformas, os padrões gerais permaneceram consistentes, com o Raspberry Pi apresentando novamente leve redução no consumo médio. O comportamento das famílias manteve-se estável, com pequenas variações relativas: o ML-DSA destacou-se por



(a) Notebook Ubuntu



(b) Raspberry Pi 3B

Figura 4.1: Comparação do pico de memória líquido do ciclo completo com mensagem de 256 bytes em ambas plataformas

um consumo proporcionalmente menor no Pi, enquanto SPHINCS+—SHA2-128 mostrou leve aumento em relação ao Ubuntu.

O aumento do tamanho da mensagem para 100 KB não resultou em variações significativas no pico de memória. No Ubuntu, houve crescimento médio de por volta de 1,2 vezes em relação aos testes de 256 bytes, enquanto no Raspberry Pi o consumo permaneceu praticamente estável, com pequenas elevações apenas nos esquemas pós-quânticos. Os algoritmos clássicos no Raspberry Pi apresentaram leve redução no consumo médio, comportamento possivelmente relacionado à maior variação observada nessa métrica. O resultado reforça que o tamanho da mensagem exerce impacto limitado sobre o pico líquido de memória para entradas até 100 KB, mantendo os valores dentro de uma faixa estreita e estável.

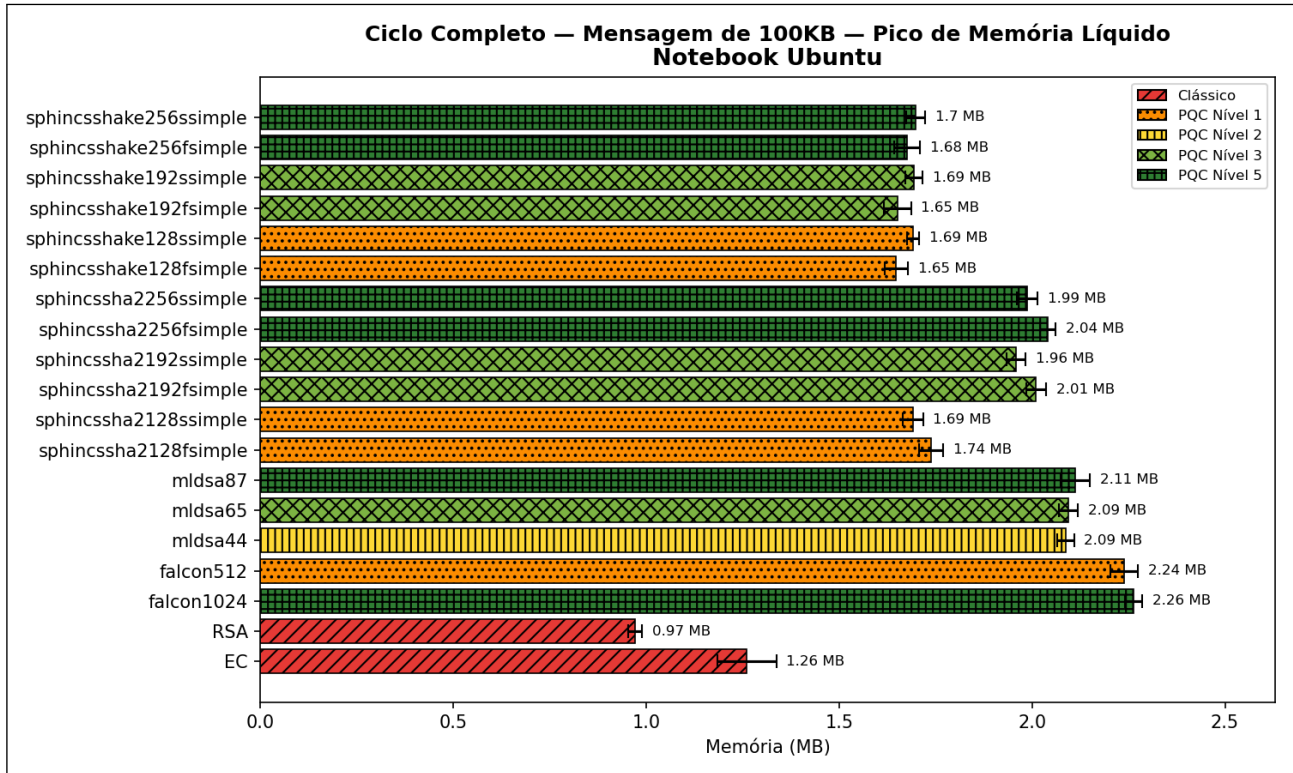
Mensagens de 100 MB

A Figura 4.3 apresenta os resultados do pico de memória líquido para o ciclo completo com mensagens de 100 MB. Nesta configuração, ambas as plataformas exibiram comportamento idêntico: todos os algoritmos pós-quânticos apresentaram valores próximos de 202 MB, indicando um padrão uniforme de consumo de memória independentemente da família, nível de segurança ou variante. Esse aumento expressivo em relação aos testes anteriores reflete o impacto direto do tamanho da mensagem sobre a alocação de memória nos algoritmos pós-quânticos.

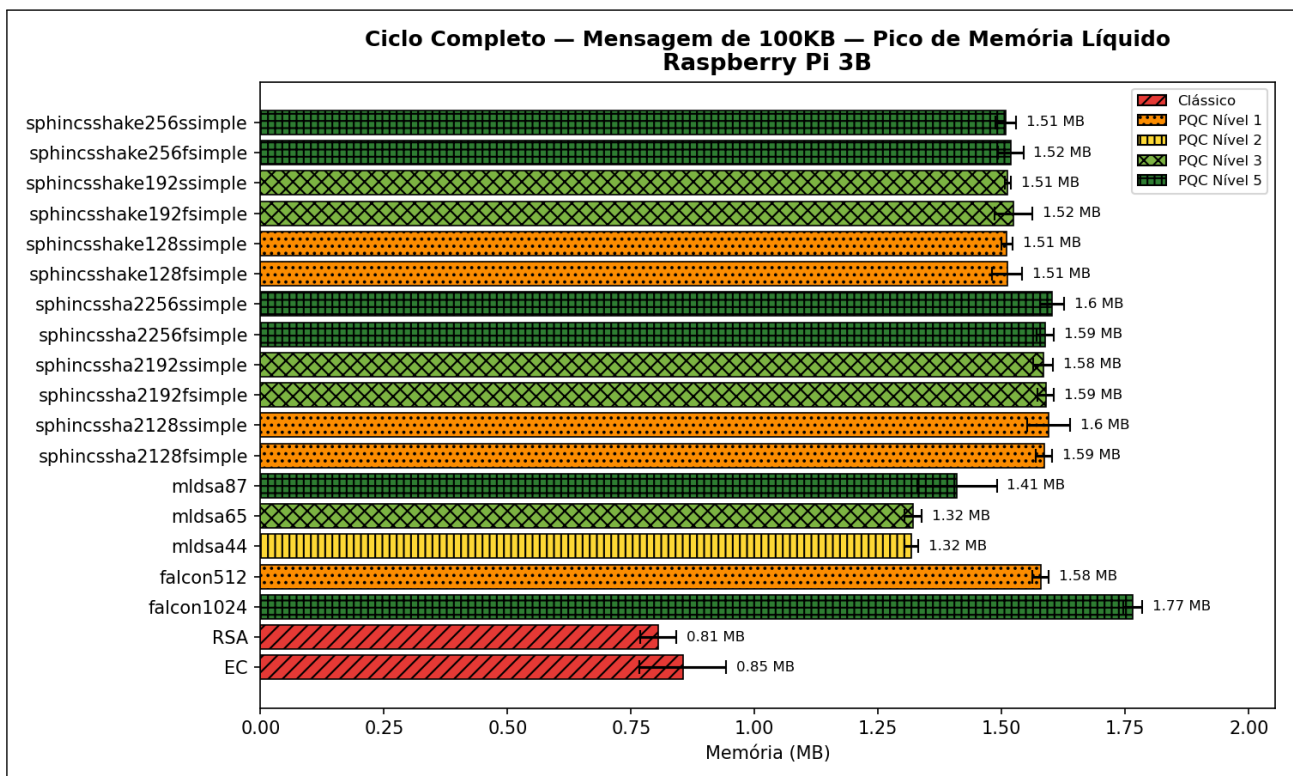
Como a memória utilizada para armazenar a mensagem original não é contabilizada no cálculo do pico líquido, por já ter sido incluída no modo *baseline*, o valor observado sugere que as implementações pós-quânticas realizaram alocações internas adicionais durante o processamento, possivelmente mantendo cópias intermediárias dos dados. Esse comportamento indica que o uso de memória cresce proporcionalmente ao tamanho da entrada, tornando-se dominante sobre o custo puramente criptográfico, que prevalecia nas mensagens menores.

Em contraste, os algoritmos clássicos RSA e ECDSA mantiveram praticamente o mesmo consumo de memória observado nos tamanhos anteriores, permanecendo próximos de 1 MB em ambas as plataformas, demonstrando que as implementações clássicas não efetuam realocação adicional significativa durante o processamento de mensagens extensas.

Consequentemente, a diferença entre os esquemas clássicos e os pós-quânticos tornou-se muito acentuada neste cenário, com os segundos exibindo consumo de mais de duas ordens de magnitude superior. Esse comportamento, entretanto, deve ser interpretado com cautela, pois pode estar relacionado a particularidades das bibliotecas utilizadas:



(a) Notebook Ubuntu



(b) Raspberry Pi 3B

Figura 4.2: Comparação do pico de memória líquido do ciclo completo com mensagem de 100 KB em ambas plataformas

os algoritmos pós-quânticos foram executados por meio da liboqs, enquanto os clássicos utilizam diretamente as rotinas do OpenSSL. Assim, parte desse aumento pode refletir diferenças na forma como cada biblioteca gerencia internamente *buffers* e estruturas de dados, não necessariamente uma característica inerente dos algoritmos em si.

4.2.2 AVALIAÇÃO DE TEMPO DE PROCESSAMENTO

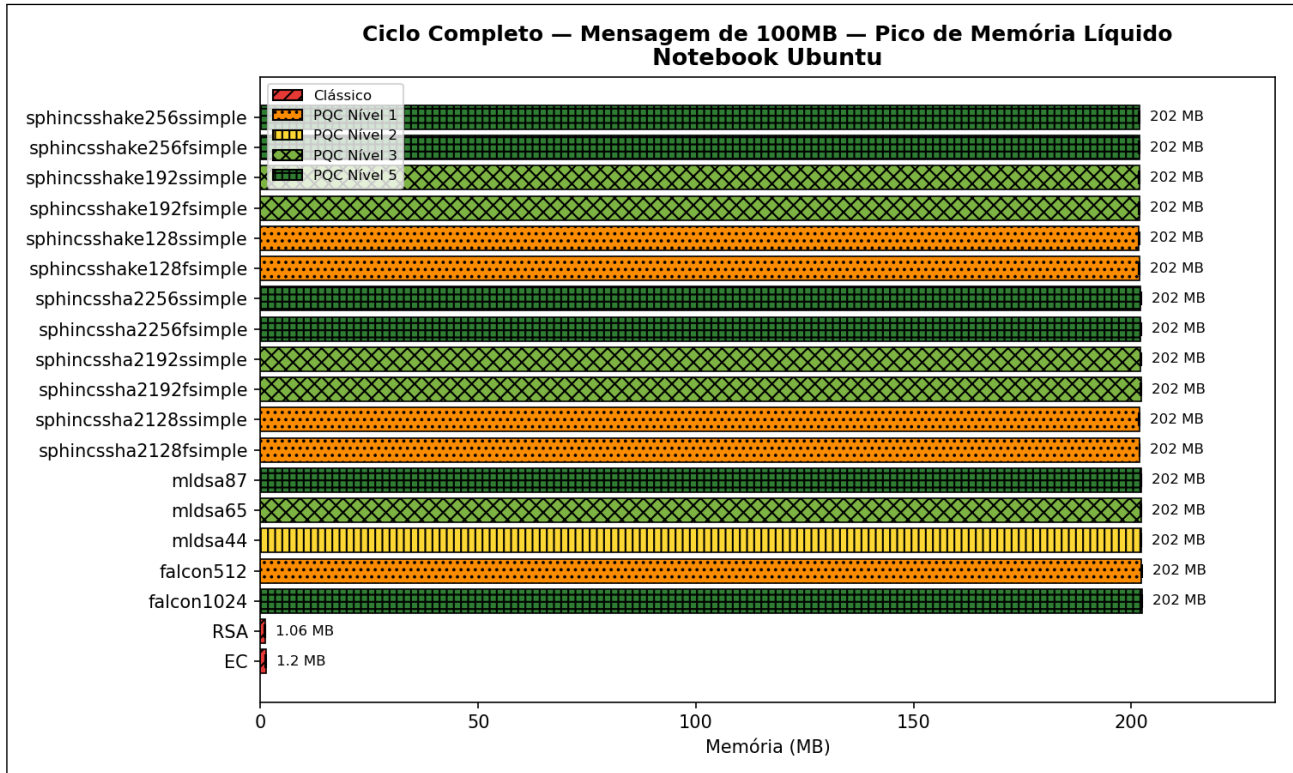
Esta subseção apresenta a análise do tempo de CPU líquido obtido para cada operação dos algoritmos avaliados. Essa métrica reflete o custo computacional efetivo de processamento e permite comparar o desempenho das diferentes famílias de algoritmos de assinatura digital em ambas as plataformas testadas.

Cada operação (geração de chaves, assinatura, verificação e ciclo completo) é avaliada separadamente, considerando ambas as plataformas. Para cada combinação de operação e tamanho de mensagem, são apresentados gráficos comparativos das duas plataformas. Os resultados são apresentados para mensagens de 256 bytes e 100 MB, representando arquivos de pequeno e grande porte, respectivamente. Mensagens de 100 KB não são exibidas por apresentarem comportamento equivalente ao de 256 bytes, sendo portanto os resultados de 256 bytes considerados representativos também para entradas de médio porte.

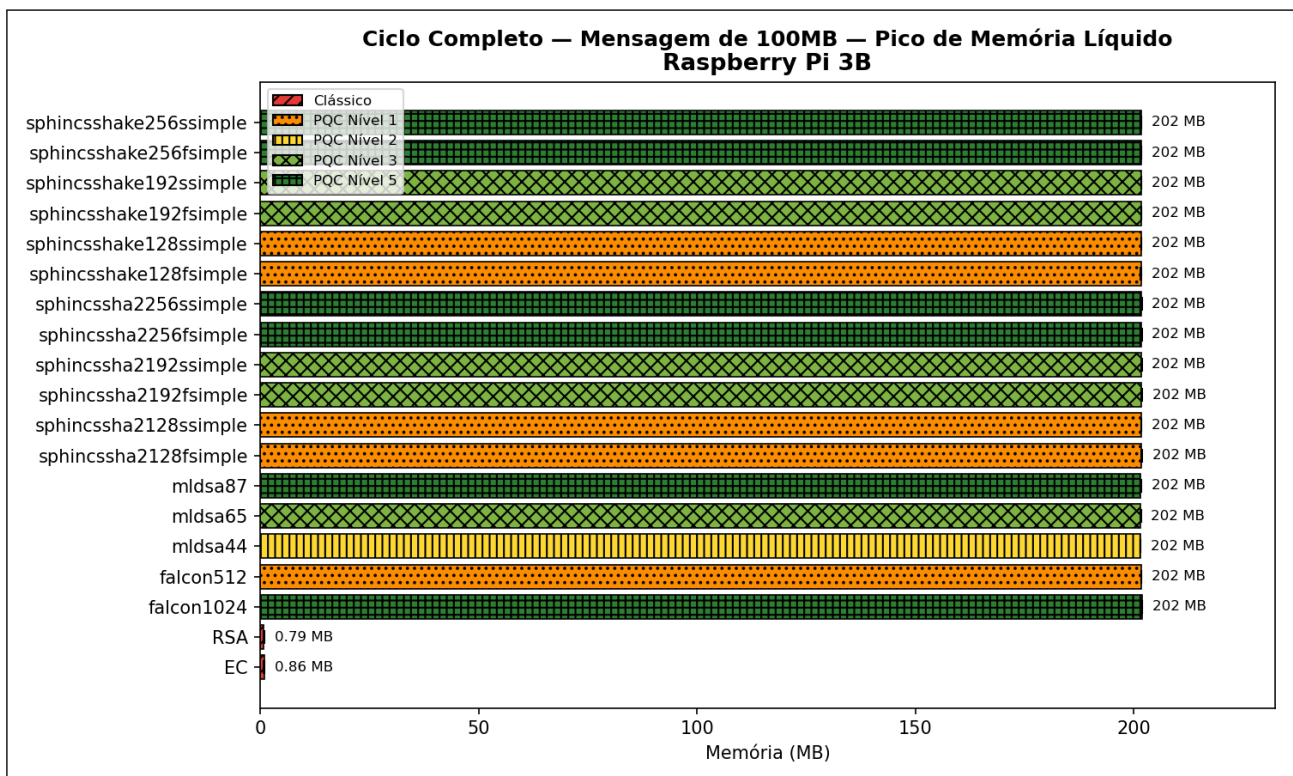
4.2.2.1 Geração de Chaves

A Figura 4.4 apresenta os resultados de tempo de CPU líquido para geração de chaves nas duas plataformas avaliadas. Analisando os esquemas clássicos, o RSA-2048 apresentou o maior tempo de geração de chaves entre todos os algoritmos, com variação extremamente alta entre execuções, possivelmente em função do custo computacional associado à geração de primos de grande porte. O ECDSA, por outro lado, demonstrou desempenho significativamente superior, apresentando o menor tempo de geração entre todos os esquemas avaliados.

Entre os algoritmos pós-quânticos, as três variantes do ML-DSA apresentaram tempos de geração muito baixos e bastante uniformes, praticamente equivalentes ao ECDSA. Já o Falcon apresentou desempenho intermediário entre os algoritmos avaliados. A variante Falcon-512 foi aproximadamente quatro vezes mais lenta que os esquemas mais eficientes, como ML-DSA e ECDSA, enquanto a Falcon-1024 mostrou custo cerca de dez vezes superior, refletindo o aumento proporcional do nível de segurança. Ainda assim, ambas as



(a) Notebook Ubuntu



(b) Raspberry Pi 3B

Figura 4.3: Comparação do pico de memória líquido do ciclo completo com mensagem de 100 MB em ambas plataformas

variantes mantiveram tempos de geração significativamente menores que os observados no RSA e em várias instâncias do SPHINCS+.

No caso do SPHINCS+, as versões *f-simple*, otimizadas para tempo de assinatura, apresentaram desempenho competitivo, situando-se na faixa intermediária entre os algoritmos mais rápidos (ML-DSA e ECDSA) e os Falcon. Com o aumento do nível de segurança, observou-se crescimento gradual no tempo de geração, fazendo com que as variantes mais robustas se aproximassem do desempenho do Falcon.

As versões *s-simple*, otimizadas para tamanho de assinatura, exibiram tempos de geração substancialmente maiores, embora ainda inferiores aos do RSA. Um comportamento curioso foi observado nessa variante: em ambas as famílias SHA2 e SHAKE, os esquemas de nível 256 apresentaram tempos menores que os de nível 192, comportamento contraintuitivo também identificado em estudos anteriores. Esse resultado pode estar relacionado a diferenças específicas de implementação entre as instâncias avaliadas.

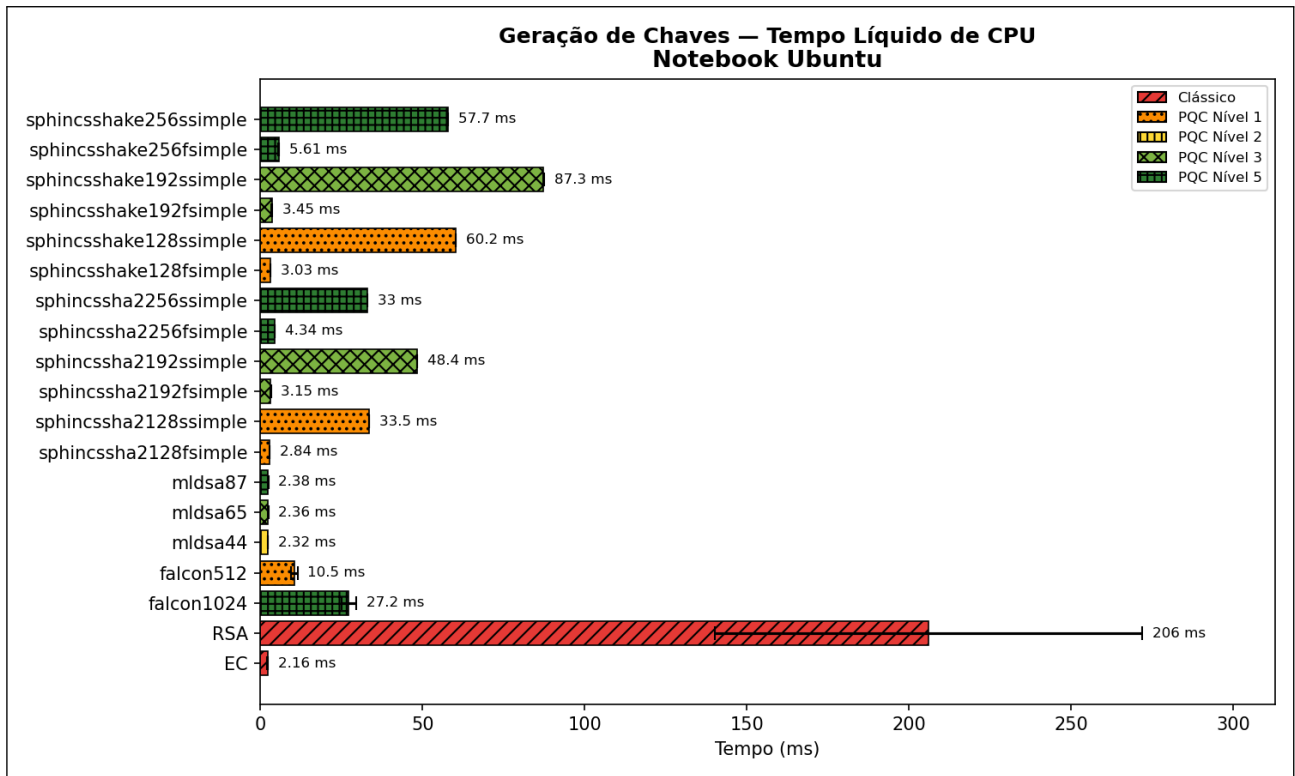
Ao comparar as duas plataformas, nota-se que o Raspberry Pi 3B apresentou tempos cerca de uma ordem de grandeza superiores aos obtidos no Ubuntu, resultado esperado diante das diferenças de capacidade computacional. Apesar disso, os padrões relativos entre algoritmos mantiveram-se consistentes. Um aspecto interessante é que, no Ubuntu, as variantes SHAKE do SPHINCS+ apresentaram maior custo de geração, enquanto no Raspberry Pi os tempos mais altos ocorreram nas variantes SHA2. Esse comportamento sugere que as bibliotecas de *hash* utilizadas possuem otimizações distintas entre as arquiteturas x86-64 e ARM, impactando diretamente o desempenho relativo das famílias de algoritmos.

4.2.2.2 Assinatura

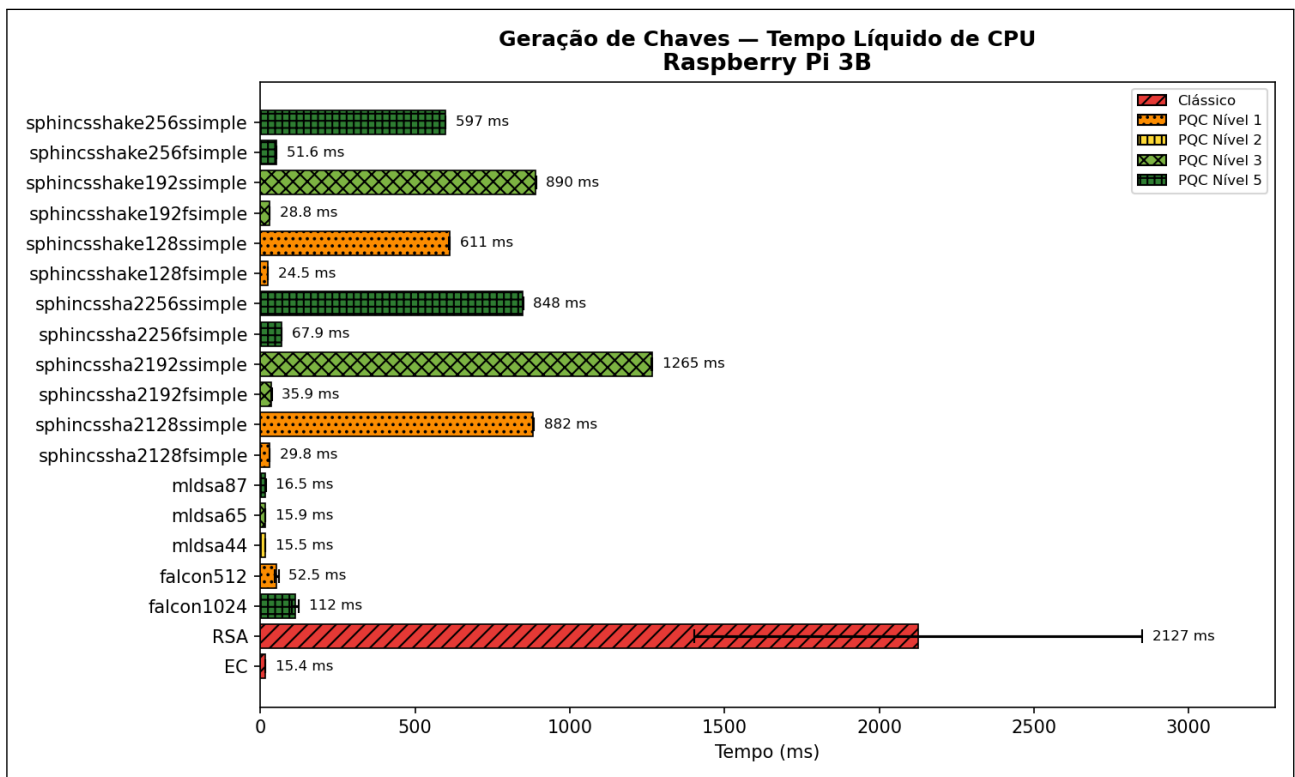
Mensagens de 256 bytes

A Figura 4.5 apresenta os resultados de tempo de CPU líquido para a operação de assinatura com mensagens de 256 bytes. Entre os esquemas clássicos, ambos apresentaram desempenho excepcional. O ECDSA obteve o melhor tempo entre todos os algoritmos, enquanto o RSA, embora aproximadamente duas vezes mais lento, manteve desempenho ótimo, situando-se entre os esquemas mais rápidos do conjunto avaliado.

Entre os algoritmos pós-quânticos, todas as variantes do ML-DSA e do Falcon apresentaram tempos de assinatura extremamente competitivos, próximos aos dos algoritmos clássicos. Ambos os esquemas mantiveram desempenho semelhante entre si, posicionando-se entre o ECDSA e o RSA em termos de tempo médio de execução.



(a) Notebook Ubuntu



(b) Raspberry Pi 3B

Figura 4.4: Comparação do tempo líquido de geração de chaves em ambas plataformas

Por outro lado, os algoritmos da família SPHINCS+ apresentaram tempos de assinatura substancialmente mais altos em comparação com todos os demais esquemas. As variantes *f-simple*, otimizadas para tempo de assinatura, foram dezenas de vezes mais lentas que os algoritmos mais rápidos, com seus tempos aumentando gradualmente conforme o nível de segurança. As variantes *s-simple*, projetadas para economia de tamanho de assinatura, mostraram-se ainda mais custosas, com tempos centenas de vezes superiores aos dos esquemas clássicos e das demais famílias pós-quânticas. Assim como na geração de chaves, observou-se novamente uma inconsistência entre os níveis 192 e 256 nas variantes *s-simple*, em que os esquemas de nível 256 apresentaram tempos ligeiramente menores, padrão que se mantém nas demais análises apresentadas ao longo deste capítulo, com exceção da verificação de assinatura.

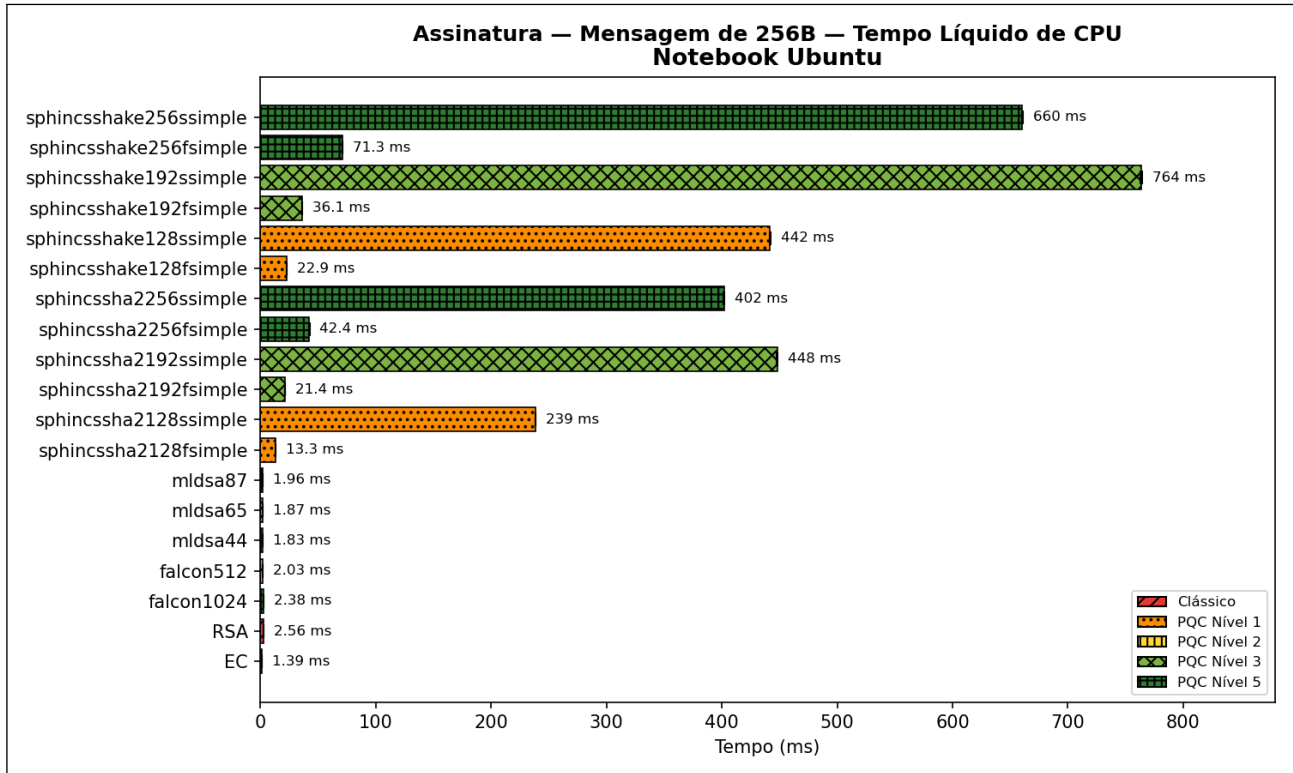
Ao comparar as duas plataformas, os padrões de desempenho mantiveram-se consistentes, com o Raspberry Pi 3B apresentando tempos cerca de uma ordem de grandeza superiores aos do Ubuntu, comportamento esperado. Novamente, as variantes SHAKE do SPHINCS+ apresentaram maior custo de processamento no Ubuntu, enquanto no Raspberry Pi isso ocorreu para as variantes SHA2, tendência também observada de forma consistente nas demais análises, com exceção da verificação de assinatura.

Mensagens de 100 MB

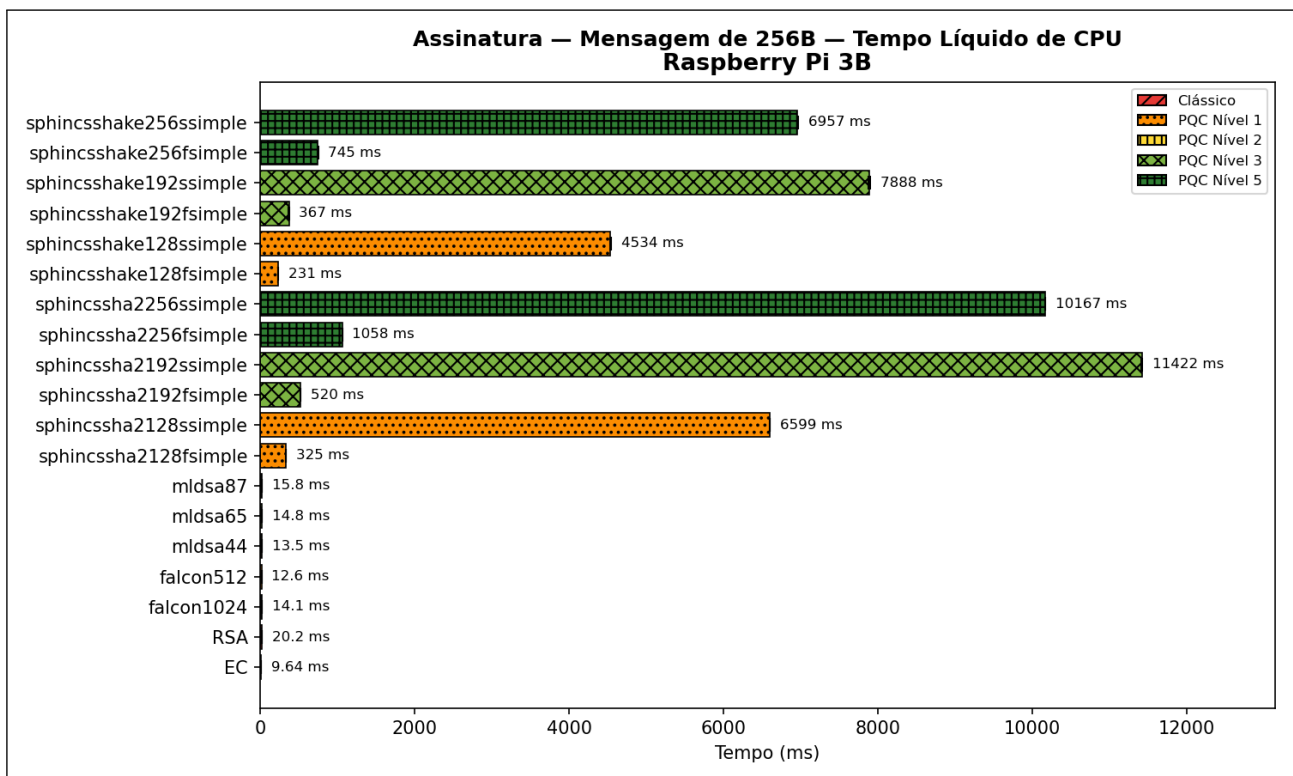
A Figura 4.6 apresenta os resultados de tempo de CPU líquido para a operação de assinatura com mensagens de 100 MB. Entre os esquemas clássicos, tanto RSA quanto ECDSA mantiveram desempenho ótimo, apresentando tempos praticamente equivalentes e ocupando novamente as primeiras posições em ambas as plataformas.

Entre os algoritmos pós-quânticos, ML-DSA e Falcon apresentaram tempos competitivos e equivalentes entre si, independentemente da variante. Contudo, observou-se diferença no comportamento entre as plataformas: no Ubuntu, esses esquemas foram aproximadamente cinco vezes mais lentos que os clássicos, enquanto no Raspberry Pi 3B a diferença foi consideravelmente menor, aproximadamente 1,5 vezes apenas. Ainda assim, em ambos os ambientes, ML-DSA e Falcon mantiveram desempenho superior ao da família SPHINCS+ no geral.

As variantes *f-simple* do SPHINCS+, otimizadas para tempo de assinatura, mostraram comportamento distinto nas duas arquiteturas. No Ubuntu, as versões baseadas em SHA2 apresentaram tempos próximos aos de ML-DSA e Falcon, com destaque para a instância SHA2-128f, que chegou a ser mais rápida, enquanto as versões baseadas em SHAKE foram em torno de duas vezes mais lentas. No Raspberry Pi 3B, tanto as versões SHA2 quanto as SHAKE da variante *f-simple* foram, em média, aproximadamente duas ve-



(a) Notebook Ubuntu



(b) Raspberry Pi 3B

Figura 4.5: Comparação do tempo líquido de assinatura com mensagem de 256 bytes em ambas plataformas

zes mais lentas que ML-DSA e Falcon, mantendo entre si diferenças menores do que as observadas no Ubuntu.

As variantes *s-simple*, otimizadas para tamanho de assinatura, apresentaram tempos de execução significativamente maiores que os de ML-DSA e Falcon. No Ubuntu, as versões baseadas em SHA2 foram cerca de uma a duas vezes mais lentas, enquanto as baseadas em SHAKE alcançaram aproximadamente três vezes o tempo desses esquemas. No Raspberry Pi 3B, o custo foi ainda mais expressivo: tanto as versões SHA2 quanto as SHAKE da variante *s-simple* apresentaram tempos entre cinco e oito vezes superiores aos de ML-DSA e Falcon.

Em ambas as plataformas, os algoritmos clássicos permaneceram como os mais rápidos, seguidos por ML-DSA e Falcon, enquanto SPHINCS+ continuou apresentando os maiores tempos de execução na maioria dos casos. As diferenças entre os algoritmos, no entanto, variam conforme a plataforma: no Ubuntu, a distância entre os clássicos e ML-DSA/Falcon foi significativamente maior, enquanto no Raspberry Pi os tempos de ML-DSA e Falcon ficaram mais próximos dos clássicos. Já o SPHINCS+ manteve desempenho inferior na maioria dos casos, embora algumas variantes baseadas em SHA2 no Ubuntu tenham se aproximado dos tempos de ML-DSA e Falcon. Além disso, a diferença de desempenho entre as variantes *f-simple* e *s-simple* foi menor no Ubuntu do que no Raspberry Pi 3B.

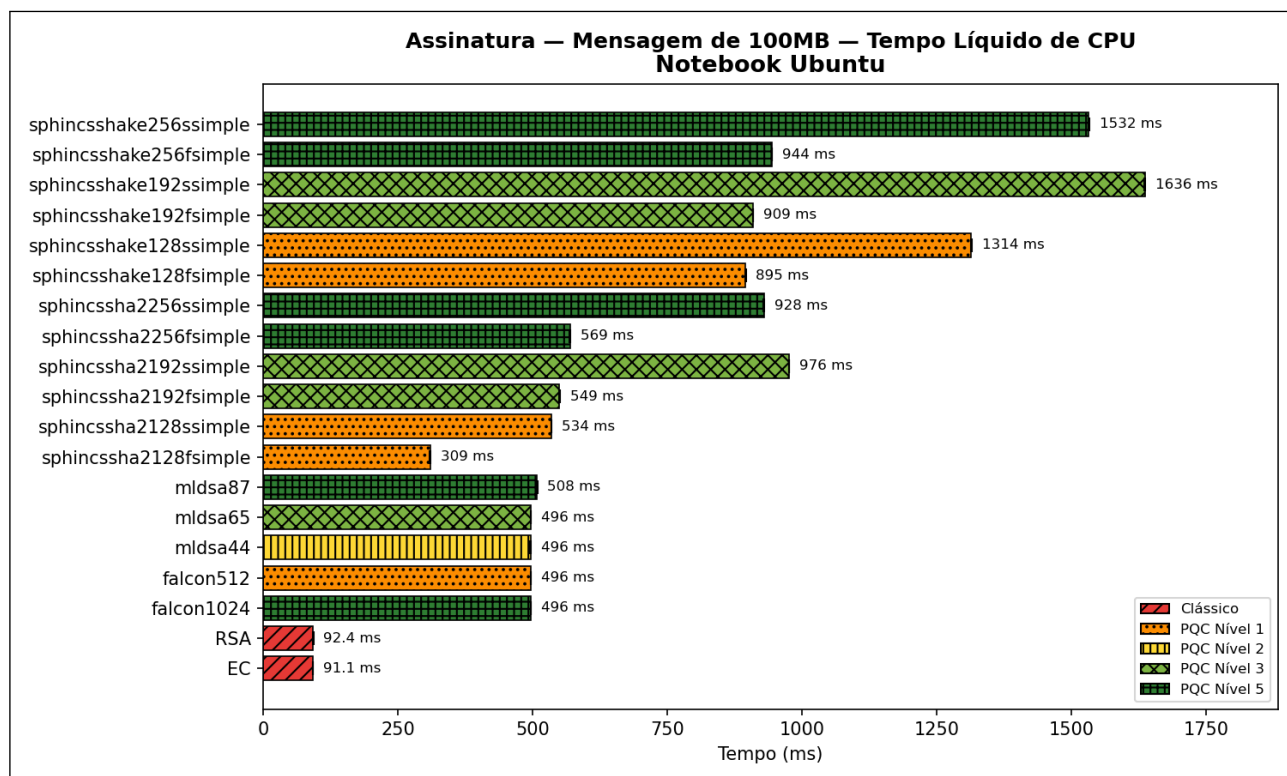
Comparando com os resultados para mensagens de 256 bytes, o aumento do tamanho da mensagem reduziu parcialmente a diferença entre os extremos mais rápidos (clássicos, ML-DSA e Falcon) e SPHINCS+, embora essa disparidade ainda persista em alguns casos. Por outro lado, o aumento das mensagens fez com que o desempenho de ML-DSA e Falcon se distanciasse um pouco dos algoritmos clássicos, principalmente no Ubuntu.

4.2.2.3 Verificação de Assinatura

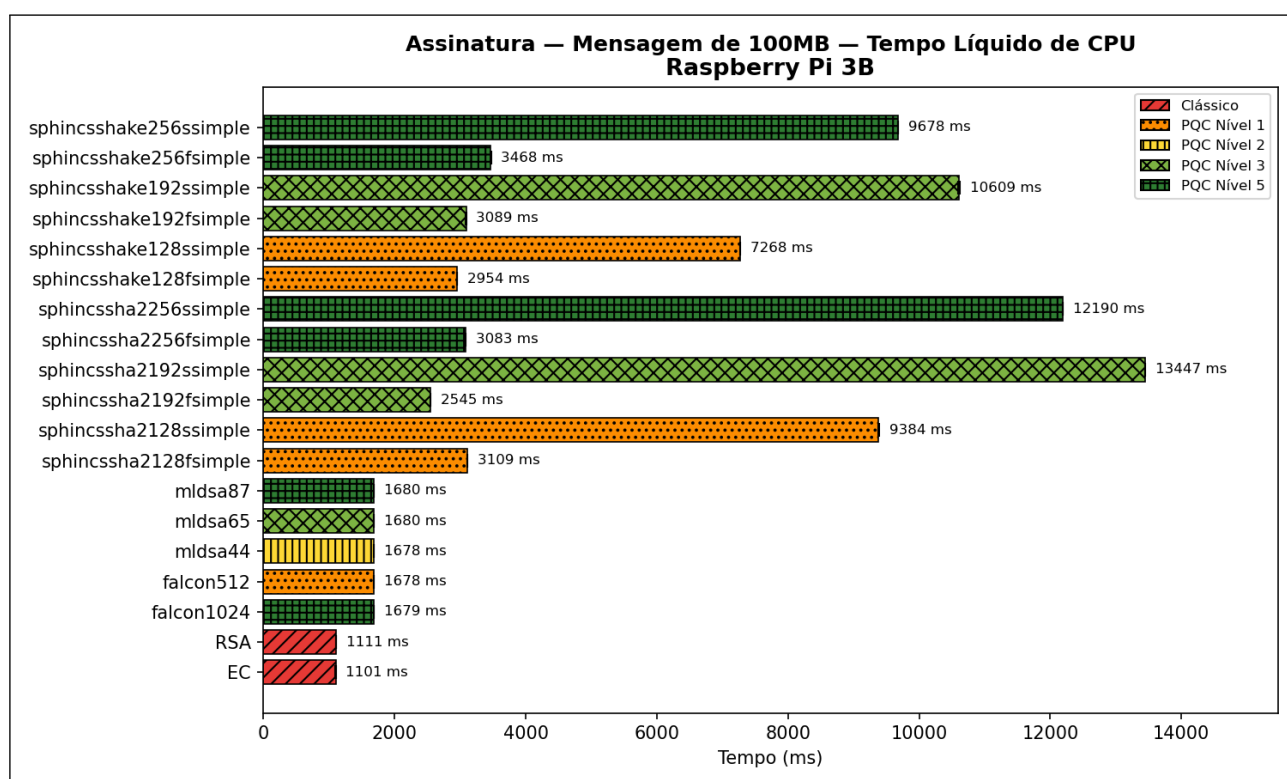
Mensagens de 256 bytes

A Figura 4.7 apresenta os resultados de tempo de CPU líquido para a operação de verificação de assinaturas com mensagens de 256 bytes. Diferentemente das etapas anteriores, em que as variações entre execuções foram praticamente desprezíveis, esta operação apresentou desvios padrão mais altos por ser consideravelmente mais rápida. Ainda assim, os valores permanecem dentro de uma faixa estável e aceitável, sem impacto significativo na análise comparativa.

Entre os esquemas clássicos, ambos os algoritmos mantiveram excelente desempenho, com o RSA apresentando desempenho ligeiramente superior ao do ECDSA nesta



(a) Notebook Ubuntu



(b) Raspberry Pi 3B

Figura 4.6: Comparação do tempo líquido de assinatura com mensagem de 100 MB em ambas plataformas

etapa. Entre os algoritmos pós-quânticos, todas as variantes de ML-DSA e Falcon apresentaram tempos de verificação competitivos e bastante próximos entre si, sendo aproximadamente uma vez e meia mais lentos que os esquemas clássicos.

Por outro lado, SPHINCS+ continuou exibindo tempos consideravelmente maiores em todas as suas variantes. Um comportamento interessante observado nesta operação foi a inversão de tendência: as variantes *f-simple*, otimizadas para reduzir o tempo de assinatura, apresentaram desempenho inferior às *s-simple*, voltadas à redução do tamanho das assinaturas. Isso aparentemente ocorre porque, na verificação, o tamanho da assinatura tem maior influência sobre o tempo total de execução, fazendo com que as versões *f-simple*, que geram assinaturas maiores, apresentem custo mais elevado nessa etapa.

Ao comparar as duas plataformas, o padrão geral manteve-se consistente, com o Raspberry Pi 3B exibindo tempos cerca de uma ordem de grandeza maiores que os do Ubuntu. Uma diferença relevante foi observada, no entanto, quanto às famílias de *hash*: no Ubuntu, as variantes SHA2 e SHAKE apresentaram resultados bastante próximos, com SHA2 sendo levemente mais lento, comportamento inverso ao observado nas etapas de geração e assinatura. No Raspberry Pi, SHA2 manteve-se como a opção mais custosa, mas com diferença ainda mais acentuada em relação ao SHAKE.

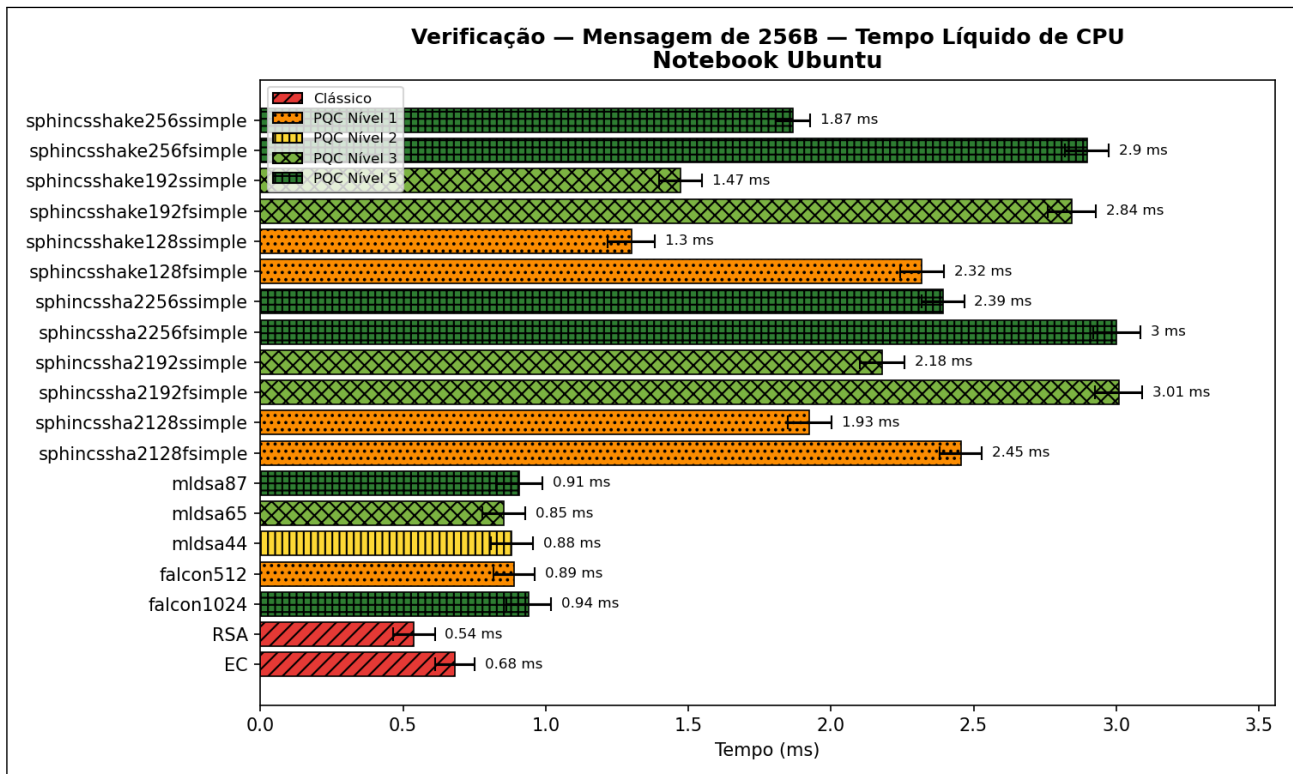
Mensagens de 100 MB

A Figura 4.8 apresenta os resultados de verificação de assinaturas para mensagens de 100 MB. Entre os esquemas clássicos, RSA e ECDSA voltaram a apresentar os melhores tempos, com desempenhos praticamente equivalentes entre si.

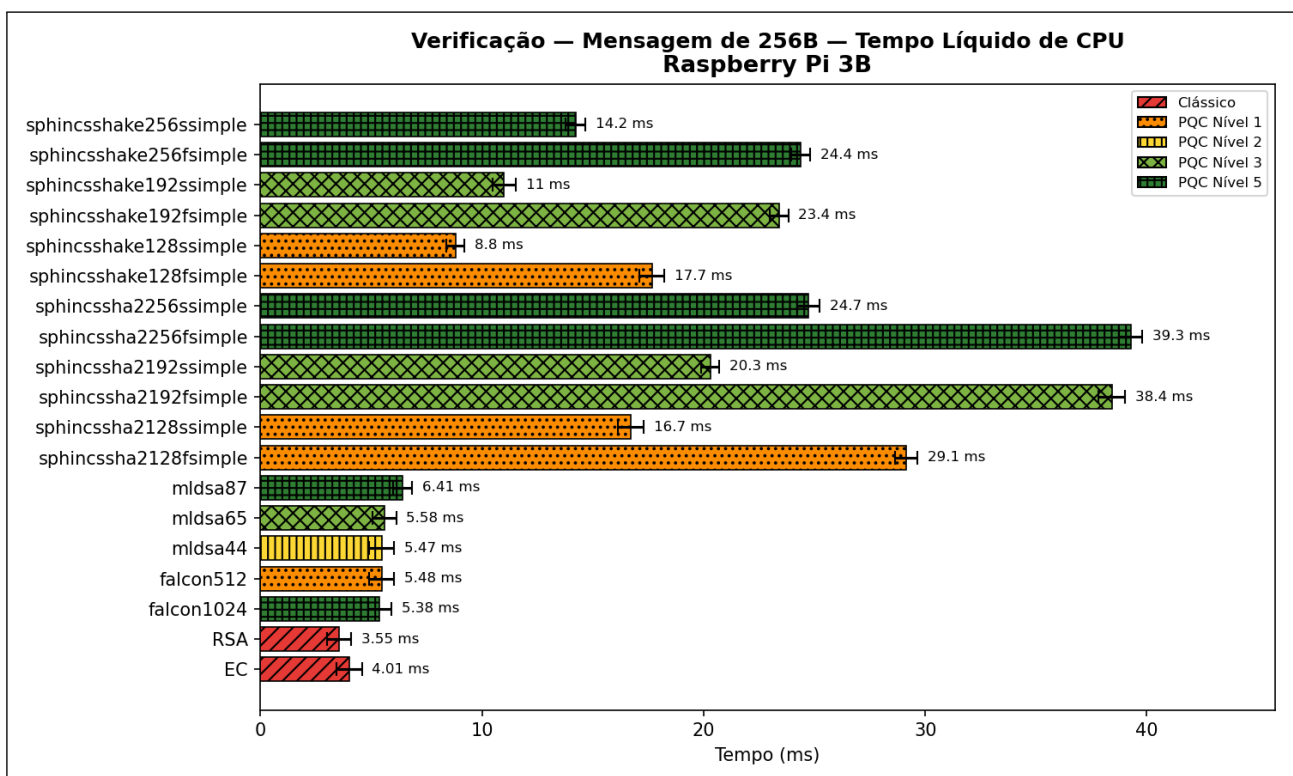
Entre os esquemas pós-quânticos, ML-DSA e Falcon mantiveram tempos muito próximos entre si em todas as variantes, porém situaram-se entre os algoritmos mais lentos nesta configuração, juntamente com todas as variantes SHAKE do SPHINCS+. É importante ressaltar que, mesmo figurando entre os algoritmos mais lentos na verificação de mensagens grandes, ML-DSA e Falcon ainda figuram dentro de uma faixa aceitável de desempenho, especialmente no Raspberry Pi, onde apresentam tempos extremamente competitivos, cerca de uma vez e meia mais lentos que os algoritmos clássicos.

Em contraste, as variantes SHA2-192 e SHA2-256 do SPHINCS+ apresentaram desempenho intermediário, sendo mais rápidas que as variantes SHAKE do SPHINCS+, ML-DSA e Falcon em ambas as plataformas.

Nas instâncias SHA2-128 do SPHINCS+, observou-se diferença de comportamento entre as plataformas: no Ubuntu, essas variantes obtiveram desempenho superior aos de todos os demais esquemas pós-quânticos, ficando atrás apenas dos algoritmos clás-



(a) Notebook Ubuntu



(b) Raspberry Pi 3B

Figura 4.7: Comparação do tempo líquido de verificação com mensagem de 256 bytes em ambas plataformas

sicos. No Raspberry Pi, apresentaram desempenho equivalente ao de ML-DSA, Falcon e SPHINCS+–SHAKE, posicionando-se entre os mais lentos.

Ao comparar as duas plataformas, diferentemente das etapas anteriores, a diferença de desempenho entre Raspberry Pi e Ubuntu não chegou a uma ordem de grandeza. Ainda assim, a distância relativa entre algoritmos clássicos e pós-quânticos permaneceu mais acentuada no Ubuntu, variando entre duas e cinco vezes, enquanto no Raspberry Pi os tempos ficaram mais próximos, com diferença máxima de aproximadamente uma vez e meia.

Comparando com os resultados para mensagens de 256 bytes, o aumento do tamanho da mensagem novamente alterou o comportamento do desempenho dos algoritmos, fazendo com que os algoritmos pós-quânticos se distanciassem um pouco dos clássicos, principalmente no Ubuntu.

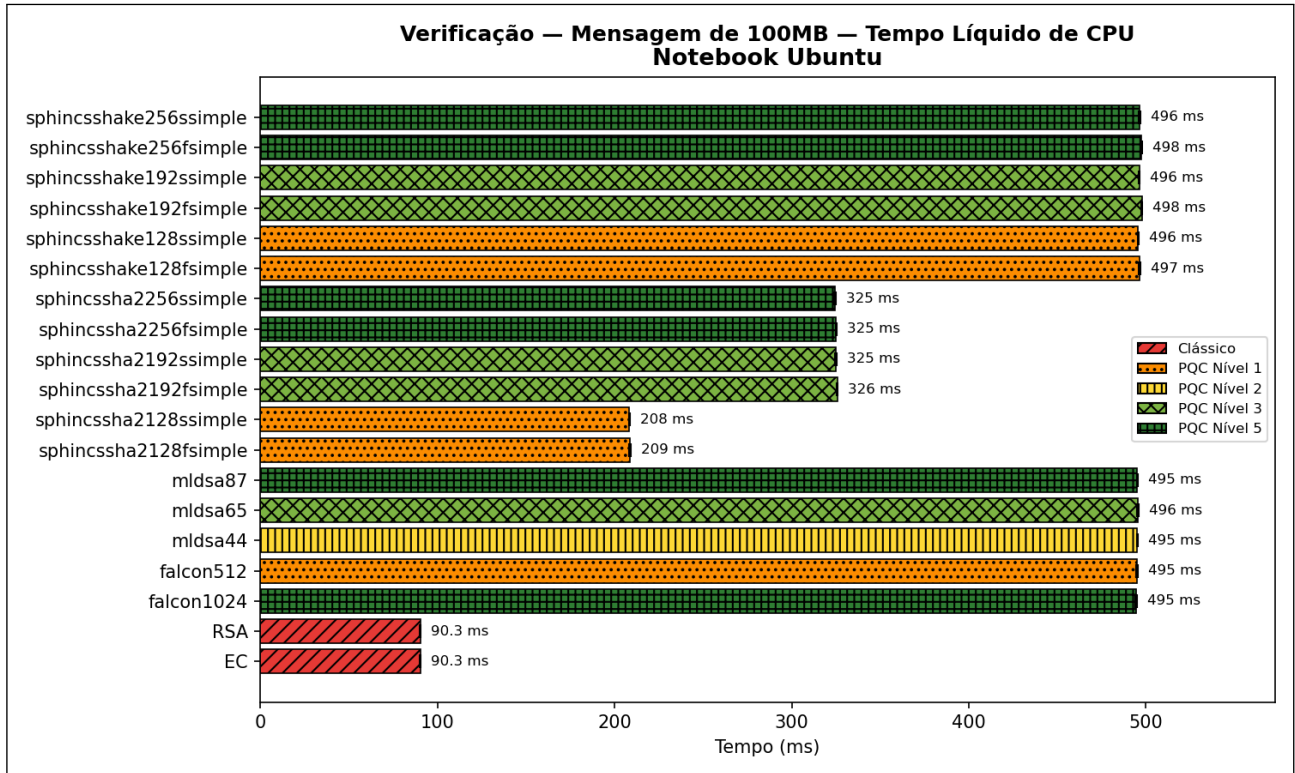
4.2.2.4 Ciclo Completo

Mensagens de 256 bytes

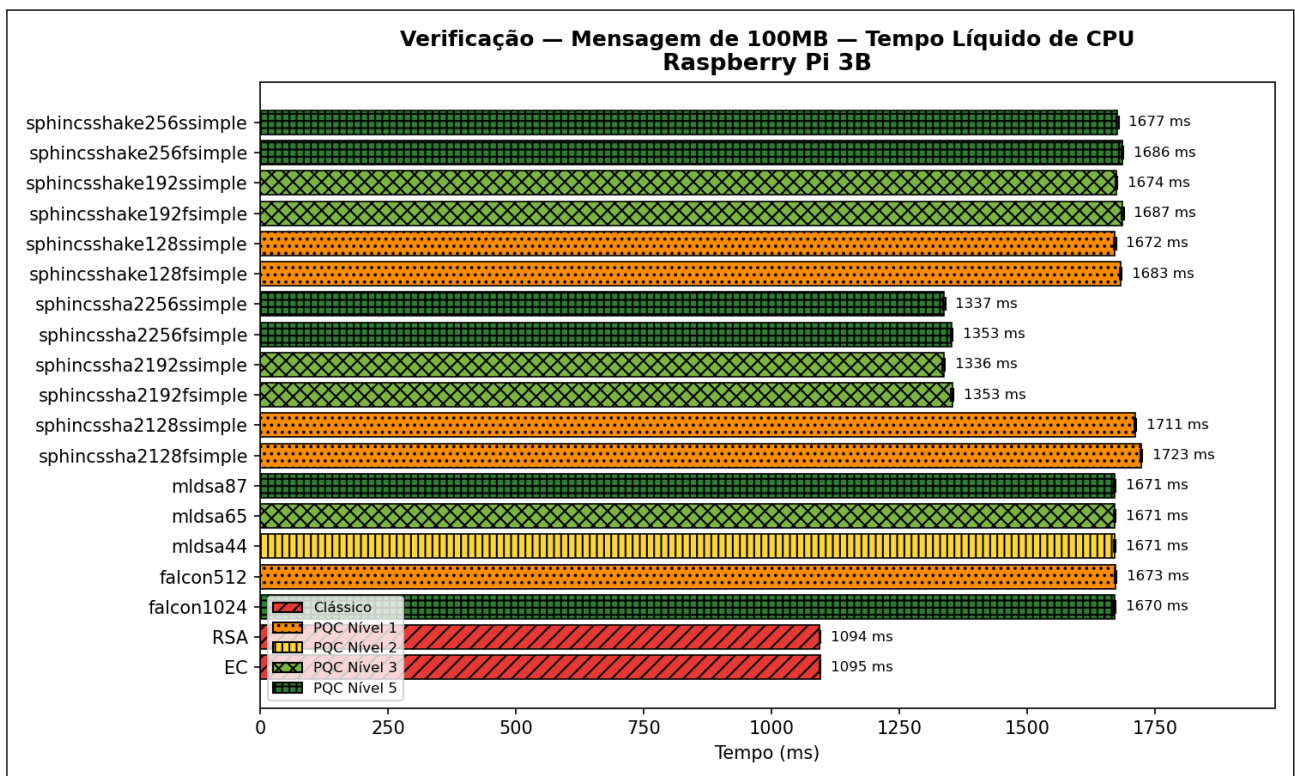
A Figura 4.9 apresenta os resultados do ciclo completo, que representa a execução sequencial das três operações: geração de chaves, assinatura e verificação. Entre os esquemas clássicos, o ECDSA manteve-se como o mais eficiente entre todos os algoritmos, enquanto o RSA apresentou um dos maiores tempos do conjunto, superando inclusive a maioria dos esquemas pós-quânticos, com exceção das variantes *s-simple* do SPHINCS+. Essa diferença é explicada principalmente pelo custo de geração de chaves, operação em que o RSA se mostrou significativamente mais custoso, apesar de seu ótimo desempenho nas etapas de assinatura e verificação.

Entre os algoritmos pós-quânticos, ML-DSA apresentou o melhor desempenho geral, com tempos próximos aos do ECDSA, por volta de 1,5 vezes mais lentos, e desempenho consistentemente equilibrado entre as três operações. O Falcon, que havia demonstrado resultados equivalentes ao ML-DSA nas etapas individuais de assinatura e verificação, apresentou leve aumento no tempo total devido ao custo de geração de chaves, mas ainda manteve-se entre os esquemas mais rápidos do conjunto.

As variantes *f-simple* do SPHINCS+ exibiram desempenho intermediário, sendo substancialmente mais lentas que ML-DSA, Falcon e ECDSA, mas ainda com tempos inferiores aos do RSA e das variantes *s-simple*. Estas últimas, voltadas à redução do tamanho de assinatura, apresentaram novamente os maiores tempos de execução, com desempenho bastante inferior ao dos demais esquemas. Assim como observado nas etapas anteri-



(a) Notebook Ubuntu



(b) Raspberry Pi 3B

Figura 4.8: Comparação do tempo líquido de verificação com mensagem de 100 MB em ambas plataformas

ores, manteve-se o padrão em que as variantes de nível 192 mostraram-se mais lentas que as de nível 256 nas variantes *s-simple*.

A comparação entre plataformas revela comportamento consistente: o Raspberry Pi 3B apresentou tempos cerca de uma ordem de grandeza superiores aos observados no Ubuntu, mantendo as mesmas relações de desempenho entre algoritmos. Também se repetiu o padrão identificado nas demais operações, com as variantes SHAKE sendo mais lentas no Ubuntu e as SHA2 apresentando maiores custos no Raspberry Pi.

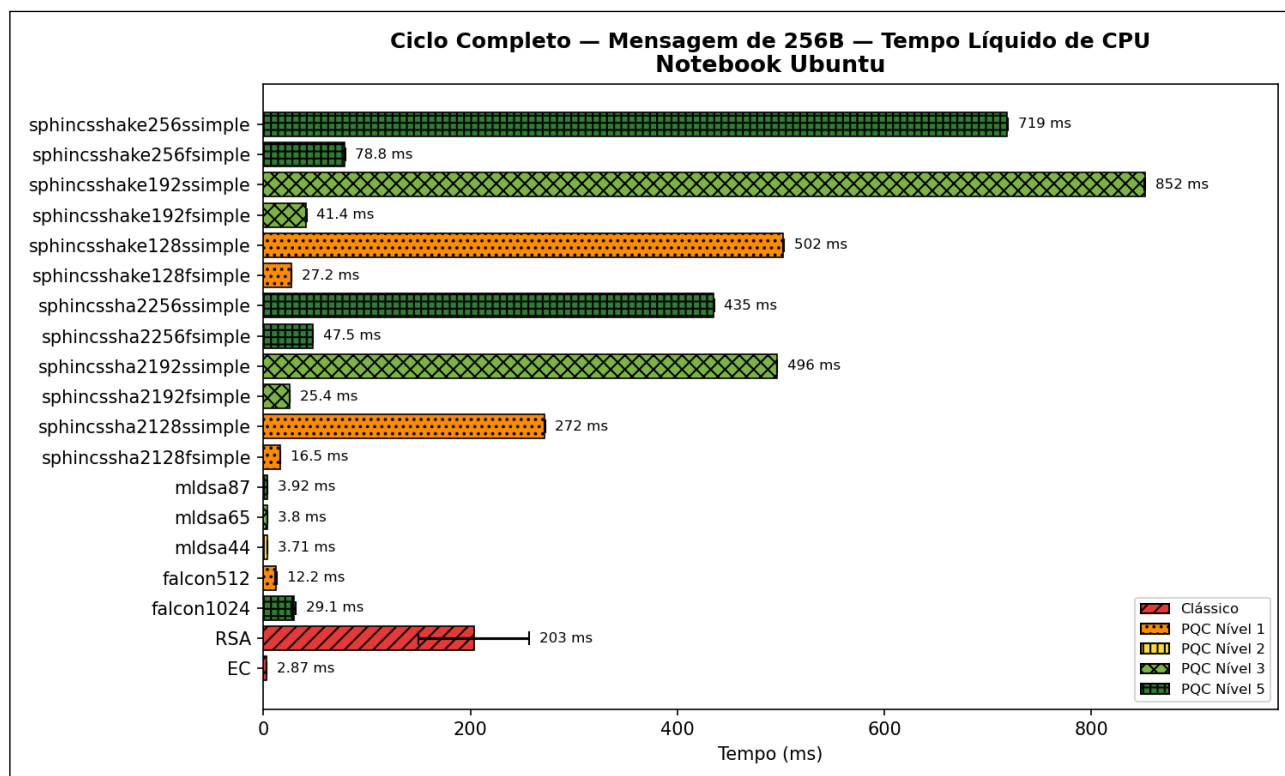
É importante destacar que o tempo total do ciclo completo não reflete diretamente o custo típico de uso dos algoritmos em aplicações reais, já que a geração de chaves é uma operação muito menos frequente que assinatura e verificação. Assim, o tempo total elevado observado no RSA ou a diferença entre Falcon e ML-DSA são amplificados pela inclusão de uma etapa que, na prática, teria impacto menor no desempenho global de um sistema.

Mensagens de 100 MB

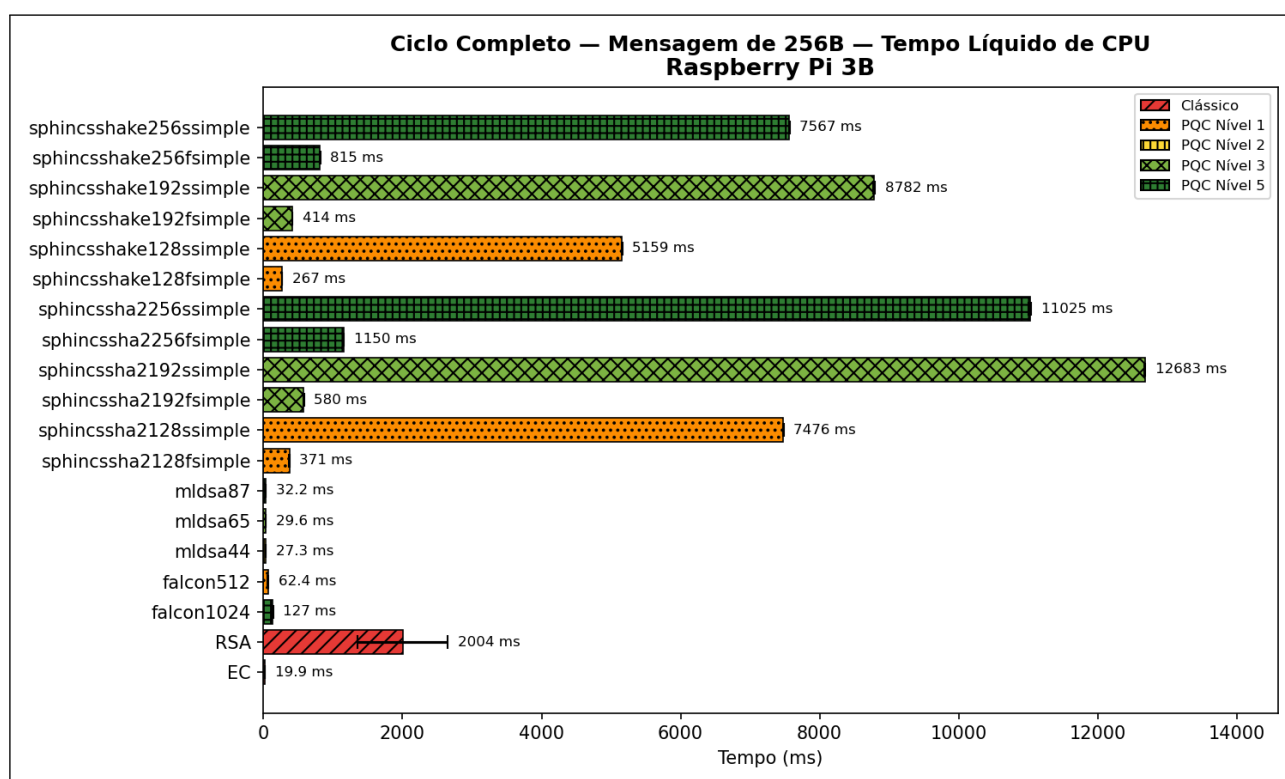
A Figura 4.10 apresenta os resultados do ciclo completo com mensagens de 100 MB. Entre os esquemas clássicos, o ECDSA manteve-se como o mais eficiente entre todos os algoritmos, enquanto o RSA apresentou novamente tempos mais altos quando comparado ao ECDSA, devido ao custo expressivo da operação de geração de chaves. No Ubuntu, RSA ainda figurou como o segundo melhor desempenho geral, mas no Raspberry Pi 3B seu tempo aumentou proporcionalmente mais, posicionando-se entre os algoritmos intermediários, atrás de ML-DSA e Falcon.

Entre os esquemas pós-quânticos, ML-DSA e Falcon mantiveram desempenhos muito próximos entre si. No Ubuntu, ambos apresentaram tempos intermediários, influenciados pelo custo mais alto das operações de assinatura e verificação com mensagens grandes, tendência já observada nos resultados individuais dessas etapas. No Raspberry Pi, no entanto, essas mesmas famílias apresentaram resultados significativamente melhores, ocupando a segunda colocação geral. Esse comportamento está alinhado ao observado na assinatura e verificação de mensagens de 100 MB: no Raspberry Pi seus tempos foram proporcionalmente mais próximos dos esquemas clássicos do que no Ubuntu.

As variantes *f-simple* do SPHINCS+ apresentaram tempos intermediários, geralmente um pouco mais lentos que ML-DSA e Falcon, mas ainda muito inferiores aos das variantes *s-simple*, que novamente exibiram os piores resultados entre todos os algoritmos. Uma exceção interessante foi observada nas versões baseadas em SHA2 no Ubuntu, que alcançaram tempos comparáveis aos de ML-DSA e Falcon, e em alguns casos até inferiores, comportamento não reproduzido no Raspberry Pi. Esse comportamento deve-se ao padrão já apresentado na verificação de mensagens de 100 MB.



(a) Notebook Ubuntu



(b) Raspberry Pi 3B

Figura 4.9: Comparação do tempo líquido do ciclo completo com mensagem de 256 bytes em ambas plataformas

Ao comparar as duas plataformas, os padrões gerais mantiveram-se, mas com algumas diferenças marcantes. No Ubuntu, ML-DSA e Falcon apresentaram tempos relativos consideravelmente mais altos que no Raspberry Pi, enquanto os esquemas SPHINCS+ baseados em SHA2 mostraram tempos relativamente menores. Essas variações refletem o impacto da operação de verificação com mensagens grandes, cujo comportamento divergiu entre as arquiteturas.

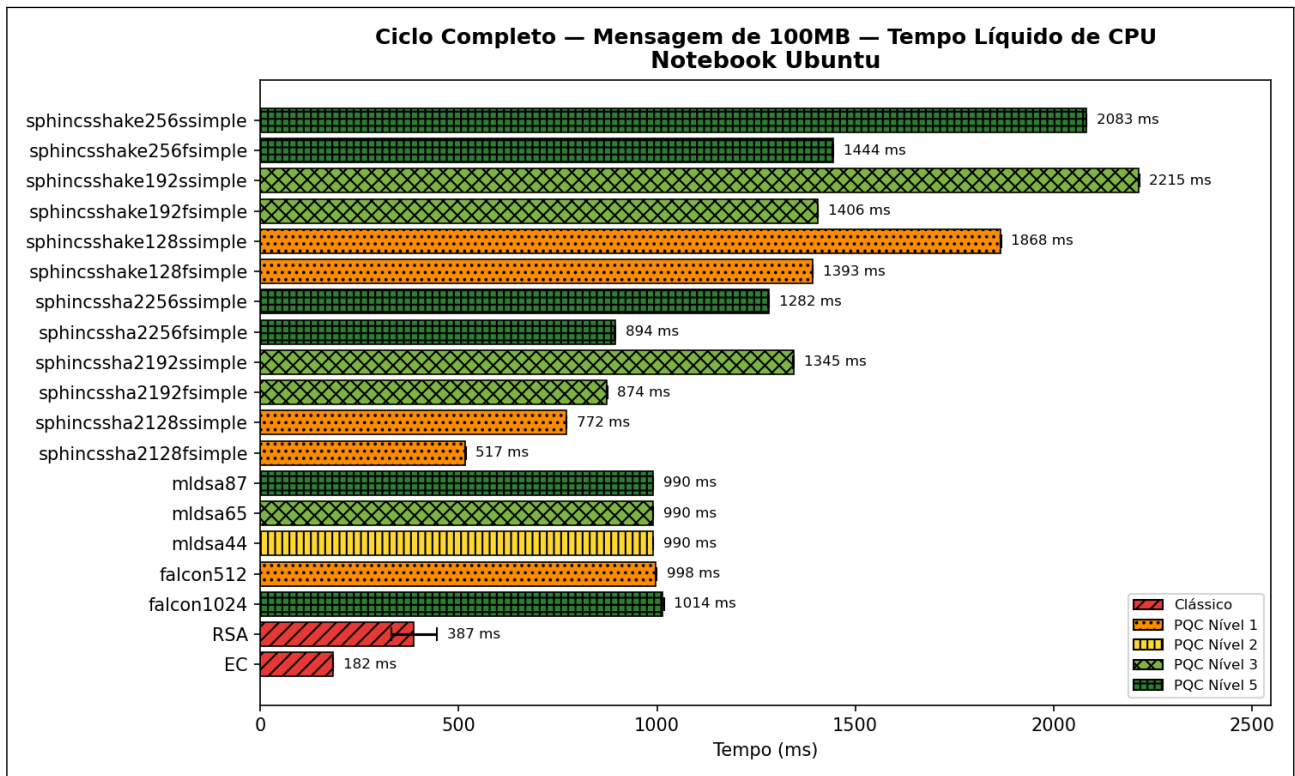
Comparando com os resultados do ciclo completo para mensagens de 256 bytes, o aumento do tamanho da entrada reduziu novamente as diferenças entre os extremos mais rápidos e mais lentos. Por outro lado, o aumento das mensagens fez com que o desempenho de ML-DSA e Falcon se distanciasse um pouco dos algoritmos clássicos no Ubuntu, comportamento também observado nas operações individuais de assinatura e verificação.

4.3 CONSIDERAÇÕES FINAIS

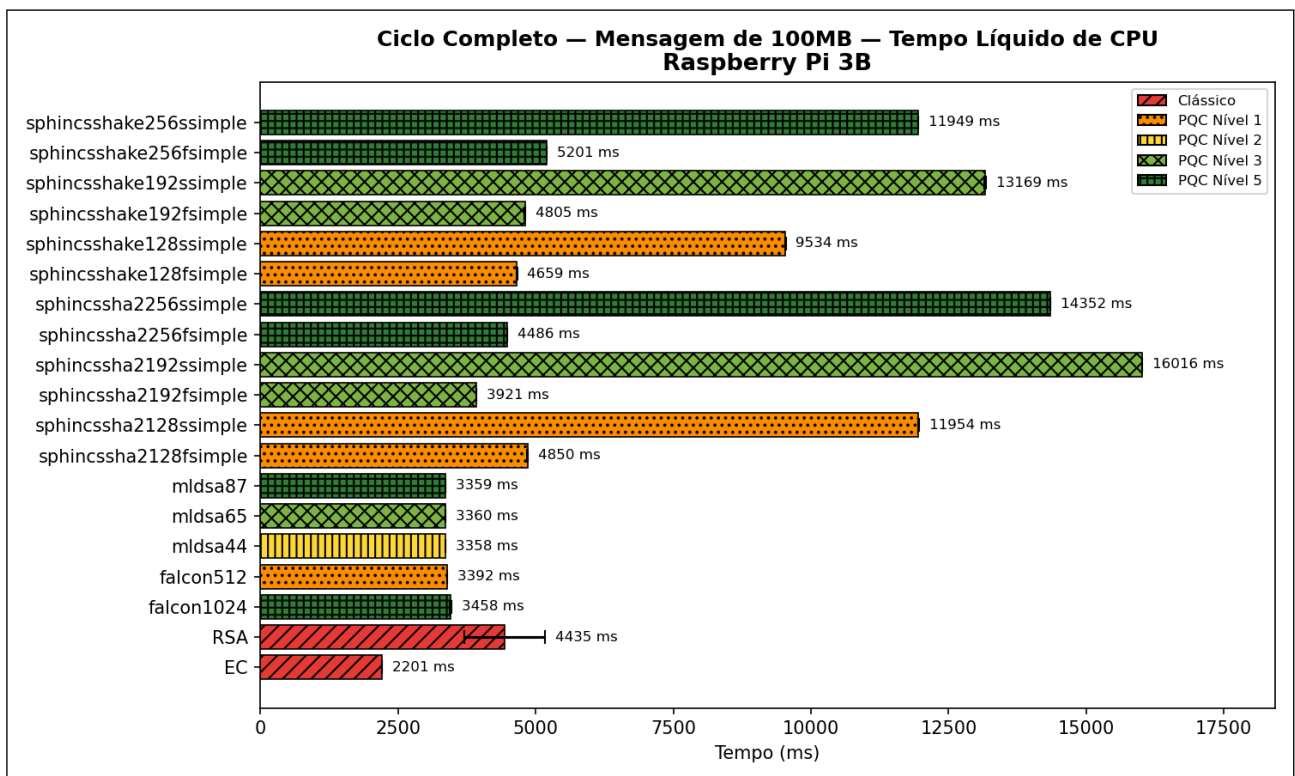
Os experimentos realizados permitiram avaliar comparativamente o desempenho de algoritmos de assinatura digital clássicos e pós-quânticos em duas plataformas com capacidades computacionais distintas. Esta seção apresenta e discute os principais resultados, com uma breve consideração sobre o consumo de memória e foco na análise do tempo de processamento.

Em relação ao consumo de memória, para mensagens pequenas e médias (256 bytes e 100 KB), os algoritmos clássicos apresentaram os menores valores, seguidos de perto pelos pós-quânticos, com todos permanecendo dentro de uma faixa relativamente estreita, entre 1 MB e 2 MB. Entre os pós-quânticos, no Ubuntu, as variantes SPHINCS+ baseadas em SHAKE apresentaram menor consumo médio, enquanto no Raspberry Pi 3B o ML-DSA destacou-se. Para mensagens de 100 MB, contudo, observou-se comportamento distinto: os algoritmos pós-quânticos convergiram para aproximadamente 202 MB, enquanto os clássicos mantiveram consumo próximo de 1 MB. Essa disparidade sugere que as implementações pós-quânticas realizam alocações adicionais durante o processamento de entradas extensas, embora esse comportamento possa estar mais relacionado às diferenças entre as bibliotecas utilizadas (liboqs vs OpenSSL) do que às características intrínsecas dos esquemas. Além disso, as medições de memória apresentaram maior variabilidade entre execuções e resultados pontuais inconsistentes, especialmente nas operações individuais, reduzindo a confiabilidade de análises detalhadas.

Diante dessas limitações e considerando que o consumo de memória mostrou-se relativamente uniforme entre os algoritmos pós-quânticos para mensagens menores, esta discussão concentra-se prioritariamente na métrica de tempo de processamento, que apre-



(a) Notebook Ubuntu



(b) Raspberry Pi 3B

Figura 4.10: Comparação do tempo líquido do ciclo completo com mensagem de 100 MB em ambas plataformas

sentou resultados mais estáveis e permite uma análise comparativa mais robusta. A análise aprofundada do consumo de memória deve ser objeto de estudos futuros, investigando se o comportamento observado decorre de características intrínsecas dos algoritmos ou de estratégias de implementação das bibliotecas criptográficas.

A análise de tempo de processamento revelou diferenças expressivas entre as famílias de algoritmos, variando conforme a operação, o tamanho da mensagem e a plataforma avaliada. Para mensagens pequenas e médias (256 bytes e 100 KB), em ambas as plataformas, o ML-DSA e o Falcon mantiveram tempos muito próximos aos dos algoritmos clássicos, demonstrando serem excelentes alternativas para aplicações com entradas de tamanho moderado. O SPHINCS+, por outro lado, apresentou tempos consideravelmente mais elevados, embora as variantes *f-simple*, otimizadas para velocidade de assinatura, tenham obtido desempenho substancialmente melhor que as variantes *s-simple*, sendo estas últimas voltadas à redução do tamanho das assinaturas.

Com mensagens grandes (100 MB), notou-se um comportamento distinto conforme a plataforma. No Raspberry Pi 3B, o ML-DSA e o Falcon mantiveram desempenho altamente competitivo, com tempos muito próximos aos dos algoritmos clássicos. Já no ambiente Ubuntu, observou-se um aumento mais expressivo no tempo de execução desses algoritmos em relação aos esquemas clássicos, embora ambos ainda tenham apresentado desempenho dentro de limites plenamente aceitáveis, confirmando sua viabilidade mesmo nesse cenário mais exigente. Nesse mesmo contexto de mensagens grandes no Ubuntu, as variantes SPHINCS+–SHA2–*f-simple*, em especial a SHA2-128f, destacaram-se ao apresentar desempenho superior ao de ML-DSA e Falcon, consolidando-se como alternativas para aplicações que processam frequentemente arquivos de grande porte em plataformas de alto desempenho.

A comparação entre as duas plataformas revelou padrões gerais consistentes, com o Raspberry Pi 3B apresentando tempos aproximadamente uma ordem de grandeza superiores aos do Ubuntu. No entanto, para mensagens grandes, a diferença de desempenho entre os algoritmos pós-quânticos e os clássicos foi menor no Raspberry Pi, indicando que a penalidade relativa desses esquemas tende a ser menos acentuada em plataformas com recursos mais limitados. Adicionalmente, as variantes SPHINCS+ baseadas em SHA2 e SHAKE apresentaram comportamento invertido entre as arquiteturas, sugerindo diferenças nas otimizações das bibliotecas de *hash* entre x86-64 e ARM.

Considerando os *trade-offs* entre desempenho e segurança, observa-se que a escolha de variantes com diferentes níveis de segurança impacta de forma distinta cada família de algoritmos. No caso do ML-DSA, que apresenta variantes correspondentes aos níveis de segurança 2, 3 e 5, representadas por ML-DSA-44, ML-DSA-65 e ML-DSA-87, respectivamente, as diferenças de desempenho entre as três configurações foram mínimas em

todas as operações avaliadas. Esse comportamento torna recomendável a adoção da variante de maior segurança, ML-DSA-87, que proporciona proteção equivalente à categoria 5 do NIST sem penalidade considerável de desempenho. Para o Falcon, que disponibiliza variantes nos níveis 1 e 5, representadas por Falcon-512 e Falcon-1024, identificou-se padrão semelhante, com desempenhos próximos nas operações de assinatura e verificação. A exceção ocorre na geração de chaves, onde o Falcon-512 apresentou desempenho superior. Contudo, dado que a geração de chaves é uma operação significativamente menos frequente que assinatura e verificação na maioria das aplicações práticas, a escolha do Falcon-1024 permanece vantajosa quando se prioriza o nível de segurança mais elevado.

A família SPHINCS+, por outro lado, apresenta comportamento mais complexo devido à diversidade de variantes disponíveis, que diferem quanto ao nível de segurança (1, 3 e 5), à estratégia de otimização (*f-simple* para velocidade de assinatura vs *s-simple* para tamanho de assinatura) e à função *hash* utilizada (SHA2 vs SHAKE). As diferenças de desempenho entre essas configurações são substancialmente mais pronunciadas que nas demais famílias. As variantes *f-simple* demonstraram desempenho consistentemente superior às *s-simple*, enquanto o desempenho relativo entre SHA2 e SHAKE mostrou-se dependente da plataforma. Em relação aos níveis de segurança, de modo geral, observou-se um escalonamento gradual do tempo de execução conforme o aumento da categoria. Assim, a escolha da variante apropriada de SPHINCS+ demanda análise cuidadosa dos *trade-offs* envolvidos, considerando as prioridades específicas da aplicação. Quanto ao desempenho, o SPHINCS+ destacou-se em relação às demais famílias pós-quânticas apenas no processamento de mensagens grandes no ambiente Ubuntu, onde as variantes SHA2-*f-simple*, particularmente a SHA2-128f, superaram o ML-DSA e o Falcon. Nesse mesmo cenário, a variante SHA2-128s foi a única configuração *s-simple* a apresentar desempenho competitivo.

A análise realizada oferece uma visão consolidada sobre o comportamento dos algoritmos pós-quânticos nas diferentes plataformas, servindo de base para as considerações finais deste trabalho.

5. CONCLUSÕES E TRABALHOS FUTUROS

Este trabalho apresentou um estudo comparativo entre algoritmos de assinatura digital clássicos e pós-quânticos, com foco na avaliação de seu desempenho em diferentes ambientes computacionais. Inicialmente, foi realizado um estudo teórico abrangendo os fundamentos da criptografia clássica, com ênfase na criptografia assimétrica e nos mecanismos de assinatura digital, abordando seus princípios, funcionamento e principais aplicações. Em seguida, foram discutidos os fundamentos da computação quântica e os desafios que ela impõe aos sistemas criptográficos tradicionais. A partir desse contexto, introduziu-se a criptografia pós-quântica, destacando suas principais famílias, as categorias de segurança definidas pelo NIST e o processo de padronização em curso. Por fim, foram explorados os conceitos de sistemas embarcados, analisando suas características e limitações e como esses fatores influenciam a adoção prática da criptografia pós-quântica.

Para viabilizar a análise comparativa proposta neste trabalho, foi desenvolvida uma ferramenta de *benchmarking* dedicada à avaliação de desempenho de algoritmos de assinatura digital. O sistema implementado abrange tanto esquemas criptográficos clássicos quanto mais de cem variantes de algoritmos de criptografia pós-quântica, permitindo a realização de medições reproduzíveis e comparáveis entre diferentes sistemas. A ferramenta possibilita configurar livremente parâmetros como algoritmo, operação criptográfica, número de repetições e tamanho das mensagens, oferecendo ampla flexibilidade experimental. As métricas coletadas incluem tempo de execução e consumo de memória; os resultados são consolidados em arquivos e gráficos que facilitam a análise comparativa. Além disso, o sistema foi projetado para operar em plataformas com diferentes níveis de capacidade computacional, viabilizando sua utilização tanto em ambientes de uso geral quanto em contextos embarcados.

Os experimentos foram conduzidos em duas plataformas com diferentes capacidades computacionais: um notebook com sistema Ubuntu e arquitetura x86-64, representando um ambiente de alto desempenho, e um Raspberry Pi 3B, representando um sistema embarcado. Foram avaliados os algoritmos ML-DSA, Falcon e SPHINCS+, em suas diversas variantes, além dos esquemas clássicos RSA e ECDSA, considerando métricas de tempo de processamento e consumo de memória nas operações de geração de chaves, assinatura, verificação e ciclo completo. A execução dos testes em diferentes tamanhos de mensagens possibilitou observar o comportamento dos algoritmos sob múltiplas condições de carga, evidenciando seus *trade-offs* de desempenho e sua adequação a distintos contextos de aplicação.

5.1 CONTRIBUIÇÕES

Os experimentos conduzidos contribuíram para um melhor entendimento do desempenho de diferentes algoritmos criptográficos em ambientes computacionais distintos. Os resultados obtidos demonstram que, diante das diretrizes do NIST que estabelecem a descontinuação dos algoritmos RSA e ECDSA para assinaturas digitais, classificados como *deprecated* a partir de 2030 e *disallowed* a partir de 2035 [24], a transição para algoritmos pós-quânticos mostra-se não apenas necessária, mas também tecnicamente viável em termos de tempo de processamento em ambas as plataformas avaliadas.

Entre os algoritmos analisados, o ML-DSA e o Falcon apresentaram desempenho extremamente competitivo em praticamente todos os cenários, funcionando como substitutos diretos dos esquemas clássicos. Ambos mantiveram tempos muito próximos aos dos algoritmos tradicionais para mensagens pequenas e médias em ambas as plataformas, além de resultados altamente satisfatórios para arquivos grandes no Raspberry Pi 3B e plenamente aceitáveis no ambiente Ubuntu. Todas as variantes dessas famílias demonstraram comportamento estável e eficiente, destacando-se em relação aos demais esquemas pós-quânticos. A variante ML-DSA-87, correspondente ao nível de segurança 5 do NIST, mostrou-se a mais recomendável por oferecer o maior nível de proteção com desempenho muito semelhante às versões de menor segurança. O Falcon-1024, também de nível 5, apresentou tempos muito próximos aos do ML-DSA nas operações de assinatura e verificação, ficando ligeiramente atrás devido ao custo mais elevado na geração de chaves. Ainda assim, o Falcon destaca-se pelo tamanho reduzido de suas assinaturas, conforme indicado na Tabela 4.1. Já a família SPHINCS+ apresentou desempenho inferior ao do ML-DSA e do Falcon na maioria dos cenários, mas destacou-se pela versatilidade de suas variantes. As versões SHA2-f-simple, especialmente a SHA2-128f, obtiveram desempenho superior no processamento de arquivos grandes em plataformas de alto desempenho, embora produzam assinaturas significativamente maiores. De modo geral, as diferenças observadas nos tempos de processamento entre os algoritmos pós-quânticos e os esquemas clássicos, embora existentes, são esperadas diante do aumento substancial no nível de segurança que oferecem. Ainda assim, essas diferenças permanecem dentro de margens aceitáveis em ambos os ambientes avaliados, especialmente considerando a proteção proporcionada frente à ameaça quântica iminente.

5.2 OPORTUNIDADES DE TRABALHOS FUTUROS

Em relação ao consumo de memória, os experimentos apresentaram resultados menos consistentes do que os obtidos nas medições de tempo, com maior variabilidade entre execuções e algumas inconsistências pontuais entre plataformas e algoritmos, além de uma disparidade significativa entre os esquemas clássicos e os pós-quânticos para mensagens grandes. Diante dessas observações, um trabalho futuro relevante consiste em realizar novas medições voltadas à análise detalhada do uso de memória, buscando distinguir o impacto de fatores de implementação daqueles efetivamente associados ao *design* dos algoritmos. Essa investigação poderá contribuir para uma compreensão mais precisa do custo de memória das primitivas pós-quânticas, avaliando sua viabilidade e comportamento em diferentes plataformas e condições de execução.

O Raspberry Pi 3B, utilizado neste estudo, representa uma categoria intermediária de sistemas embarcados, com recursos mais limitados do que os de um computador pessoal, mas ainda superiores aos de dispositivos altamente restritos. Sua utilização permitiu avaliar a viabilidade prática dos algoritmos em um ambiente embarcado de capacidade moderada, servindo como ponto de referência para estudos futuros. Como continuidade, seria relevante estender a análise para plataformas ainda mais limitadas, como microcontroladores e dispositivos de Internet das Coisas, a fim de compreender o comportamento dos algoritmos em cenários de recursos críticos.

Além disso, a incorporação de métricas de consumo energético em estudos futuros possibilitaria uma análise mais abrangente da eficiência das soluções pós-quânticas, aspecto fundamental para sua aplicação em dispositivos embarcados e sistemas com restrições de energia.

REFERÊNCIAS BIBLIOGRÁFICAS

- [1] Arakadakis, K.; Charalampidis, P.; Makrogiannakis, A.; Fragkiadakis, A. “Firmware over-the-air programming techniques for IoT networks: A survey”, *ACM Computing Surveys (CSUR)*, vol. 54–9, 2021, pp. 1–36.
- [2] Banks, A. S.; Kisiel, M.; Korsholm, P. “Remote attestation: A literature review”, *arXiv preprint arXiv:2105.02466*, 2021.
- [3] Bavdekar, R.; Jayant Chopde, E.; Agrawal, A.; Bhatia, A.; Tiwari, K. “Post-quantum cryptography: A review of techniques, challenges and standardizations”. In: 2023 International Conference on Information Networking (ICOIN), 2023, pp. 146–151.
- [4] Bernstein, D. J.; Hopwood, D.; Hülsing, A.; Lange, T.; Niederhagen, R.; Papachristodoulou, L.; Schneider, M.; Schwabe, P.; Wilcox-O’Hearn, Z. “SPHINCS: Practical stateless hash-based signatures”. In: *Advances in Cryptology – EUROCRYPT 2015*, Oswald, E.; Fischlin, M. (Editores), 2015, pp. 368–397.
- [5] Bernstein, D. J.; Hülsing, A.; Kölbl, S.; Niederhagen, R.; Rijneveld, J.; Schwabe, P. “The SPHINCS+ signature framework”. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 2129–2146.
- [6] Beyer, D.; Löwe, S.; Wendler, P. “Reliable benchmarking: Requirements and solutions”, *International Journal on Software Tools for Technology Transfer*, vol. 21–1, 2019, pp. 1–29.
- [7] Bürstinghaus-Steinbach, K.; Krauß, C.; Niederhagen, R.; Schneider, M. “Post-quantum TLS on embedded systems: Integrating and evaluating Kyber and SPHINCS+ with mbed TLS”. In: *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*, 2020, pp. 841–852.
- [8] Callou, G.; Maciel, P.; Tavares, E.; Andrade, E.; Nogueira, B.; Araujo, C.; Cunha, P. “Energy consumption and execution time estimation of embedded system applications”, *Microprocessors and Microsystems*, vol. 35–4, 2011, pp. 426–440.
- [9] Cooper, D.; Regenscheid, A.; Souppaya, M.; Bean, C.; Boyle, M.; Cooley, D.; Jenkins, M. “Security considerations for code signing”, *NIST Cybersecurity White Paper*, 2018, vol. 5.
- [10] De Micco, L.; Vargas, F. L.; Fierens, P. I. “A literature review on embedded systems”, *IEEE Latin America Transactions*, vol. 18–02, 2020, pp. 188–205.

- [11] Ducas, L.; Kiltz, E.; Lepoint, T.; Lyubashevsky, V.; Schwabe, P.; Seiler, G.; Stehlé, D. “Crystals-dilithium: A lattice-based digital signature scheme”, *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018, pp. 238–268.
- [12] Fang, W.; Chen, W.; Zhang, W.; Pei, J.; Gao, W.; Wang, G. “Digital signature scheme for information non-repudiation in blockchain: A state of the art review”, *EURASIP Journal on Wireless Communications and Networking*, vol. 2020–1, 2020, pp. 56.
- [13] Fournaris, A. P.; Tasopoulos, G.; Brohet, M.; Regazzoni, F. “Running longer to slim down: Post-quantum cryptography on memory-constrained devices”. In: 2023 IEEE International Conference on Omni-layer Intelligent Systems (COINS), 2023, pp. 1–6.
- [14] Friesen, M.; Karthikeyan, G.; Heiss, S.; Wisniewski, L.; Trsek, H. “A comparative evaluation of security mechanisms in DDS, TLS and DTLS”, 2018.
- [15] Kapoor, V.; Abraham, V. S.; Singh, R. “Elliptic curve cryptography”, *Ubiquity*, vol. 2008–May, Maio 2008.
- [16] Kim, Y.; Song, J.; Seo, S. C. “Accelerating Falcon on ARMv8”, *IEEE Access*, vol. 10, 2022, pp. 44446–44460.
- [17] Kumar, V. B. Y.; Gupta, N.; Chattopadhyay, A.; Kasper, M.; Krauß, C.; Niederhagen, R. “Post-quantum secure boot”. In: 2020 Design, Automation & Test in Europe Conference & Exhibition (DATE), 2020, pp. 1582–1585.
- [18] Liu, T.; Ramachandran, G.; Jurdak, R. “Post-quantum cryptography for internet of things: A survey on performance and optimization”, *arXiv preprint arXiv:2401.17538*, 2024.
- [19] Lopez, J.; Cadena, V.; Rahman, M. S. “Evaluating post-quantum cryptographic algorithms on resource-constrained devices”, *arXiv preprint arXiv:2507.08312*, 2025.
- [20] Madhu, G. C.; Vijayakumar, P.; Gao, X. Z. “Resource constrained IoT environments: A survey”, *Journal of Advanced Research in Dynamical and Control Systems*, vol. 9–Special Issue 16, 2017, pp. 445–457.
- [21] Madsen, J.; Bjørn-Jørgensen, P. “Embedded system synthesis under memory constraints”. In: Proceedings of the Seventh International Workshop on Hardware/Software Codesign, 1999, pp. 188–192.
- [22] Mason, S. “Electronic signatures in law”. University of London Press, 2016.
- [23] Milanov, E. “The RSA algorithm”. Capturado em: http://susanka.org/MathPhysics2/RSA_Algorithm_Yevgeny.pdf, 2009.

- [24] Moody, D.; Perlner, R.; Regenscheid, A.; Robinson, A.; Cooper, D. “Transition to post-quantum cryptography standards”, Relatório Técnico NIST IR 8547 ipd, National Institute of Standards and Technology, 2024.
- [25] Mosca, M. “Quantum algorithms”. 0808.0369, Capturado em: <https://arxiv.org/abs/0808.0369>, 2008.
- [26] National Institute of Standards and Technology. “Nist announces first four quantum-resistant cryptographic algorithms”. Capturado em: <https://www.nist.gov/news-events/news/2022/07/nist-announces-first-four-quantum-resistant-cryptographic-algorithms>, Jul 2022.
- [27] National Institute of Standards and Technology. “About NIST”. Acesso em: 19 maio 2025, Capturado em: <https://www.nist.gov/about-nist>, 2023.
- [28] National Institute of Standards and Technology. “FIPS 186-5: Digital signature standard (DSS)”, Relatório Técnico, U.S. Department of Commerce, 2023.
- [29] National Institute of Standards and Technology. “FIPS 204: Module-lattice-based digital signature standard”, Relatório Técnico, U.S. Department of Commerce, 2024.
- [30] Oonishi, K.; Kunihiro, N. “Shor’s algorithm using efficient approximate quantum Fourier transform”, *IEEE Transactions on Quantum Engineering*, vol. 4, 2023, pp. 1–16.
- [31] Open Quantum Safe Project. “liboqs: C library for quantum-resistant cryptography”. Acesso em: 02 set. 2025, Capturado em: <https://github.com/open-quantum-safe/liboqs>, 2025.
- [32] Open Quantum Safe Project. “oqs-provider: OpenSSL 3 provider for liboqs algorithms”. Acesso em: 02 set. 2025, Capturado em: <https://github.com/open-quantum-safe/oqs-provider>, 2025.
- [33] OpenSSL Software Foundation. “OpenSSL: Cryptography and SSL/TLS Toolkit”, 2025, versão 3.x. Acesso em: 02 set. 2025, Capturado em: <https://www.openssl.org/>.
- [34] Opitka, F.; Niemiec, M.; Gagliardi, M.; Kourtis, M. A. “Performance analysis of post-quantum cryptography algorithms for digital signature”, *Applied Sciences*, vol. 14–12, 2024.
- [35] Putranto, D. S. C.; Wardhani, R. W.; Larasati, H. T.; Ji, J.; Kim, H. “Depth-optimization of quantum cryptanalysis on binary elliptic curves”, *IEEE Access*, vol. 11, 2023, pp. 45083–45097.
- [36] Qadir, A. M.; Varol, N. “A review paper on cryptography”. In: 2019 7th International Symposium on Digital Forensics and Security (ISDFS), 2019, pp. 1–6.

- [37] Quan, Y. “Improving Bitcoin’s post-quantum transaction efficiency with a novel lattice-based aggregate signature scheme based on CRYSTALS-Dilithium and a STARK protocol”, *IEEE Access*, vol. 10, 2022, pp. 132472–132482.
- [38] Rietsche, R.; Dremel, C.; Bosch, S.; Steinacker, L.; Meckel, M.; Leimeister, J. M. “Quantum computing”, *Electronic Markets*, vol. 32, Ago 2022, pp. 2525–2536.
- [39] Samie, F.; Bauer, L.; Henkel, J. “IoT technologies for embedded computing: A survey”. In: Proceedings of the Eleventh IEEE/ACM/IFIP International Conference on Hardware/Software Codesign and System Synthesis, 2016, pp. 1–10.
- [40] Seo, S. C.; An, S. W.; Choi, D. “Accelerating Falcon post-quantum digital signature algorithm on graphic processing units”, *Computers, Materials and Continua*, vol. 75–1, 2023, pp. 1963–1980.
- [41] Shaaban, M. A.; Alsharkawy, A. S.; AbouKreisha, M. T.; Razek, M. A. “Efficient ECC-based authentication scheme for fog-based IoT environment”, *arXiv preprint arXiv:2408.02826*, 2024.
- [42] Shafique, M. A.; Munir, A.; Latif, I. “Quantum computing: Circuits, algorithms, and applications”, *IEEE Access*, vol. 12, 2024, pp. 22296–22314.
- [43] Shibu, K. V. “Introduction to embedded systems”. Tata McGraw-Hill Education, 2009.
- [44] Smith, B. “Pre- and post-quantum Diffie-Hellman from groups, actions, and isogenies”. 1809.04803, Capturado em: <https://arxiv.org/abs/1809.04803>, 2019.
- [45] Sosy-Lab. “BenchExec: A framework for reliable benchmarking and resource measurement”. Capturado em: <https://github.com/sosy-lab/benchexec>.
- [46] Stallings, W. “Criptografia e segurança de redes: Princípios e práticas”. Pearson Brasil, 2014.